

Programiranje za UNIX

Damir Krstinić

Maja Braović

travanj 2026.

Sadržaj

Programiranje za UNIX	3
Preduvjeti	3
Struktura	3
P01-Osnove_UNIXa/	3
P02-Osnove_programiranja/	3
P03-Ulazno_izlazne_operacije/	4
P04-Upravljanje_datotekama/	4
P05-Okruzenje_procesa/	4
P06-Signali/	4
Prevođenje i pokretanje	4
Licenca	4
Osnove UNIX-a	5
Arhitektura UNIX operacijskog sustava	5
Sustav datoteka	7
Tipovi datoteka	7
Struktura direktorija	7
Prava pristupa	8
Naredbena ljuska	9
Osnovne naredbe	9
Pokretanje programa u pozadini	10
Priručnik — naredba man	10
Preusmjeravanje ulaza i izlaza	10
Ulančavanje naredbi (pipes)	11
UNIX procesi	12
Shell skripte	12
Najjednostavnija skripta	13
Petlje, varijable i argumenti	13
Praktičan primjer — backup direktorija	14
Pretraga datoteka — find, grep, brojanje	15
Što dalje?	17
Osnove programiranja	18
Najjednostavniji program	18
Program u više datoteka	18
Prevođenje	19
Ručno prevođenje u jednom koraku	19
Ručno prevođenje u dva koraka	19
Ručno prevođenje programa iz više datoteka	19
Korištenje alata make	20
Pokretanje	23
Ulazno/izlazne operacije	24
Deskriptori datoteka	24
Sistemske pozive za rad s datotekama	25
Funkcija open()	25

Funkcija <code>creat()</code>	26
Funkcija <code>close()</code>	26
Funkcija <code>read()</code>	27
Funkcija <code>write()</code>	27
Primjeri	28
Otvaranje, čitanje i pisanje	28
Pozicioniranje unutar datoteke (<code>lseek</code>)	29
Prava pristupa i maska (<code>umask</code>)	32
Argumenti naredbenog retka	33
Dijeljenje datoteka i preusmjerenje (<code>dup</code> , <code>dup2</code>)	35
Prevođenje	39
Upravljanje datotekama	41
Tipovi datoteka	41
Svojstva datoteka	41
Struktura <code>struct stat</code>	42
Otkrivanje tipa datoteke	44
Ispis atributa datoteke	44
Razlika <code>stat</code> i <code>lstat</code>	46
Korisnici, grupe i prava pristupa	47
Procesi i njihovo vlasništvo	47
Provjera prava pristupa — <code>access()</code>	48
Promjena prava pristupa — <code>chmod()</code> i <code>fchmod()</code>	50
Promjena vlasništva — <code>chown()</code> , <code>fchown()</code> , <code>lchown()</code>	51
Linkovi	52
Čvrsti linkovi	52
Simbolički linkovi	54
Vremena pristupa	56
Rad s direktorijima	58
Čitanje sadržaja direktorija	60
Što smo zapravo radili	64
Prevođenje	64
Okruženje procesa	65
Argumenti naredbenog retka	65
Varijable okruženja	68
Životni ciklus procesa	72
Pokretanje novog programa	77
Ograničavanje resursa (<code>setrlimit</code>)	83
Osirotljeli procesi (engl. <i>orphan processes</i>)	84
Prevođenje	85
Signali	86
Najčešći signali	86
Hvatanje signala	88
Vlastiti alarm — <code>SIGALRM</code> i <code>alarm()</code>	90
Korištenje signala za komunikaciju između procesa	94
Sistemske pozive <code>sigaction</code>	96
Blokiranje i ignoriranje signala	99
Sistemske pozive <code>sigprocmask</code>	99
Pokupljanje djece — <code>SIGCHLD</code>	102
Prevođenje	104

Programiranje za UNIX

Skripta s primjerima koda iz kolegija **Programiranje za UNIX** (Damir Krstinić, Maja Braović, travanj 2026.), Sveučilište u Splitu, Fakultet elektrotehnike, strojarstva i brodogradnje (FESB).

Skripta je organizirana po poglavljima koja postupno uvode čitatelja u ključne koncepte UNIX sustava — od osnova rada u ljusci, preko C programiranja i sistemskih poziva, do procesa i signala. Svaki primjer ilustrira specifičnu funkcionalnost (ljuska, sistemski pozivi, rad s procesima, datotečni sustav, signali) i predviđen je za prevođenje standardnim gcc-om na proizvoljnom POSIX-kompatibilnom sustavu. Svaki direktorij ima vlastiti README s detaljnim opisom primjera i teorijskim uvodom u temu poglavlja.

Preduvjeti

- POSIX-kompatibilan sustav (Linux, macOS, WSL na Windowsu, BSD varijante)
- gcc (ili drugi C prevodilac koji razumije GNU sintaksu Makefilea)
- make
- bash (za primjere shell skripti u P01)

Struktura

Svaki direktorij odgovara jednom poglavlju skripte:

Direktorij	Poglavlje
P01-Osnove_UNIXa/	Osnove UNIX-a
P02-Osnove_programiranja/	Osnove programiranja u C-u
P03-Ulazno_izlazne_operacije/	Ulazno/izlazne operacije
P04-Upravljanje_datotekama/	Upravljanje datotekama
P05-Okruženje_procesa/	Okruženje procesa
P06-Signali/	Signali

P01-Osnove_UNIXa/

Uvodno poglavlje koje upoznaje čitatelja s arhitekturom UNIX operacijskog sustava, organizacijom datotečnog sustava, pravima pristupa, naredbenom ljuskom, preusmjeravanjem ulaza i izlaza, ulančavanjem naredbi te osnovama pisanja shell skripti. Sadrži četiri **bash skripte** koje ilustriraju ključne koncepte: `pozdrav.sh`, `brojac.sh`, `backup.sh`, `trazi.sh`.

P02-Osnove_programiranja/

Uvod u proces prevođenja i povezivanja C programa s primjerima. Demonstrira osnovnu strukturu C programa, uporabu zaglavlja i razlaganje koda na više prevodbenih jedinica. Detaljno se obrađuje korištenje alata `make` — od ručnog prevođenja preko jednostavnih pravila do potpunog `Makefilea` s varijablama, implicitnim pravilima i konvencionalnim pseudo-pravilima `default`, `all` i `clean`.

P03-Ulazno_izlazne_operacije/

Primjeri koji ilustriraju UNIX systemske pozive za rad s datotekama (open, creat, close, read, write, lseek, umask, dup, dup2) i temeljni UNIX koncept “sve je datoteka” — datoteke, uređaji, terminali, cijevi i mrežni socketi koriste se kroz isto sučelje file deskriptora. Obraduje se i dijeljenje datoteka među procesima i unutar istog procesa, s mehanizmom preusmjeravanja standardnih tokova.

P04-Upravljanje_datotekama/

Upravljanje datotekama na razini datotečnog sustava: rad s metapodacima, prava pristupa, vlasništvo, vremenski žigovi, simboličke i tvrde veze, te navigacija direktorijima.

P05-Okruženje_procesa/

Primjeri koji obrađuju životni ciklus UNIX procesa: argumente naredbenog retka, varijable okruženja, stvaranje novih procesa pozivom fork(), pokretanje programa funkcijama iz exec obitelji, čekanje završetka djece pomoću wait(), ograničavanje resursa kroz setrlimit(), te problematiku zombi i osirotjelih procesa.

P06-Signali/

Primjeri koji obrađuju signale — UNIX-ov primarni mehanizam za asinkronu komunikaciju s procesom. Pokrivaju hvatanje signala (signal, sigaction), korištenje SIGALRM za vlastite alarme, signalnu komunikaciju između procesa, blokiranje i ignoriranje signala, te pokupljanje djece preko SIGCHLD rukovatelja.

Prevođenje i pokretanje

Svaki direktorij od P02 nadalje sadrži vlastiti Makefile s istim konvencijama (varijable CC, CFLAGS, LDFLAGS, TARGETS; implicitno pravilo .c.o; pseudo-pravila default, all, clean). Pozicionirajte se u željeni direktorij i pokrenite:

```
make all      # prevede sve primjere iz direktorija
make clean    # briše generirane izvršne i objektne datoteke
```

Za prevođenje pojedinačnog primjera dovoljno je:

```
make <ime_primjera>
```

Konkretnu upute za pokretanje pojedinačnih primjera (uključujući očekivane argumente i izlaz) nalaze se u README datoteci svakog poglavlja.

P01 ne sadrži C kod nego shell skripte; one se pokreću izravno nakon dodjeljivanja prava izvršavanja:

```
chmod +x *.sh
./pozdrav.sh
```

Licenca

Kod je objavljen pod licencom **GNU General Public License v3.0**. Detalji su u datoteci **LICENSE**.

Osnove UNIX-a

U ovom poglavlju dan je uvod u UNIX operacijski sustav: njegovu arhitekturu, organizaciju datotečnog sustava, prava pristupa, naredbenu ljusku, preusmjeravanje ulaza i izlaza, ulančavanje naredbi, te pisanje shell skripti. Nije riječ o C programiranju — sve što slijedi odvija se u **terminalu**, izravnom interakcijom korisnika s ljuskom. Razumijevanje ovih osnova preduvjet je za sve sljedeće poglavlje, gdje ćemo se baviti programiranjem aplikacija koje se izvršavaju na UNIX sustavima.

Arhitektura UNIX operacijskog sustava

UNIX je organiziran u nekoliko slojeva apstrakcije, prikazanih na sljedećoj slici:

U središtu se nalazi **jezgra** (engl. *kernel*) — najniži sloj softvera koji izravno upravlja sklopovljem računala (procesorom, memorijom, diskovima, mrežnim sučeljima, ulazno-izlaznim uređajima). Jezgra je jedini dio sustava koji ima privilegirani pristup hardveru; sve ostale komponente — uključujući vlastite uvodne programe poput ljuske — moraju s hardverom komunicirati posredno, kroz nju.

Komunikacija s jezgrom odvija se putem **sistemskih poziva** (engl. *system calls*) — strogo definiranog skupa funkcija koje korisnički programi pozivaju kad žele zatražiti uslugu od jezgre. Sistemski pozivi su jedino sučelje između korisničkih programa i jezgre. Na taj način programer ne mora poznavati detalje implementacije sklopovlja, nego radi protiv unificiranog i stabilnog sučelja.

Iznad sistemskih poziva nalaze se **biblioteke funkcija** (engl. *libraries*, *libovi*) — zbirke funkcija više razine koje predstavljaju dodatnu razinu apstrakcije. Najpoznatiji primjer je standardna C biblioteka (*libc*) koja sadrži funkcije poput *printf*, *fopen*, *malloc* i mnoge druge. Funkcije iz biblioteka u konačnici se svode na jedan ili više sistemskih poziva, ali programeru pružaju znatno jednostavnije sučelje.

Naredbena ljuska (engl. *shell*) je interpreter naredbenog retka koji korisniku omogućuje interaktivan pristup svim funkcijama sustava. Bitno je naglasiti da ljuska **nije dio jezgre** — ona je obični korisnički program koji koristi sistemske pozive i biblioteke kao i svaki drugi program. UNIX ljuska istovremeno je i kompletan programski jezik: korisnik može pisati skripte koje uključuju varijable, uvjetno grananje, petlje i funkcije, sve koristeći isključivo ljusku. Jedna od temeljnih prednosti je **filozofija ulančavanja** — izlaz jedne naredbe može postati ulaz druge, što omogućuje izgradnju složenih lanaca obrade podataka od jednostavnih, specijaliziranih programa. Ova ideja sažeta je u UNIX maksimi: *mali programi koji rade jednu stvar dobro mogu se kombinirati u moćne lance obrade*.

Najviši sloj čine **aplikacije** — svi korisnički programi: tekstualni editori, web preglednici, baze podataka, inženjerski alati. Sve operacije ostvaruju pozivanjem funkcija iz biblioteka ili izravnim sistemskim pozivima.

Iako je na slici prikazan kao zaseban sloj, **datotečni sustav** zapravo prožima cijelu arhitekturu. Jezgra ga implementira i izlaže putem sistemskih poziva. UNIX-ova filozofija “*sve je datoteka*” znači da se i uređaji, međuprocena komunikacija i mnogi drugi resursi sustava prikazuju kao datoteke — što znatno pojednostavljuje programiranje jer se sve resursi koriste kroz isto sučelje.



Slika 1: Arhitektura UNIX operacijskog sustava

Sustav datoteka

Tipovi datoteka

UNIX razlikuje nekoliko tipova datoteka. Tip se može vidjeti prvim znakom u ispisu naredbe `ls -l`:

Oznaka	Tip	Opis
-	obična datoteka	bilo koji podaci na disku — tekst, binarni programi, slike, arhive
d	direktorij	popis zapisa o datotekama i poddirektorijima
l	simbolički link	datoteka koja sadrži putanju na drugu datoteku ("prečac")
c	character special	uređaj kojemu pristupamo znak po znak (/dev/tty)
b	block special	uređaj kojem pristupamo blok po blok (/dev/sda)
s	socket	krajnja točka mrežne ili lokalne komunikacije
p	imenovani cjevovod (FIFO)	jednosmjerni komunikacijski kanal između procesa

Ključna posljedica filozofije "*sve je datoteka*": svi ovi tipovi resursa koriste se istim malim skupom sistemskih poziva — `open()`, `read()`, `write()`, `close()`. Program ne treba znati radi li o običnoj datoteci na disku, terminalu, mrežnom socketu ili imenovanom cjevovodu — sa svima radi na isti način.

Struktura direktorija

UNIX datotečni sustav organiziran je kao stablo: jedan korijenski direktorij označen znakom `/` na vrhu, ispod njega proizvoljno mnogo razina poddirektorija. Svaki direktorij u trenutku stvaranja automatski dobiva dva posebna unosa koji se ne mogu izbrisati:

- `.` — pokazivač na **trenutni** direktorij,
- `..` — pokazivač na **roditeljski** direktorij (jednu razinu više). Iznimka je korijenski direktorij `/`, gdje `.` i `..` pokazuju na sam `/`.

Pri navigaciji datotečnim sustavom razlikujemo apsolutnu i relativnu putanju.

Apsolutna putanja uvijek počinje znakom `/` i jednoznačno određuje položaj datoteke neovisno o tome gdje se trenutno nalazimo. Primjer:

```
/home/dkrst/nastava/unix
```

Ova putanja označava isti direktorij neovisno o korisnikovoj trenutnoj lokaciji. Možemo je zamisliti kao kompletnu kućnu adresu — bez obzira odakle u svijetu napišemo adresu, razglednica će stići pravoj osobi.

Relativna putanja polazi od trenutnog radnog direktorija i ne počinje s `/`. Možemo je zamisliti kao uputu kako stići do određenog mjesta u gradu: "*na prvom križanju lijevo, pa dva ravno...*" — vrijedi samo ako polazimo s točno određene lokacije. Tako, ako se nalazimo u `/home/dkrst`, putanja `nastava/unix` označava `/home/dkrst/nastava/unix`. Ako se nalazimo u `/home/dkrst/vjezbe`, do istog cilja stižemo putanjom `../nastava/unix` — `..` nas vraća jedan korak gore.

Trenutni radni direktorij ispisujemo naredbom `pwd` (*print working directory*), a mijenjamo naredbom `cd` (*change directory*).

Prava pristupa

UNIX je višekorisnički sustav, što znači da na istom računalu istovremeno može raditi više korisnika. Da različitih korisnici ne bi međusobno smetali — slučajno ili namjerno mijenjajući tuđe datoteke — uveden je **sustav prava pristupa**.

Prava se dodjeljuju u tri razine, prema skupini korisnika:

- **user** — vlasnik datoteke (obično onaj tko ju je stvorio),
- **group** — grupa kojoj datoteka pripada (npr. zaposlenici istog odjela),
- **others** — svi ostali korisnici sustava.

Za svaku skupinu definirana su tri prava:

Oznaka	Naziv	Značenje za datoteke
r	čitanje (<i>read</i>)	dopušta otvaranje i čitanje sadržaja
w	pisanje (<i>write</i>)	dopušta izmjenu sadržaja datoteke
x	izvršavanje (<i>execute</i>)	dopušta pokretanje datoteke kao programa

Dakle, svaka datoteka ima ukupno **devet bitova** prava — tri prava puta tri skupine. Ovih devet bitova pri ispisu naredbom `ls -l` izgleda ovako:

-rw-r--r--
user group others

Slika 2: Prikaz prava rwx

Primjer: prava `rw-r--r--` znače da vlasnik može čitati i pisati, a grupa i ostali samo čitati. Prava `rw-r-x---` znače da vlasnik ima sva tri prava, grupa može čitati i izvršavati, a ostali nemaju nikakva prava.

Posebnu pažnju zaslužuje pravo **izvršavanja (x)**. Korisnicima koji dolaze s Windowsa ovaj koncept može biti neobičan: na Windowsima sustav prepoznaje izvršne datoteke po **ekstenziji** (`.exe`, `.bat`, `.com`). Na UNIX-u taj mehanizam ne postoji — ekstenzija je samo dio imena i ne nosi nikakvo posebno značenje za sustav. Umjesto toga, UNIX koristi bit **x** kao jedini i dovoljan kriterij: ako je postavljen, datoteka se može pokrenuti. Kao posljedica:

- datoteka `program.txt` može biti potpuno valjani izvršni program (binarni ili shell skripta) ako joj je postavljen bit **x**,
- datoteka `program.exe` na UNIX-u nije ništa posebno — izvršit će se samo ako joj je postavljen bit **x**.

Naravno, davanje prava izvršavanja ne čini bilo koju datoteku stvarno izvršnom: ako sadržaj nije ni binarni kod ni interpretirajuća skripta, pri pokušaju izvršavanja dobit ćemo grešku.

Za **direktorije**, prava **r**, **w**, **x** imaju nešto izmijenjeno značenje:

- **r** — pravo pregleda popisa datoteka (`ls`),
- **w** — pravo izmjene popisa: stvaranja, brisanja, preimenovanja datoteka **u** direktoriju,
- **x** — pravo “ulaska” u direktorij, tj. pristupa datotekama u njemu (uključujući mogućnost da direktorij bude radni direktorij).

Bitno je razlikovati pravo brisanja datoteke od prava pisanja na samu datoteku: brisanje datoteke kontrolira se pravom **w** na **direktoriju** u kojem se ona nalazi, ne na samoj datoteci.

Promjena prava pristupa

Prava pristupa mijenjamo naredbom `chmod`. Postoje dva načina zadavanja:

Numerički (apsolutni) način — svako pravo predstavlja jedan bit s težinom: $r=4$, $w=2$, $x=1$. Zbrajanjem dobivamo brojku za svaku skupinu, a tri brojke (vlasnik, grupa, ostali) zapisujemo zajedno:

```
chmod 644 dat1.txt
```

Postavlja `rw-` za vlasnika ($6 = 4+2$), `r--` za grupu (4) i `r--` za ostale (4) — što odgovara zapisu `rw-r--r--`. Apsolutni način uvijek **u potpunosti** zamjenjuje prethodna prava.

Simbolički način — koristi slova za skupine (`u`, `g`, `o`, ili `a` za sve) i operatore `+` (dodaj), `-` (oduzmi):

```
chmod u+x skripta.sh      # daj vlasniku pravo izvršavanja
chmod go+rx skripta.sh    # daj grupi i ostalima čitanje i izvršavanje
chmod a-x dat1.txt        # oduzmi izvršavanje svima
```

Simbolički način **mijenja samo navedena prava**, ostala ostaju netaknuta. Ovo je razlika u odnosu na apsolutni način, gdje uvijek navodimo cjelokupna prava odjednom.

Pored običnih korisnika postoji i poseban korisnik s neograničenim ovlastima — **administrator** ili **superuser**, u UNIX terminologiji nazvan **root**. Za razliku od običnih korisnika, `root` može čitati, mijenjati i brisati bilo koju datoteku te pokretati i zaustavljati bilo koji proces. Upravo zbog toga administratorski račun treba koristiti s velikim oprezom — greška izvršena s `root` ovlastima može nepovratno oštetiti sustav.

Riječ “`root`” u UNIX kontekstu ima dva potpuno različita značenja: **administratorski korisnički račun** (s matičnim direktorijem `/root`) i **korijski direktorij** datotečnog stabla (`/`). Ova dva pojma nemaju nikakve veze osim što dijele isti engleski naziv — početnicima ova podudarnost zna stvarati zabunu, ali iskusni korisnici iz konteksta uvijek znaju o čemu se radi.

Naredbena ljuska

Ljuska je program koji čita naredbe sa standardnog ulaza (`stdin`), interpretira ih, izvršava i ispisuje rezultat na standardni izlaz (`stdout`), a poruke o greškama na standardni izlaz za greške (`stderr`).

Osnovne naredbe

Naredba	Opis	Primjer
<code>pwd</code>	ispisuje apsolutnu putanju trenutnog direktorija	<code>pwd</code>
<code>cd</code>	mijenja trenutni direktorij; bez argumenta vraća u matični (<code>\$HOME</code>)	<code>cd /home/dkrst</code> , <code>cd ..</code> , <code>cd</code>
<code>ls</code>	ispisuje sadržaj direktorija (<code>-l</code> dugi format, <code>-a</code> skrivene, <code>-t</code> po vremenu)	<code>ls -la</code>
<code>mkdir</code>	stvara novi direktorij (<code>-p</code> stvara i sve roditelje)	<code>mkdir projekti</code> , <code>mkdir -p a/b/c</code>
<code>cp</code>	kopira datoteke (<code>-r</code> rekurzivno)	<code>cp dat1.txt dat2.txt</code> , <code>cp -r dir1/ dir2/</code>

Naredba	Opis	Primjer
mv	premješta ili preimenovava	mv dat1.txt arhiva/, mv staro.txt novo.txt
rm	briše datoteke (-r rekurzivno, -f prisilno bez upita)	rm dat1.txt, rm -rf stari_dir/
cat	ispisuje sadržaj jedne ili više datoteka na standardni izlaz	cat dat1.txt, cat *.log
man	otvara priručnik za zadanu naredbu ili sistemski poziv	man ls

Upozorenje za rm: UNIX nema “koš za smeće” — brisanje je trajno. Posebno opasna kombinacija je `rm -rf` koja briše rekurzivno bez ikakvog upita. Naredba `rm -rf /` (ili neka njezina slučajna varijacija s razmacima) može obrisati cijeli sustav. Uvijek dvaput provjerite što ste utipkali prije nego pritisnete Enter.

Pokretanje programa u pozadini

Po zadanom, kad pokrenemo neki program iz ljsuke, ona čeka da on završi prije nego nam vrati kontrolu. To je u redu za kratke naredbe, ali za one koje traju dugo (kompresiranje velikog direktorija, kompilacija velikog projekta, mrežna preuzimanja) često želimo nastaviti raditi paralelno. UNIX to omogućuje znakom `&` na kraju naredbe — program se pokreće “u pozadini”, a ljsuka odmah vraća kontrolu:

```
$ tar -czf backup.tar.gz /home/dkrst &
[1] 3963
$
```

Ljsuka ispisuje **redni broj posla** u uglatim zagrada ([1]) i **PID** (Process ID) procesa (3963), te odmah pokazuje promptom da je spremna za novu naredbu. Status pokrenutih poslova provjeravamo naredbom `jobs`, premještamo ih iz pozadine u prvi plan s `fg`, a iz prvog plana u pozadinu s `bg` (uz prethodno suspendiranje pomoću `Ctrl+Z`).

Priručnik — naredba man

`man` je tradicionalni UNIX alat za dokumentaciju, dostupan izravno u terminalu bez potrebe za internetskom vezom ili pretraživačem. Iako većina dokumentacije danas postoji i online, `man` stranice ostaju nezamjenjive na ugradbenim sustavima, serverskim računalima i udaljenim sustavima koji nemaju grafičko sučelje, a ponekad ni pristup internetu — a to su upravo okruženja u kojima UNIX pokazuje svoje najveće prednosti. Uz to, `man` stranice su uvijek usklađene s točnom verzijom alata instaliranog na konkretnom sustavu, što online izvori ne mogu jamčiti.

Pretraživanje stranica vrši se naredbom `man <naredba>`, navigacija strelicama ili Space tipkom, a izlazi pritiskom na `q`. Stranice su organizirane u sekcije: - sekcija 1 — korisničke naredbe (`man 1 ls`), - sekcija 2 — sistemski pozivi (`man 2 open`), - sekcija 3 — bibliotečke funkcije (`man 3 printf`).

Preusmjeravanje ulaza i izlaza

Svaki program u UNIX-u standardno koristi tri toka podataka: standardni ulaz (`stdin`), standardni izlaz (`stdout`) i standardni izlaz za greške (`stderr`). Ljsuka pruža mehanizam **preusmjeravanja** kojim te tokove možemo prije pokretanja programa povezati s datotekama. Sam program time se ne mijenja — i dalje misli da čita sa `stdin` i piše na `stdout` — ali ljsuka tiho preusmjeri te tokove na zadane datoteke.

Najčešći operatori preusmjeravanja u **bash** ljsuci:

Operator	Opis
>	preusmjeri stdout u datoteku (briše postojeći sadržaj)
>>	dodaj stdout na kraj datoteke
<	čitaj stdin iz datoteke
2>	preusmjeri stderr u datoteku
2>>	dodaj stderr na kraj datoteke
&>	preusmjeri stdout i stderr zajedno
&>>	dodaj stdout i stderr na kraj
2>&1	spoji stderr na stdout (vrlo čest "trik")

Primjer — spasimo izlaz `ls -la` u datoteku, pa pogledajmo sadržaj:

```
$ ls -la > popis.txt
$ cat popis.txt
total 8
drwxr-xr-x 4 dkrst users 168 2026-03-11 18:04 .
drwxr-xr-x 11 dkrst users 352 2026-03-11 12:38 ..
-rw-r--r-- 1 dkrst users 33 2026-03-11 12:58 dat1.txt
-rw-r--r-- 1 dkrst users 33 2026-03-11 16:12 dat2.txt
-rw-r--r-- 1 dkrst users 0 2026-03-11 18:04 popis.txt
```

Zasebno preusmjeravanje stdout i stderr:

```
$ ls /home /nepostoji > rezultati.txt 2> greske.txt
$ cat rezultati.txt
/home:
marko ana petar
$ cat greske.txt
ls: cannot access '/nepostoji': No such file or directory
```

Posebna datoteka `/dev/null` je "crna rupa" jezgre — sve što se u nju upiše tiho nestaje. Korisna je kad želimo potpuno odbaciti neki izlaz:

```
$ neka_naredba 2> /dev/null          # zanemari greške, prikaži samo stdout
$ neka_naredba > /dev/null 2>&1      # zanemari sve, samo izvrši
```

Ulančavanje naredbi (pipes)

Pored preusmjeravanja u datoteku, izlaz jedne naredbe možemo izravno spojiti na ulaz druge — operatorom `|` (vertikalna crta, *pipe*). Ovo je realizacija temeljne UNIX filozofije: *mali programi koji rade jednu stvar dobro mogu se kombinirati u moćne lance obrade*.

Klasičan primjer — koliko redaka u log datoteci sadrži riječ `ERROR`?

```
$ cat program.log | grep "ERROR" | wc -l
42
```

Lanac radi ovako: `cat` čita datoteku i šalje sadržaj na stdout; taj stdout ulančan je s stdin-om naredbe `grep` koja propušta samo retke s riječi `ERROR`; izlaz `grep`-a ulančan je s `wc -l` koji broji retke. Konačni rezultat — broj redaka s greškom.

Drugi primjer — prikaži samo `.txt` datoteke u direktoriju:

```
$ ls -al | grep "\.txt"
-rw-r--r-- 1 dkrst users 33 2026-03-11 dat1.txt
-rw-r--r-- 1 dkrst users 45 2026-03-11 biljeske.txt
```

Lanci mogu biti dugi koliko god je potrebno. Klasičan primjer iz administracije — ispiši **različita** korisnička imena koja su u `auth.log` izazvala grešku:

```
$ grep "ERROR" auth.log | awk '{print $5}' | sort | uniq
admin
```

ana
marko

Ovdje grep filtrira retke s greškom, awk izvlači peto polje (korisničko ime), sort ih poredi, a uniq uklanja duplikate (uniq traži susjedne duplikate, pa zato sortiramo prije).

Anonimni cjevovod stvoren operatorom | postoji samo dok se naredbe izvršavaju. Pored njega postoji i **imenovani cjevovod** (FIFO), koji je trajan objekt u datotečnom sustavu — može ga otvoriti bilo koji proces koji zna njegovu putanju, čak i nakon završetka procesa koji ga je stvorio. Stvara se naredbom `mkfifo`, a u ispisu `ls -l` prikazuje se oznakom `p`.

UNIX procesi

U trenutku kada se program s diska učitava u memoriju, nastaje **proces** koji odgovara pokrenutom programu. Procesu se dodjeljuju resursi, stvara se njegovo radno okruženje (*environment*), definiraju se ovlasti i prava pristupa, te se dodjeljuju identifikacijske oznake. Najvažnije od njih:

- **PID** (*Process ID*) — jedinstveni broj kojim sustav identificira proces. Jezgra svakom procesu dodjeljuje vlastiti PID.
- **PPID** (*Parent Process ID*) — PID roditeljskog procesa, odnosno onog koji je ovaj proces pokrenuo. Ako roditelj završi prije djeteta, dijete nasljeđuje proces `init` (PID 1) kao novog roditelja.
- **UID** (*User ID*) — broj korisnika koji je vlasnik procesa. Određuje ovlasti procesa nad resursima sustava.
- **GID** (*Group ID*) — broj grupe vlasnika procesa.
- **EUID, EGID** — *Effective UID/GID*; koriste se za privremeno podizanje ovlasti, npr. pri pokretanju programa s postavljenim *setuid* bitom (klasičan primjer: `passwd` — alat za promjenu lozinke koji mora pisati u `/etc/shadow`, datoteku kojoj obični korisnici nemaju pristup).

Procese pregledavamo naredbom `ps`. `ps -ef` prikazuje sve procese u sustavu s ključnim atributima:

```
$ ps -ef | head -5
UID          PID  PPID  C  STIME TTY          TIME CMD
root           1      0  0  Mar01 ?           00:00:42 /sbin/init
root           2      0  0  Mar01 ?           00:00:00 [kthreadd]
dkrst       14567 14566  0  09:15 pts/0       00:00:00 -bash
dkrst       25016 14567  0  12:30 pts/0       00:00:00 ./program
```

Bitna napomena: novi proces u UNIX-u uvijek nastaje iz **postojećeg** procesa — sistemskim pozivom `fork()` koji dijeli postojeći proces u dva. Time se cijeli sustav procesa organizira kao stablo s `init`-om u korijenu. O ovome će biti više riječi u poglavlju o procesima.

Shell skripte

Pored interaktivnog načina rada, ljusku možemo koristiti i **programski**. Niz naredbi zapisan u tekstualnoj datoteci kojoj smo dali pravo izvršavanja postaje cjelovit program — **shell skripta**. Skripte koristimo za automatizaciju, od jednostavnih svakodnevnih radnji do složenih administracijskih i programerskih zadataka.

Pri izradi skripti potrebno je imati na umu **tip ljuske** za koji je skripta pisana. Iako su osnovne naredbe iste, detalji poput definiranja varijabli, uvjetnog grananja i petlji razlikuju se između ljuski. Najčešće ljuske su `bash` (Bourne Again Shell, najraširenija na Linuxu) i `csh` (C Shell). Razlike u sintaksi:

Operacija	bash	csh
Prva linija skripte	<code>#!/bin/bash</code>	<code>#!/usr/bin/csh</code>

Operacija	bash	csh
Dodjela varijable	ime="Marko"	set ime="Marko"
Čitanje varijable	echo \$ime	echo \$ime
Uvjetni izraz	if [\$a = \$b]; then	if (\$a == \$b) then
Kraj if bloka	fi	endif
for petlja	for i in lista; do	foreach i (lista)
Kraj for petlje	done	end
while petlja	while [uvjet]; do	while (uvjet)
Izlazni status	\$?	\$status
Argumenti skripte	\$1, \$2, ...	\$argv[1], \$argv[2], ...

Prvi redak skripte (`#!/bin/bash`) zove se **shebang** — to je posebna direktiva koju jezgra prepoznaje pri pokretanju skripte i koristi za odabir interpretera. Komentari u skripti počinju znakom `#` i traju do kraja retka.

Da bi skripta postala izvršna, treba joj dati pravo izvršavanja (x):

```
$ chmod +x skripta.sh
$ ./skripta.sh
```

U ovom direktoriju nalazi se nekoliko jednostavnih bash skripti koje ilustriraju ključne koncepte:

Najjednostavnija skripta

- **pozdrav.sh** — minimalan primjer skripte: shebang, komentar, echo poziv, korištenje varijabli okruženja i naredbenih supstitucija.

```
#!/bin/bash
# Najjednostavnija shell skripta - ispisuje pozdrav korisniku.

echo "Pozdrav, $USER!"
echo "Trenutni radni direktorij: $(pwd)"
echo "Datum i vrijeme: $(date)"
```

Prvi redak je shebang. Drugi redak je komentar. U pozivima echo koristimo dva mehanizma za umetanje vrijednosti u string:

- **\$USER** — varijabla okruženja koju ljuska automatski postavlja na ime trenutnog korisnika.
- **\$(naredba)** — naredbena supstitucija (engl. *command substitution*): ljuska izvrši naredbu unutar `$()`, pa njezin izlaz umetne na to mjesto. U ovom primjeru `$(pwd)` zamjenjuje se ispisom trenutnog radnog direktorija.

Pokretanje:

```
$ chmod +x pozdrav.sh
$ ./pozdrav.sh
Pozdrav, dkrst!
Trenutni radni direktorij: /home/dkrst/Programiranje_za_UNIX/P01-Osnove_UNIXa
Datum i vrijeme: Mon May 4 11:33:43 UTC 2026
```

Petlje, varijable i argumenti

- **brojac.sh** — broji od 1 do N, gdje N može biti zadan kao argument naredbenog retka, ili je 5 ako argument nije zadan.

```
#!/bin/bash
# Broji od 1 do N (N se daje kao argument, ili 5 ako nije zadan).
```

```

# Provjera broja argumenata
if [ $# -eq 0 ]; then
    n=5
else
    n=$1
fi

echo "Brojim od 1 do $n..."

# for petlja
for i in $(seq 1 $n); do
    echo "  $i"
done

echo "Gotovo!"

```

Skripta uvodi nekoliko bitnih koncepata:

- **\$#** — broj argumenata kojima je skripta pozvana (analogno argc u C-u).
- **\$1** — prvi argument (analogno argv[1]). Slično tome \$2, \$3 itd. su drugi, treći argument; \$0 je ime skripte (analogno argv[0]).
- **if [uvjet]; then ... else ... fi** — uvjetno grananje. Razmaci unutar zagrada su obavezni.
- **-eq** — operator jednakosti za cijele brojeve. Za znakovne nizove koristimo = (npr. `if [ "$ime" = "Marko" ]`).
- **for i in lista; do ... done** — petlja kroz vrijednosti u listi. Lista se može zadati eksplicitno (`for i in 1 2 3; do`), ili generirati naredbom (`for i in $(seq 1 5); do`), ili kao popis datoteka (`for f in *.txt; do`).
- **\$(seq 1 \$n)** — naredbena supstitucija; seq generira slijed brojeva.

Pokretanje:

```

$ ./brojac.sh
Brojim od 1 do 5...
1
2
3
4
5
Gotovo!
$ ./brojac.sh 3
Brojim od 1 do 3...
1
2
3
Gotovo!

```

Praktičan primjer — backup direktorija

- **backup.sh** — stvara komprimiranu arhivu zadanog direktorija, s timestampom u imenu arhive. Tipičan primjer skripte kakvu bismo zaista mogli koristiti svakodnevno.

```

#!/bin/bash
# Stvara komprimiranu arhivu zadanog direktorija s timestampom u imenu.
#
# Korištenje: ./backup.sh <direktorij>

# Provjera argumenata
if [ $# -ne 1 ]; then
    echo "Korištenje: $0 <direktorij>"

```

```

        exit 1
    fi

    dir=$1

    # Provjera da direktorij postoji
    if [ ! -d "$dir" ]; then
        echo "Greška: '$dir' nije direktorij ili ne postoji."
        exit 2
    fi

    # Ime archive: ime-direktorija_godina-mjesec-dan_sat-min.tar.gz
    basename=$(basename "$dir")
    timestamp=$(date +%Y-%m-%d_%H-%M)
    arhiva="${basename}_${timestamp}.tar.gz"

    echo "Stvaram arhivu '$arhiva' iz direktorija '$dir'..."
    tar -czf "$arhiva" "$dir"

    if [ $? -eq 0 ]; then
        velicina=$(du -h "$arhiva" | cut -f1)
        echo "Gotovo. Arhiva '$arhiva' (veličina: $velicina) uspješno stvorena."
    else
        echo "Greška pri stvaranju arhive."
        exit 3
    fi
fi

```

Skripta uvodi nove koncepte:

- **exit N** — završi skriptu s izlaznim statusom N. Konvencija: 0 znači uspjeh, ostale vrijednosti (1-255) različite tipove grešaka.
- **-d "\$dir"** — test je li putanja direktorij. Slično: -f za običnu datoteku, -e za bilo što (samo postoji li), -r/-w/-x za prava pristupa, -z za prazan string. Operator ! negira test.
- **navodnici oko "\$dir"** — uvijek navodimo varijable u dvostrukim navodnicima da bismo izbjegli probleme ako vrijednost sadrži razmake ili specijalne znakove. Bez njih bi tar -czf "\$arhiva" \$dir puknuo na imenima poput Moji dokumenti.
- **\$?** — izlazni status zadnje izvršene naredbe. 0 je uspjeh, sve drugo je neka greška.
- **\${ime}_dodatak** — vitičaste zagrade oko imena varijable kad se nadovezuje s tekstom koji bi se inače mogao tumačiti kao dio imena.
- **tar -czf** — alat za arhiviranje: c create, z gzip kompresija, f ime datoteke.

Pokretanje:

```

$ chmod +x backup.sh
$ ./backup.sh slike
Stvaram arhivu 'slike_2026-05-04_11-33.tar.gz' iz direktorija 'slike'...
Gotovo. Arhiva 'slike_2026-05-04_11-33.tar.gz' (veličina: 120K) uspješno stvorena.

```

Bez argumenta ili s neispravnim argumentom skripta uredno javlja grešku i izlazi:

```

$ ./backup.sh
Korištenje: ./backup.sh <direktorij>
$ ./backup.sh nepostojeci
Greška: 'nepostojeci' nije direktorij ili ne postoji.

```

Pretraga datoteka — find, grep, brojanje

- **trazi.sh** — pretražuje sve .txt datoteke u zadanom direktoriju (rekurzivno) i ispisuje koliko puta se u svakoj pojavljuje zadana ključna riječ. Skripta kombinira većinu dosad

spomenutih alata: testovi, petlje, find, grep, aritmetika.

```
#!/bin/bash
# Pretražuje sve tekstualne datoteke u zadanom direktoriju koje sadrže
# zadanu ključnu riječ i ispisuje broj pojavljivanja u svakoj datoteci.
#
# Korištenje: ./trazi.sh <direktorij> <kljucna_rijec>

if [ $# -ne 2 ]; then
    echo "Korištenje: $0 <direktorij> <kljucna_rijec>"
    exit 1
fi

dir=$1
rijec=$2

if [ ! -d "$dir" ]; then
    echo "Greška: '$dir' nije direktorij."
    exit 2
fi

echo "Tražim '$rijec' u tekstualnim datotekama unutar '$dir'..."
echo

ukupno=0
nadenno_u=0

# Pretraga svih .txt datoteka u direktoriju i poddirektorijima
for f in $(find "$dir" -type f -name "*.txt"); do
    broj=$(grep -c "$rijec" "$f")
    if [ "$broj" -gt 0 ]; then
        echo "  $f: $broj"
        ukupno=$((ukupno + broj))
        nadenno_u=$((nadenno_u + 1))
    fi
done

echo
echo "Ukupno pojavljivanja: $ukupno (u $nadenno_u datoteka)"
```

Novi koncepti:

- **find <gdje> -type f -name "*.txt"** — moćan alat za pretragu datoteka. -type f traži obične datoteke, -name "*.txt" traži po imenu, -mtime, -size po vremenu/veličini itd.
- **grep -c "\$rijec" "\$f"** — -c (count) vraća broj redaka koji sadrže uzorak (umjesto same retke).
- **\$((aritmetika))** — aritmetička evaluacija. $((a + b))$ zbraja, $((a * b))$ množi, $((a / b))$ cjelobrojno dijeli.

Pokretanje:

```
$ ./trazi.sh /tmp/dokumenti linux
Tražim 'linux' u tekstualnim datotekama unutar '/tmp/dokumenti'...
```

```
  /tmp/dokumenti/biljeske.txt: 5
  /tmp/dokumenti/projekt/readme.txt: 2
```

Ukupno pojavljivanja: 7 (u 2 datoteka)

Što dalje?

U sljedećem poglavlju krećemo s osnovama programiranja u C-u i procesa prevođenja, što su preduvjeti za C programiranje na UNIX sustavima koje obrađujemo u ostatku skripte. Sve što smo naučili u ovom poglavlju — rad u terminalu, prava pristupa, preusmjeravanje — koristit ćemo praktično u radu s primjerima.

Osnove programiranja

U ovom poglavlju dan je uvod u proces prevođenja i povezivanja C programa ilustriran jednostavnim primjerima.

Najjednostavniji program

- **pozdrav.c** — program u jednoj datoteci koji ispisuje poruku na standardni izlaz. Najmanji smisleni C program koji se može prevesti i pokrenuti; služi za demonstraciju osnovne sintakse (main, #include, printf) i osnovnog toka prevođenja.

```
#include <stdio.h>

int main() {
    printf("Dobar jutar!\n");

    return 0;
}
```

Program u više datoteka

Funkcionalno, primjer iz ovog odjeljka radi potpuno isto što i prethodni pozdrav — ispisuje istu poruku na standardni izlaz. Razlika je u tome što je kod razbijen u tri datoteke: zaglavlje s deklaracijom funkcije, C datoteku izvornog koda s njezinom definicijom i glavni program koji funkciju poziva. Ovaj umjetno složeniji oblik služi nam da objasnimo proces prevođenja i povezivanja u prisutnosti više prevodbenih jedinica. U realnim programima organizacija koda u više datoteka je gotovo univerzalno pravilo, pa je ovaj minimalni primjer dobra polazna točka za razumijevanje takvog načina rada.

- **funkcije.h** — zaglavlje s deklaracijom funkcije `dobar_jutar()`. Pretprocesorska direktiva `#include` doslovno čita sadržaj navedene datoteke i “zalijepi” ga na svoje mjesto, prije nego što prevodilac počne s pravim prevođenjem. Zato bi se zaglavlje koje se u istoj prevodbenoj jedinici uključi više puta (npr. preko više drugih zaglavlja koja ga svako sa svoje strane uključuju) doslovno višestruko zalijepilo, što bi izazvalo greške višestruke deklaracije. Da bismo to spriječili, koristimo zaštitu od višestrukog uključivanja (engl. *include guards*) — par pretprocesorskih direktiva (`#ifndef _FUNKCIJE_H_ / #define ... / #endif`) koji osigurava da se sadržaj zaglavlja ubaci samo prilikom prvog uključivanja, a sva kasnija ignoriraju.

```
#ifndef _FUNKCIJE_H_
#define _FUNKCIJE_H_

void dobar_jutar();

#endif
```

- **funkcije.c** — definicija (implementacija) funkcije `dobar_jutar()`.

```
#include <stdio.h>
```

```
void dobar_jutar() {
    printf("Dobar jutar!\n");
}
```

- **pozdrav_fn.c** — glavni program koji uključuje funkcije.h i poziva dobar_jutar().

```
#include "funkcije.h"

int main() {
    dobar_jutar();

    return 0;
}
```

Ovaj skup datoteka ilustrira osnovnu organizaciju projekta u više prevodbenih jedinica: zaglavlje .h sadrži deklaraciju i dijeli se između jedinica koje funkciju pozivaju i one koja ju definira, dok se .c datoteke prevode neovisno i kasnije povezuju u izvršni program.

Prevođenje

Ručno prevođenje u jednom koraku

Za pozdrav.c, koji ne ovisi o drugim datotekama izvornog koda, dovoljan je jedan poziv prevodioca:

```
gcc -Wall pozdrav.c -o pozdrav
```

Zastavica -Wall uključuje tipična upozorenja prevodioca; -o pozdrav određuje ime izlazne izvršne datoteke. Bez -o, rezultat bi se zvao a.out.

Ručno prevođenje u dva koraka

Postupak generiranja izvršne datoteke zapravo se sastoji od dvije odvojene faze. U prvoj fazi (**prevođenje**, engl. *compilation*) prevodilac iz datoteke izvornog koda proizvodi datoteku objektnog koda. U drugoj fazi (**povezivanje**, engl. *linking*) jedna ili više objektnih datoteka spaja se u izvršni program, uz eventualno uključivanje simbola iz vanjskih biblioteka. Oba koraka mogu se pokrenuti odvojeno:

```
gcc -Wall -c pozdrav.c           # prevođenje: pozdrav.c -> pozdrav.o
gcc -Wall pozdrav.o -o pozdrav   # povezivanje: pozdrav.o -> pozdrav
```

Zastavica -c nalaže prevodiocu da stane nakon faze prevođenja i ne pokreće povezivanje.

Ručno prevođenje programa iz više datoteka

Za pozdrav_fn potrebno je prevesti dvije izvorne datoteke u objektne, a zatim ih povezati u jedan izvršni program:

```
gcc -Wall -c pozdrav_fn.c        # -> pozdrav_fn.o
gcc -Wall -c funkcije.c          # -> funkcije.o
gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn
```

Ovdje se jasno vidi razlika između dviju faza koje smo ranije spomenuli — **prevođenja** i **povezivanja**. Prevođenje je sporiji proces u kojem prevodilac iz svake .c datoteke generira pripadnu .o (objektnu) datoteku — strojni kod te jedinice u oblik pripravan za povezivanje. Povezivanje je relativno brza operacija u kojoj se sve .o datoteke spajaju u izvršni program. Razdvajanje na dvije faze ima dvije važne koristi: **objektne datoteke možemo međusobno povezivati i u različitim kombinacijama** (ista funkcije.o može završiti u više različitih programa); još važnije, **kad u nekoj .c datoteci promijenimo neki redak, dovoljno je iznova prevesti samo tu jednu datoteku**, dok preostale

.o datoteke ostaju kakve su bile. Povezivanje, međutim, uvijek treba ponoviti — ono je obavezni zadnji korak nakon svake izmjene.

Ovaj uvid je upravo razlog postojanja alata make. U projektu od nekoliko datoteka, ručno paziti koja se .c smije propustiti pri ponovnom prevođenju brzo postaje naporno i podložno greškama. make taj posao automatizira: na temelju vremena zadnje izmjene svake datoteke, sam odluči koje .o treba ponovno generirati, a koje su već svježije, i izvodi samo nužne korake.

Korištenje alata make

make je alat koji automatizira upravo takav proces. Iz datoteke s pravilima (Makefile) čita koje ulazne datoteke čine projekt, kako ovise jedna o drugoj i kojim se naredbama iz njih generiraju izlazne datoteke. Na temelju vremena zadnje izmjene datoteka, make ponovno prevodi samo one koje su mijenjane od posljednjeg prevođenja, i ništa više. Pokreće se tako da se u direktoriju s Makefile-om zada:

```
make                # izvršava prvo pravilo u Makefileu
make ime_pravila    # izvršava navedeno pravilo
```

Struktura pravila

Svako pravilo ima oblik:

cilj: ovisnosti
naredbe

- **cilj** je najčešće ime datoteke koja nastaje izvršavanjem pravila (u pravilu izvršna ili objektna datoteka).
- **ovisnosti** je popis datoteka o kojima cilj ovisi — ako je bilo koja od njih novija od cilja, pravilo se izvršava.
- **naredbe** su naredbe ljske koje pravilo izvršava. **Moraju** biti uvučene tabulatorom, ne razmacima.

Gradnja Makefilea korak po korak

Krenut ćemo od najjednostavnije verzije i postupno je poboljšavati.

Korak 1: jednostavna pravila za svaki primjer

Prevedemo li logiku dvofaznog prevođenja iz prethodnog odjeljka u make pravila, za pozdrav dobivamo dva pravila — jedno za povezivanje (cilj pozdrav) i jedno za prevođenje (cilj pozdrav.o):

```
pozdrav: pozdrav.o
        gcc -Wall pozdrav.o -o pozdrav

pozdrav.o: pozdrav.c
        gcc -Wall -c pozdrav.c
```

Isti pristup za pozdrav_fn, koji se povezuje iz dvije objektna datoteke:

```
pozdrav_fn: pozdrav_fn.o funkcije.o
        gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn

pozdrav_fn.o: pozdrav_fn.c
        gcc -Wall -c pozdrav_fn.c

funkcije.o: funkcije.c
        gcc -Wall -c funkcije.c
```

Promotrimo strukturu i redoslijed izvođenja pravila. Kada korisnik zatraži `make pozdrav_fn`, `make` čita `Makefile` u potrazi za pravilom čiji je cilj `pozdrav_fn`. To pravilo kao ovisnosti navodi `pozdrav_fn.o` i `funkcije.o`. Kako ove dvije datoteke ne postoje, `make` traži pravila u kojima su one ciljevi, provjerava njihove ovisnosti, i izvršava zadane naredbe — najprije za `pozdrav_fn.o`, zatim za `funkcije.o` — i tek na kraju za `pozdrav_fn`.

Što ako neka od objektnih datoteka navedena u ovisnostima već postoji? U tom slučaju `make` u pravilu kojem je ta datoteka cilj provjerava ovisnosti, odnosno odgovarajuću datoteku izvornog koda: ako je datoteka izvornog koda (ovisnost) **novija** od objektna datoteke (cilj), naredbe iz pravila izvršit će se iznova kako bi objektna datoteka uključila izmjene u izvornom kodu nastale od posljednje primjene pravila. Ako je objektna datoteka novija od izvora — odnosno već “svježā” — `make` taj korak preskače. Na ovaj način `make` rekurzivno kroz `Makefile` provjerava jesu li pojedini koraci “zastarjeli”, tj. postoje li izmjene u kodu od posljednje primjene pravila, i provodi samo one korake koji su nužni.

Ovakav `Makefile` je funkcionalan, ali pokazuje dva problema: (a) pravila za generiranje `.o` datoteka su praktički identična — razlikuju se samo po imenu datoteke, i (b) naredba `gcc -Wall` je ponovljena u svakom pravilu, pa promjena prevodioca ili zastavica traži izmjenu na više mjesta.

Korak 2: implicitna pravila

Postupak prevođenja `.c` → `.o` uvijek je isti: iz datoteke `ime.c` generira se `ime.o` istim pozivom prevodioca. Za takve unificirane postupke `make` nudi **implicitna pravila** koja se primjenjuju na temelju uzorka ekstenzije. Pravilo:

```
.c.o:
    gcc -Wall -c $<
```

govori `make`-u kako iz bilo koje `.c` datoteke izgraditi istoimenu `.o` datoteku. Unutar naredbe, automatska varijabla `$<` zamjenjuje se imenom ulazne datoteke (one iz popisa ovisnosti). Isto se tako može koristiti `$@` za ime cilja.

Ovim jednim implicitnim pravilom zamijenjena su sva pojedinačna `.c` → `.o` pravila. `Makefile` sad izgleda ovako:

```
pozdrav: pozdrav.o
    gcc -Wall pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn

.c.o:
    gcc -Wall -c $<
```

Korak 3: varijable

I dalje je prevodilac ugrađen u pravila kao `gcc`, a zastavica `-Wall` ponovljena na više mjesta. Uvođenjem **varijabli** se dio koji se ponavlja izvuče iz pravila i centralizira:

```
CC = /usr/bin/gcc
CFLAGS = -Wall

pozdrav: pozdrav.o
    $(CC) $(CFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(CFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

.c.o:
    $(CC) $(CFLAGS) -c $<
```

Varijabla se deklarira imenom, znakom jednakosti i tekstualnom vrijednošću. Vrijednost se dohvaća kao \$(IME). Promjena prevodioca ili zastavica sada se svodi na izmjenu jednog retka.

Postoji konvencija iz GNU svijeta prema kojoj se zastavice za **prevođenje** drže u varijabli CFLAGS, a zastavice za **povezivanje** (npr. -L/put/do/lib ili biblioteke) u zasebnoj varijabli LDFLAGS. Naši primjeri u ovoj fazi ne trebaju posebne zastavice za linker, pa LDFLAGS ostaje prazna — ali ju uvodimo radi konzistentnosti i lakšeg proširivanja:

```
CC = /usr/bin/gcc
CFLAGS = -Wall
LDFLAGS =

pozdrav: pozdrav.o
    $(CC) $(LDFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(LDFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

.c.o:
    $(CC) $(CFLAGS) -c $<
```

Korak 4: specijalna pravila default, all, clean

Ostaje još nekoliko praktičnih poboljšanja kojima se uvodi nekoliko konvencionalnih pravila. Bitno je odmah napomenuti: **imena default, all i clean nisu rezervirane ključne riječi** — pravilo all jednako bismo mogli nazvati sve, clean bismo mogli nazvati ocisti, i sve bi radilo identično. Ova imena su dogovorna konvencija koje se programeri drže kako bi Makefile bio čitljiv i predvidljiv svakome tko ga pogleda. Slično kao s imenima varijabli u kodu ili s imenima programa, dobra je praksa odabrati imena iz kojih se odmah vidi što pravilo radi; držanje navedene konvencije čini naš kod razumljivim drugima — a često i nama samima kad se vratimo svom kodu nakon izvjesnog vremena.

default — pravilo koje se izvršava kad korisnik pokrene make bez argumenata. make u tom slučaju jednostavno izvršava prvo pravilo navedeno u Makefile-u, bez obzira na njegovo ime — pa se po konvenciji to prvo pravilo zove default i smješta na vrh datoteke. Da smo mu dali bilo koje drugo ime, svejedno bi bilo pokrenuto kad korisnik upiše make bez argumenata.

```
default: pozdrav_fn
```

Ovo pravilo ima samo cilj (default) i ovisnost (pozdrav_fn), ali nema naredbi. make provjerava ovisnosti i, prema ranije spomenutoj logici, traži pravilo u kojem je pozdrav_fn cilj te ga izvršava (rekurzivno, kako smo opisali — samo ako su pripadne datoteke zastarjele). Nakon toga default ne radi ništa jer nema vlastitih naredbi. Pravilo default koristi upravo taj trik da nas vodi do “korisnog” pravila koje stvarno gradi program na kojem trenutno radimo.

all — koristi isti trik za izgradnju svih ciljeva u direktoriju odjednom. Za lakše održavanje koristimo varijablu TARGETS koja popisuje sve izvršne datoteke:

```
TARGETS = pozdrav pozdrav_fn
```

```
all: $(TARGETS)
```

I ovo pravilo ima samo ovisnosti, bez naredbi: make rekurzivno gradi svaku navedenu izvršnu datoteku, a samo all ne radi ništa.

clean — pravilo koje briše sve izvršne, objektne i privremene datoteke iz direktorija. Za razliku od default i all, ovdje imamo cilj i naredbe, ali **nema ovisnosti**. To znači da se pravilo izvršava bezuvjetno svaki put kad korisnik upiše make clean — neovisno o stanju datoteka u direktoriju.

```
clean:
    rm -f $(TARGETS) *.o *~ a.out
```

I ovo pravilo bi se moglo nazvati drukčije (npr. ocisti), ali ime `clean` toliko je rasprostranjena konvencija da bi nestandardno ime samo zbunjivalo druge programere — pa ga zato i koristimo.

Konačni Makefile

Objedinimo sve navedeno u finalnu Makefile datoteku u ovom direktoriju:

```
CC = /usr/bin/gcc
CFLAGS = -Wall
LDFLAGS =
TARGETS = pozdrav pozdrav_fn

default: pozdrav_fn

all: $(TARGETS)

pozdrav: pozdrav.o
    $(CC) $(LDFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(LDFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

clean:
    rm -f $(TARGETS) *.o *~ a.out

.c.o:
    $(CC) $(CFLAGS) -c $<
```

Tipična uporaba:

```
make          # izvršava "default", tj. gradi pozdrav_fn
make all      # gradi oba primjera
make pozdrav  # gradi samo pozdrav
make clean    # briše izvršne i objektne datoteke
```

Pokretanje

Izvršni programi pokreću se navođenjem relativne putanje (./) iz istog direktorija:

```
./pozdrav
./pozdrav_fn
```

Oba primjera ispisuju istu poruku; razlika je isključivo u unutarnjoj organizaciji koda.

Ulazno/izlazne operacije

U ovom direktoriju nalaze se programi koji ilustriraju UNIX sistemske pozive za rad s datotekama: `open()`, `creat()`, `close()`, `read()`, `write()`, `lseek()` i `umask()`. Svi primjeri pisani su izravno korištenjem sistemskih poziva (bez uporabe funkcija standardne C biblioteke poput `fopen`, `fread`, `fprintf`), s ciljem da se prikaže kako operacijski sustav zapravo obavlja ulazno/izlazne operacije i na kojoj razini počiva koncept **“sve je datoteka”** — temeljno UNIX načelo prema kojem se datoteke, uređaji, terminali, cijevi i mrežni sockets svi koriste kroz isto skućeno sućelje file deskriptora.

Vrijedi napomenuti: funkcije standardne C biblioteke (`fopen`, `fread`, `fprintf`, `fputs`, ...) **na UNIX sustavima u konaćnici se svode na ove iste sistemske pozive**. Funkcije više razine pružaju zgodne dodatke poput interno upravljanoć mećuspremnika za smanjenje broja sistemskih poziva, formatiran ulaz/izlaz (`fprintf`), ili portabilnost izmeću operacijskih sustava — ali fizićka komunikacija s jezgrom uvijek se odvija kroz `read()`, `write()`, `open()` i ostale poziva opisane u nastavku. Razumijevanje sistemskih poziva otkriva što se zapravo dogaća “ispod” funkcija standardne biblioteke koje ste do sada vjerojatno koristili.

Deskriptori datoteka

Jezgra operacijskog sustava sve otvorene datoteke referencira pomoću nenegativnih cijelih brojeva koje nazivamo **deskriptorima datoteke** (engl. *file descriptors*). Deskriptor je apstraktna referenca koju proces dobiva u trenutku otvaranja datoteke i koristi za sve daljnje operacije na njoj (ćitanje, pisanje, pozicioniranje), do njezinog zatvaranja.

Pri tom proces ne mora znati ništa o samoj datoteci otvorenoj na nekom deskriptoru: gdje se fizićki nalazi, radi li se uopće o datoteci na disku, ili je možća rijeć o standardnom izlazu na koji program ispisuje poruke korisniku, ili o otvorenom portu na mreži. Zahvaljujući konceptu **“sve je datoteka”**, svim tim resursima upravlja se na isti naćin, korištenjem malog skupa standardiziranih sistemskih poziva. Naravno, upisivanje u datoteku na lokalnom disku fizićki se ne odvija na isti naćin kao komunikacija s udaljenim raćunalom putem mreže — jezgra ćuva informacije o svakom otvorenom deskriptoru i na temelju toga odabire odgovarajuću komunikacijsku rutinu. Programer te detalje ne vidi: sućelje je jedinstveno za sve tipove datoteka.

Pri pokretanju novog programa, najćešće su već zauzeti file deskriptori 0, 1 i 2. Ovi deskriptori koriste se za komunikaciju s korisnikom putem naredbenog retka:

Deskriptor	Naziv	Izvor / odrećište
0	standardni ulaz (<i>standard input</i>)	naredbeni redak — unos korisnika putem tipkovnice
1	standardni izlaz (<i>standard output</i>)	naredbeni redak — ispis u korisnićki terminal
2	standardni izlaz za greške (<i>standard error</i>)	naredbeni redak — ispis u korisnićki terminal

Ova konvencija datira iz ranih UNIX sustava 1970-ih i danas je neodvojiv dio POSIX standarda. Budući da su deskriptori 0, 1 i 2 pri pokretanju programa u pravilu već zauzeti, novokreirani deskriptori — oni koje vraća `open()` ili `creat()` — u pravilu imaju vrijednost 3 ili veću. Sama jezgra pri otvaranju nove datoteke uvijek dodjeljuje **najniži slobodan deskriptor**.

Vrijednosti deskriptora 0, 1 i 2 definirane su u zaglavlju `<unistd.h>` kao simboličke konstante `STDIN_FILENO`, `STDOUT_FILENO` i `STDERR_FILENO`. Preporuka je u kodu koristiti ove konstante umjesto golih brojeva 0, 1 i 2 — kod time postaje čitljiviji.

Sistemske pozive za rad s datotekama

Funkcija `open()`

Osnovna UNIX funkcija za otvaranje (i po potrebi kreiranje) datoteka.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Povratna vrijednost: file deskriptor otvorene datoteke (nenegativni cijeli broj), ili -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do datoteke koja se otvara ili kreira.
- **oflag** — kombinacija bitovnih konstanti koja specificira način otvaranja. Mora sadržavati **točno jednu** od sljedeće tri konstante:
 - `O_RDONLY` — otvori samo za čitanje,
 - `O_WRONLY` — otvori samo za pisanje,
 - `O_RDWR` — otvori za čitanje i pisanje.

Uz tu obaveznu konstantu može se kombinacijom bitovnog **OR**-a (`|`) dodati jedna ili više opcionalnih:

- `O_CREAT` — kreiraj datoteku ako ne postoji (zahtijeva treći argument `mode`),
- `O_APPEND` — pri svakom pisanju pomakni offset na kraj datoteke, bez obzira na trenutni file offset,
- `O_TRUNC` — ako je datoteka otvorena za pisanje, izbriši njezin sadržaj (dužina postaje 0),
- `O_EXCL` — u kombinaciji s `O_CREAT`: vrati grešku ako datoteka već postoji; provjera i kreiranje izvode se kao **atomska operacija**,
- `O_NONBLOCK` — postavi neblokirajući način rada za I/O operacije,
- `O_SYNC` — čeka da se svaka operacija pisanja fizički dovrši na disku.

Tipične kombinacije zastavica koje ćete često sresti u UNIX kodu:

Kombinacija	Značenje
<code>O_WRONLY O_TRUNC</code>	otvori za pisanje i skрати na 0 (obriši postojeći sadržaj)
<code>O_WRONLY O_CREAT</code>	otvori za pisanje; ako datoteka ne postoji, stvori je
<code>O_WRONLY O_CREAT O_TRUNC</code>	otvori za pisanje i obriši postojeći sadržaj; ako datoteka ne postoji, stvori je
<code>O_WRONLY O_CREAT O_EXCL</code>	stvori novu datoteku i otvori je za pisanje; ako datoteka već postoji, vrati grešku

Kombinacija	Značenje
O_WRONLY O_APPEND	otvori za pisanje; svako pisanje dodaje se na kraj datoteke, bez obzira na trenutni file offset

- **mode** — opcionalni treći argument koji se navodi pri kreiranju nove datoteke (uz zastavicu O_CREAT). Specificira prava pristupa novostvorene datoteke. Može se zadati kao troznamenkasti oktalni broj (svaka znamenka definira prava za vlasnika, grupu i ostale korisnike), ili kombinacijom konstanti tipa mode_t definiranih u zaglavlju <sys/stat.h>:

Konstanta	Oktalna vrijednost	Značenje
S_IRUSR	0400	čitanje za vlasnika
S_IWUSR	0200	pisanje za vlasnika
S_IXUSR	0100	izvršavanje za vlasnika
S_IRGRP	0040	čitanje za grupu
S_IWGRP	0020	pisanje za grupu
S_IXGRP	0010	izvršavanje za grupu
S_IROTH	0004	čitanje za ostale
S_IWOTH	0002	pisanje za ostale
S_IXOTH	0001	izvršavanje za ostale

Tako je primjerice S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH ekvivalentno oktalnom zapisu 0644 — vlasnik smije čitati i pisati, grupa i ostali samo čitati.

Funkcija creat()

Namijenjena je kreiranju novih datoteka.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int creat(const char *pathname, mode_t mode);
```

Povratna vrijednost: file deskriptor otvorene datoteke, ili -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do datoteke koja se kreira.
- **mode** — prava pristupa novostvorene datoteke (kombinacija konstanti tipa mode_t, kako je opisano kod open()-a).

Ova funkcija ekvivalentna je pozivu:

```
open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Iz definicije slijede dva ograničenja: creat() datoteku otvara **samo za pisanje**, a ako datoteka već postoji, njezin sadržaj se briše (O_TRUNC). Zato je creat() jednostavno kratki oblik za vrlo čest slučaj korištenja open()-a — u modernom kodu obično se ide direktno preko open()-a koji nudi punu kontrolu.

Funkcija close()

Zatvara datoteku otvorenu na file deskriptoru filedes, čime jezgra oslobađa deskriptor za ponovnu upotrebu.

```
#include <unistd.h>

int close(int filedес);
```

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke koju treba zatvoriti.

Kada proces završi s radom, automatski se zatvaraju sve otvorene datoteke, pa eksplicitno pozivanje `close()` nije *strogo* nužno. Ipak, preporučuje se eksplicitno zatvaranje jer sustav ima ograničen broj deskriptora po procesu, a za neke tipove datoteka (npr. mrežne sockete) zatvaranje pokreće važne završne operacije.

Funkcija `read()`

Čita podatke iz datoteke i upisuje ih u memorijski prostor na koji pokazuje `buff`.

```
#include <unistd.h>

ssize_t read(int filedес, void *buff, size_t nbytes);
```

Povratna vrijednost: broj stvarno pročitanih bajtova (može biti manji od `nbytes`), 0 po dosizanju kraja datoteke (*EOF*), ili -1 u slučaju greške.

Povratni tip `ssize_t` (*signed size*) je POSIX-om definiran ekvivalent `size_t` koji može imati i negativnu vrijednost — neophodan da bi funkcija mogla vratiti -1 kao oznaku greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **buff** — pokazivač na statički ili dinamički alociran memorijski blok u koji se upisuju pročitani podaci. `read()` ne alocira memoriju sam — to je zadaća programera.
- **nbytes** — maksimalan broj bajtova koji će se pokušati pročitati.

Čitanje počinje na trenutnom file offsetu. Ova vrijednost je nakon otvaranja datoteke uvijek 0, tj. pokazuje na početak datoteke. Nakon svakog uspješnog poziva `read()`-a offset se automatski pomiče za broj pročitanih bajtova. Razlozi zbog kojih stvarno pročitani broj može biti manji od traženog uključuju dostignut kraj datoteke, čitanje s terminala (koji vraća redak koji je korisnik utipkao odjednom), ili čitanje s mrežnog socketeta.

Funkcija `write()`

Upisuje podatke iz memorijskog bloka u datoteku identificiranu deskriptorom `filedes`.

```
#include <unistd.h>

ssize_t write(int filedес, const void *buff, size_t nbytes);
```

Povratna vrijednost: broj stvarno upisanih bajtova (po POSIX-u garantirano jednak `nbytes` u uspješnom slučaju za obične datoteke; može biti manji za neke tipove datoteka poput mrežnih socketeta), ili -1 u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **buff** — pokazivač na memorijski blok koji se upisuje.
- **nbytes** — broj bajtova koji se piše.

Pisanje počinje na trenutnom file offsetu, a nakon uspješnog upisa offset se pomiče za broj upisanih bajtova. Iznimka je kad je datoteka otvorena s `O_APPEND` zastavicom — tada jezgra prije svakog pisanja postavlja offset na kraj datoteke.

Primjeri

Otvaranje, čitanje i pisanje

- **read_file.c** — otvara datoteku moja_datoteka.txt i ispisuje njen sadržaj na standardni izlaz.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int n, fd;
    char s;

    fd = open("moja_datoteka.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    while ((n = read(fd, &s, 1)) > 0) {
        if (write(STDOUT_FILENO, &s, 1) != 1) {
            perror("write");
            return 1;
        }
    }

    close(fd);
    return 0;
}
```

Datoteka se otvara sistemskim pozivom `open()` u načinu samo za čitanje (`O_RDONLY`), čita se znak po znak pozivima `read()`, a svaki znak se prosljeđuje na standardni izlaz pozivom `write()`. Program demonstrira osnovni slijed rada s datotekom (`open` → `read/write` → `close`) i rukovanje greškama preko povratnih vrijednosti sistemskih poziva.

./read_file

- **io_copy.c** — kopira standardni ulaz na standardni izlaz, čitajući u međuspremnik konstantne veličine (`BUFSIZE`).

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#define BUFSIZE 1024

int main() {
    int n;
    char buf[BUFSIZE];

    while ((n = read(STDIN_FILENO, buf, BUFSIZE)) > 0) {
        if (write(STDOUT_FILENO, buf, n) != n) {
            perror("write");
        }
    }
}
```

```

        return 1;
    }
}

if (n < 0)
    perror("read");

return 0;
}

```

Za razliku od `read_file.c` koji čita znak po znak, ovdje se u jednom pozivu `read()` pokušava pročitati cijeli blok bajtova, čime se višestruko smanjuje broj sistemskih poziva potrebnih za istu količinu prenesenih podataka. Primjer ujedno ilustrira koncept "sve je datoteka": pri pokretanju bez preusmjeravanja, standardni ulaz vezan je na tipkovnicu, a standardni izlaz na terminal — isti `read()` i `write()` koji bi radili s običnim datotekama na disku ovdje rade s terminalom. Svaki redak koji korisnik utipka program odmah ispiše natrag, a petlja se prekida pritiskom `Ctrl+D` (oznaka kraja ulaza, EOF):

```

$ ./io_copy
Prvi red teksta
Prvi red teksta
Drugi red teksta
Drugi red teksta
^D
$

```

U kombinaciji s preusmjeravanjem standardnog izlaza, isti program može poslužiti kao jednostavan alat za upis teksta u datoteku — sve što korisnik utipka do `Ctrl+D` završi kao sadržaj datoteke, a terminal u međuvremenu ne prikazuje ništa dodatno jer je standardni izlaz preusmjeren:

```

$ ./io_copy > datoteka.txt
Prvi red teksta
Drugi red teksta
^D
$ cat datoteka.txt
Prvi red teksta
Drugi red teksta

```

Pozicioniranje unutar datoteke (`lseek`)

Sistemske pozive `lseek()` mijenja file offset otvorene datoteke. Koristi se kad želimo čitati ili pisati na specifičnoj poziciji bez sekvencijalnog prolaska.

```

#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedes, off_t offset, int whence);

```

Povratna vrijednost: novi file offset (broj bajtova od početka datoteke), ili `(off_t)-1` u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **offset** — pomak; tumači se relativno prema vrijednosti `whence`.
- **whence** — referentna točka:
 - `SEEK_SET` — pomak je apsolutni, mjereno od početka datoteke,
 - `SEEK_CUR` — pomak je relativan u odnosu na trenutni offset,

- SEEK_END — pomak je relativan u odnosu na kraj datoteke (može biti i negativan da se vrati unazad).

lseek() se zove “seek” jer pomiče glavu za čitanje na proizvoljnu poziciju u datoteci — ali to “pomicanje” je samo logičko, na razini jezgrinog file offseta. Stvarni pristup disku događa se tek pri sljedećem read()-u ili write()-u.

- **f_strip.c** — demonstrira lseek() s SEEK_SET (apsolutno pozicioniranje od početka datoteke).

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    char buf1[] = "Prvi redak teksta";
    char buf2[] = "Drugi redak teksta";
    int fd;

    fd = open("file.strip", O_WRONLY | O_CREAT | O_TRUNC, (mode_t)0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1) {
        perror("write buf1");
        return 1;
    }

    if (lseek(fd, 15, SEEK_SET) == -1) {
        perror("lseek");
        return 1;
    }

    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1) {
        perror("write buf2");
        return 1;
    }

    close(fd);
    exit(0);
}
```

Program otvara datoteku za pisanje, upisuje prvi niz znakova, lseek-om postavlja offset na 15. bajt i upisivanjem drugog niza **prepisuje** dio postojećeg sadržaja. Rezultat pokazuje da se pozicioniranje unutar otvorene datoteke može slobodno kombinirati s čitanjem i pisanjem — offset koji jezgra pamti nije povezan s fizičkim rasporedom blokova na disku.

Rezultat se može provjeriti UNIX naredbom cat, koja ispisuje sadržaj jedne ili više datoteka na standardni izlaz:

```
$ ./f_strip
$ cat file.strip
Prvi redak teksDrugi redak teksta
```

Prvih 15 bajtova (Prvi redak teks) ostalo je netaknuto iz prvog upisa, a od 15. bajta nadalje vidljiv je sadržaj drugog niza koji je lseek + write upisao preko ostatka prethodnog teksta.

- **f_hole.c** — demonstrira lseek() s SEEK_CUR (relativno pomicanje od trenutne pozicije).

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    char buf1[] = "Prvi redak teksta";
    char buf2[] = "Drugi redak teksta";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1) {
        perror("creat");
        return 1;
    }

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1) {
        perror("write buf1");
        return 1;
    }

    if (lseek(fd, 15, SEEK_CUR) == -1) {
        perror("lseek");
        return 1;
    }

    if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1) {
        perror("write buf2");
        return 1;
    }

    close(fd);
    exit(0);
}
```

Nakon upisa prvog niza, offset se pomiče 15 bajtova naprijed — **iza** trenutnog kraja datoteke — i tek se tada upisuje drugi niz. Petnaest bajtova između prve i druge linije ostaje kao **rupa**, područje koje pri čitanju iščitava kao bajtovi s vrijednošću 0 (null terminator), iako write() ondje ništa nije upisao.

```
$ ./f_hole
$ ls -l file.hole
-rw-r--r-- 1 dkrst users 52 Apr 22 12:54 file.hole
$ du -h file.hole
4.0K    file.hole
$ od -c file.hole
00000000  P   r   v   i           r   e   d   a   k           t   e   k   s   t
00000020  a  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0
00000040  \0  D   r   u   g   i           r   e   d   a   k           t   e   k
00000060  s   t   a  \0
```


0000064

Tri naredbe daju pogled na datoteku iz tri različita kuta:

- `ls -l` prijavljuje **logičku veličinu** datoteke — 52 bajta. To je raspon od početka datoteke do posljednjeg upisanog bajta, uključujući i bajtove kroz rupu koje `write()` nikad nije dotaknuo. Isti broj vratio bi i `lseek(fd, 0, SEEK_END)` u programu.
- `du -h` prijavljuje **stvarno zauzeće diska** — 4.0K (odnosno 4096 bajtova, tipična veličina bloka Linux datotečnih sustava poput *ext4*; na drugim sustavima može biti 512 B, 8 KB ili više). Razlog zašto datoteka od 52 bajta zauzima puni 4 KB blok leži u činjenici da su tvrdi diskovi u UNIX-u **blok specijalni uređaji** (vidi poglavlje *Osnove UNIX-a*): podaci se na njima prenose i adresiraju u blokovima fiksne veličine, ne bajt po bajt. Datotečni sustav slijedi tu granularnost — najmanja jedinica alokacije za svaku datoteku je jedan cijeli blok, pa i datoteka od jednog jedinog bajta zauzima koliko i puni blok.
- `od -c` otkriva unutarnju strukturu — niz od 15 uzastopnih `\0` bajtova između dva upisana niza točno je područje rupe. S gledišta procesa, tih 15 bajtova postoji i čitanjem bi se dobile nule; jezgra ih pri čitanju transparentno popunjava bez da su ti podaci ikad bili fizički upisani na disk.

Prava pristupa i maska (umask)

Sistemske poziv `umask()` postavlja masku kreiranja datoteka za trenutni proces. Maska predstavlja bitove **koji će biti uklonjeni** iz prava pristupa kada proces stvara nove datoteke (npr. preko `open()`-a s `O_CREAT` ili `creat()`-a).

```
#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t cmask);
```

Povratna vrijednost: prethodna vrijednost maske.

Argumenti:

- **cmask** — nova vrijednost maske; kombinacija istih konstanti kao za `mode` u `open()`-u.

Algoritam je jednostavan: za svako kreiranje datoteke, jezgra uzima `mode` koji je program tražio i **iz njega ukloni** sve bitove koji su postavljeni u maski. Tako je rezultatni način pristupa `mode & ~cmask`. Maska se nasljeđuje od roditelja procesa; tipična vrijednost u shellu je `0022` (oduzima pravo pisanja grupi i ostalima).

- **perm_mask.c** — ilustrira djelovanje maske kreiranja datoteke (`umask`) na prava pristupa pri pozivu `creat()`.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>

#define PRAVA S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH

int main() {
    umask(0);
    if (creat("datoteka1", PRAVA) < 0)
        perror("creat datoteka1");

    umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
    if (creat("datoteka2", PRAVA) < 0)
```

```

        perror("creat datoteka2");

    return 0;
}

```

Program dva puta stvara datoteku s istim zatraženim pravima (rw-rw-r--), jednom s maskom postavljenom na 0 (nikakva prava se ne oduzimaju), a jednom s maskom koja eksplicitno isključuje pisanje za grupu i čitanje/pisanje za ostale. Usporedbom stvarno dobivenih prava vidi se učinak maske:

```

$ ./perm_mask
$ ls -al datoteka1 datoteka2
-rw-rw-r-- 1 dkrst users 0 Jan 16 16:23 datoteka1
-rw----- 1 dkrst users 0 Jan 16 16:23 datoteka2

```

Datoteka datoteka1 zadržala je sva zatražena prava jer je maska bila 0, dok su u slučaju datoteke datoteka2 maskom isključena sva prava za grupu i ostale korisnike.

Argumenti naredbenog retka

Naredbe i programi na UNIX sustavu često prihvaćaju **argumente naredbenog retka** — niz tokena koje korisnik upisuje uz ime programa kako bi izmijenio njegovo ponašanje, ili da bi mu predao ulaz (npr. ime datoteke nad kojom program radi). Primjer iz svakodnevnog rada s ljuskom je `cp izvor.txt cilj.txt`, gdje `cp` prima dva argumenta. Da bi C program mogao iščitati te argumente, `main` se deklarira u proširenom obliku:

```
int main(int argc, char *argv[])
```

Ovdje `argc` sadrži broj argumenata, a `argv` je polje nizova znakova u kojemu je svaki argument zaseban niz. Bitno je naglasiti da `argc` uključuje i samu naredbu, ne samo argumente koje korisnik dodaje uz nju. Po konvenciji, `argv[0]` je ime kojim je program pokrenut (uključujući i putanju ako je upisana), a stvarni argumenti slijede od `argv[1]` nadalje. Tako pri pozivu `./f_write izlaz.txt`, vrijednosti će biti `argc = 2`, `argv[0] = "./f_write"`, `argv[1] = "izlaz.txt"` — `argc` je 2 jer broji i samo ime programa i jedan argument koji mu je dan. Programi obično odmah na početku provjeravaju je li `argc` u očekivanom rasponu i ako nije, ispišu uputu o korištenju te uredno završe.

- **f_write.c** — čita sa standardnog ulaza i upisuje u novokreiranu datoteku čije se ime zadaje kao argument naredbenog retka.

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#define FMODE S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH

int main(int argc, char *argv[]) {
    int fd, n;
    char s;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 0;
    }

    fd = creat(argv[1], FMODE);
    if (fd == -1) {
        perror("creat");
    }
}

```

```

        return 1;
    }

    while ((n = read(STDIN_FILENO, &s, 1)) > 0)
        if (write(fd, &s, 1) < 0) {
            perror("write");
            return 1;
        }

    close(fd);
    return 0;
}

```

Odmah na početku programa provjerava se `argc != 2` — očekuje se točno jedan argument (ime izlazne datoteke) uz ime programa. Ako korisnik program pozove bez argumenta ili s previše njih, program javlja uputu o korištenju i završava. Konverzija `%s` zamjenjuje se upravo vrijednošću `argv[0]` — imenom kojim je program pokrenut — pa poruka korisniku uvijek automatski odražava točan naziv pod kojim je program bio zvan, neovisno o tome je li preimenovan ili pozvan preko simboličke veze. Ovakva provjera ulaznih argumenata uobičajen je uzorak u svim UNIX programima.

Program se poziva s imenom izlazne datoteke kao jedinim argumentom; nakon toga sve što korisnik utipka do oznake kraja ulaza (Ctrl+D) zapisuje se u tu datoteku:

```

$ ./f_write izlaz.txt
Prvi red teksta
Drugi red teksta
^D
$ cat izlaz.txt
Prvi red teksta
Drugi red teksta

```

- **f_cat.c** — pojednostavljena implementacija UNIX naredbe `cat`.

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int rw(int fdin, int fdout) {
    int n;
    char s;
    while ((n = read(fdin, &s, 1)) > 0)
        write(fdout, &s, 1);

    return n;
}

int main(int argc, char **argv) {
    int k, fd;
    if (argc < 2) {
        rw(STDIN_FILENO, STDOUT_FILENO);
    } else {
        for (k = 1; k < argc; k++) {
            fd = open(argv[k], O_RDONLY);
            if (fd < 0) /* ne mogu otvoriti */
                perror("open");
            else {
                rw(fd, STDOUT_FILENO);
            }
        }
    }
}

```

```

        close(fd);
    }
}

return 0;
}

```

Osnovna petlja čitanja i pisanja enkapsulirana je u pomoćnoj funkciji `rw(fdin, fdout)` koja čita s jednog deskriptora i piše na drugi. Ponašanje programa ovisi o argumentima naredbenog retka:

- **Bez argumenata** program poziva `rw(STDIN_FILENO, STDOUT_FILENO)` i tako praktički radi istu stvar kao `io_copy`: čita sa standardnog ulaza i ispisuje na standardni izlaz.
- **S jednim ili više argumenata** program redom otvara svaku navedenu datoteku pozivom `open()` i njezin sadržaj prosljeđuje na standardni izlaz pozivom iste funkcije `rw()`, ali sad s file deskriptorom otvorene datoteke umjesto standardnog ulaza.

Program time u bitnome reproducira funkcionalnost UNIX naredbe `cat` i ujedno sažima sve koncepte iz prethodnih primjera: otvaranje datoteka s provjerom grešaka, jedinstveno sučelje preko file deskriptora, rad s argumentima naredbenog retka, i uniformno korištenje istih sistemskih poziva bez obzira na izvor (datoteka ili standardni tok).

Pokretanje:

```
$ ./f_cat
```

Bez argumenata, program se ponaša kao `io_copy` — čita sa standardnog ulaza i ispisuje na standardni izlaz, sve dok ne dobije EOF.

```
$ ./f_cat f_cat.c
```

S jednim argumentom, program ispisuje sadržaj te datoteke na standardni izlaz. U gornjem primjeru program ispisuje vlastiti izvorni kod — i to bez ičega posebnoga: jednako bi ispisao bilo koju drugu tekstualnu datoteku navedenu kao argument.

```
$ ./f_cat datoteka1.txt datoteka2.txt
```

Sa više argumenata, program ispisuje navedene datoteke jednu za drugom — u ovom slučaju najprije sadržaj `datoteka1.txt`, a odmah za njim sadržaj `datoteka2.txt` (naravno, pod uvjetom da obje datoteke postoje u radnom direktoriju). Time se reproducira ponašanje UNIX naredbe `cat` po kojoj je program i nazvan: spaja (engl. *concatenate*) sadržaje više datoteka u jedinstveni izlazni tok. Ako neka od navedenih datoteka ne postoji, program će za nju ispisati poruku o grešci pozivom `perror()`, ali **neće prekinuti izvođenje** — sve ostale datoteke koje postoje uredno će biti ispisane.

Dijeljenje datoteka i preusmjeravanje (`dup`, `dup2`)

Da bismo razumjeli kako UNIX upravlja otvorenim datotekama i kako više procesa ili više deskriptora unutar istog procesa može pristupati istoj datoteci, potrebno je upoznati interne strukture jezgre. Razumijevanje ovih koncepata bit će nam korisno i u kasnijim poglavljima jer se na te iste strukture stalno vraćamo kad govorimo o procesima i nasljeđivanju resursa.

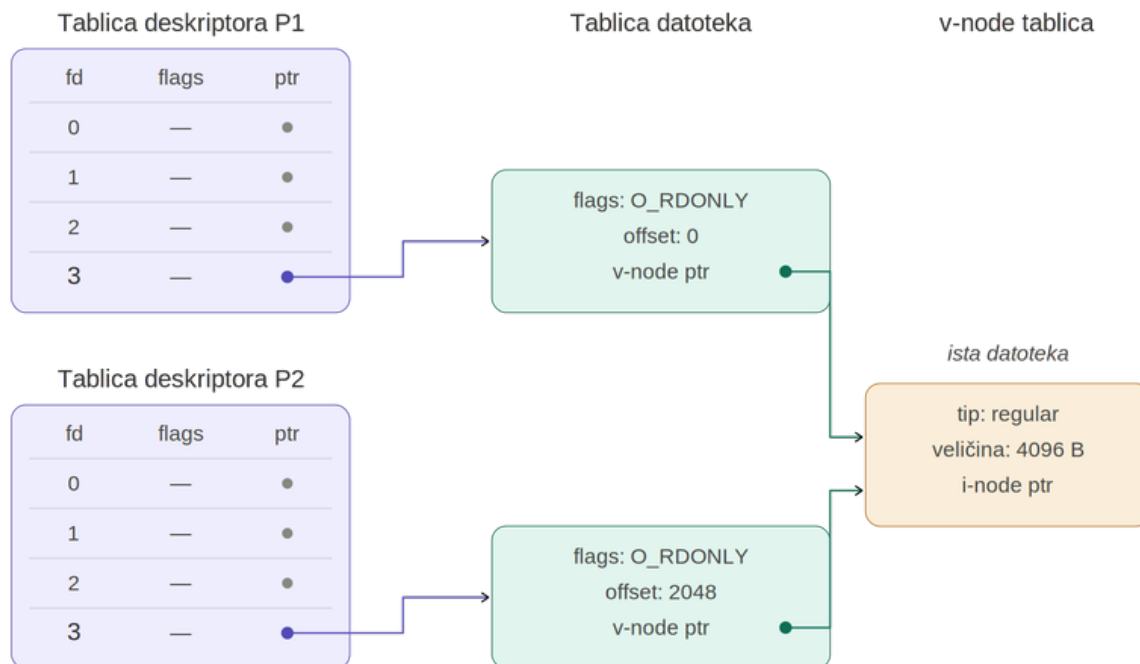
Jezgra održava tri strukture u memoriji koje surađuju pri rukovanju otvorenim datotekama:

- **Tablica procesa** — jezgra svakom aktivnom procesu dodjeljuje jedan zapis u tablici procesa. Zapis sadrži sve informacije o procesu (identifikator, stanje, otvorene datoteke, masku `umask` itd.). U sklopu zapisa o procesu nalazi se **tablica deskriptora**:

privatna lista svih deskriptora otvorenih unutar tog procesa. Tablicu procesa detaljnije ćemo razraditi u kasnijim poglavljima.

- **Tablica datoteka** — globalna struktura jezgre. U nju se upisuje novi zapis pri **svakom** pojedinom otvaranju datoteke. Zapis sadrži statusne zastavice (O_RDONLY, O_APPEND, ...), trenutni file offset, te pokazivač na zapis u v-node tablici.
- **v-node tablica** — apstrakcija fizičke datoteke. Svaki zapis opisuje samu datoteku (tip, lokaciju, ...). Više otvaranja iste datoteke pokazuju na isti v-node, ali svaki ima vlastiti zapis u tablici datoteka.

Slika u nastavku prikazuje tipičnu situaciju u kojoj dva procesa nezavisno otvaraju **istu** fizičku datoteku — svaki vlastitim pozivom `open()`:



Dva procesa nezavisno otvaraju istu datoteku: vlastiti zapisi u tablici datoteka, zajednički v-node.

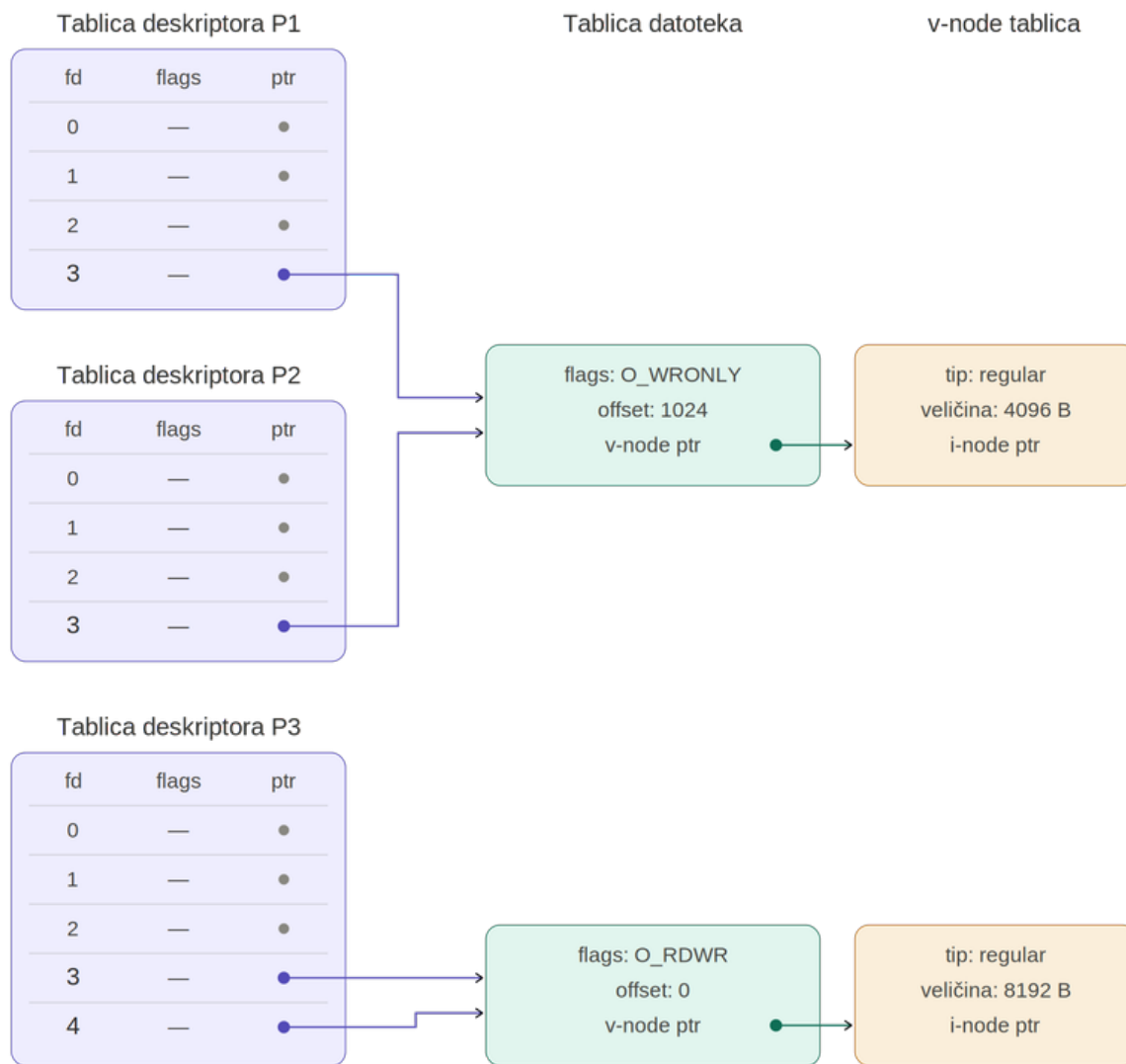
Slika 3: Dva procesa nezavisno otvaraju istu datoteku

Iako se radi o istoj datoteci, jezgra svakom procesu dodjeljuje zaseban zapis u tablici datoteka — pa svaki ima vlastiti, neovisni file offset i vlastite zastavice. Dijeljenje se događa samo na razini v-node tablice. Posljedica: pomak jednog procesa u datoteci ne utječe na poziciju drugoga, ali eventualne izmjene sadržaja postaju vidljive obama procesima.

Drugi oblik dijeljenja događa se **na razini same tablice datoteka**: dva ili više deskriptora pokazuju na *isti* zapis u tablici datoteka, pa dijele i isti file offset i iste statusne zastavice. Do ovoga može doći na dva načina:

1. **nasljeđivanjem deskriptora** kad jedan proces pozivom `fork()` stvori novi (vidi poglavlje o procesima),
2. **dupliciranjem deskriptora** unutar istog procesa pozivom `dup()` ili `dup2()`.

Slika pokazuje oba slučaja: procesi P1 i P2 dijele zapis u tablici datoteka jer je P2 stvoren `fork()`-om iz P1; unutar procesa P3 dva deskriptora (3 i 4) pokazuju na isti zapis jer je jedan nastao dupliciranjem drugoga.



Dijeljenje na razini tablice datoteka: dijeljeni zapis između P1 i P2, te dijeljeni zapis između dva deskriptora unutar procesa P3.

Slika 4: Dijeljenje na razini tablice datoteka

Dupliciranje deskriptora — dup() i dup2()

```
#include <unistd.h>

int dup(int fildes);
int dup2(int fildes, int fildes2);
```

Povratna vrijednost: novi file deskriptor, ili -1 u slučaju greške.

Argumenti:

- **fildes** — otvoreni deskriptor koji se duplicira.
- **fildes2** (samo za dup2) — ciljni deskriptor na koji se duplicira fildes. Ako je fildes2 već otvoren, dup2() ga **atomski zatvara** prije dupliciranja. Ako je fildes2 == fildes, poziv nema učinka.

dup() duplicira fildes na **najniži slobodni deskriptor**. dup2() se razlikuje po tome što ciljni deskriptor zadajemo eksplicitno — i ta operacija je atomska.

Na prvi pogled nije jasno čemu unutar istog procesa služe dva deskriptora koja pokazuju na isti zapis u tablici datoteka. Praktična primjena je upravo **preusmjeravanje standardnog ulaza i izlaza** — mehanizam kojim ljuska ostvaruje preusmjeravanje pomoću operatora < i >.

- **dup_redirect.c** — preusmjerava standardni izlaz na datoteku korištenjem close() + dup().

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd, newfd;

    fd = open("izlaz.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    close(STDOUT_FILENO); /* zatvori standardni izlaz */
    newfd = dup(fd);      /* dup vraća najnižu slobodnu vrijednost = 1 */

    if (newfd != fd)
        close(fd);

    printf("Ovaj tekst nece završiti u terminalu!\n");

    return 0;
}
```

Otvorimo datoteku, zatvorimo deskriptor 1 (standardni izlaz, čime taj broj postane slobodan), pa pozovemo dup(fd). Budući da dup() vraća **najniži slobodni** deskriptor, dobit ćemo upravo 1 — od ovog trenutka standardni izlaz pokazuje na našu datoteku, pa svaki printf u nastavku završava u datoteci, ne u terminalu.

```
$ ./dup_redirect
$ cat izlaz.txt
Ovaj tekst nece završiti u terminalu!
```

Provjera if (newfd != fd) close(fd) štiti nas od posebnog slučaja kad bi dup() slučajno vratio isti broj kao stari fd (tada bi close(fd) zatvorio jedini preostali deskriptor).

Iako program radi, sadrži suptilan problem: pozivi `close(1)` i `dup(fd)` su **dvije odvojene operacije**. U višenitnom programu druga nit bi između njih mogla pozvati `open()` ili `creat()` i preuzeti upravo oslobođeni deskriptor 1 za sebe. Race condition kakav smo opisali ranije, ali na razini niti unutar istog procesa.

- **dup2_redirect.c** — isti zadatak, samo s `dup2()`.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd;

    fd = open("izlaz.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) != fd)
        close(fd);

    printf("Ovaj tekst završava u datoteci izlaz.txt!\n");

    return 0;
}
```

Ovo je preferirana varijanta jer:

- **atomski** zatvara odredišni deskriptor (`STDOUT_FILENO`) i postavlja novi na njegovo mjesto u jednoj nedjeljivoj operaciji,
- eksplicitno zadajemo odredišni deskriptor pa se ne oslanjamo na pretpostavku da je 1 najniži slobodni.

```
$ ./dup2_redirect
$ cat izlaz.txt
Ovaj tekst završava u datoteci izlaz.txt!
```

U produkcijskom kodu uvijek se koristi `dup2()` (ili moderniju `dup3()`). Pri pisanju programa koji preusmjeravaju ulaz/izlaz — npr. ljusku koja izvršava program > datoteka — ovaj uzorak je standardna tehnika.

Bitna napomena: čak i kad u programu koristimo `printf`, koji je funkcija standardne C biblioteke, ona se u konačnici svodi na sistemski poziv `write()` s `STDOUT_FILENO` (tj. 1) kao prvim argumentom. Stoga preusmjeravanje deskriptora 1 utječe i na `printf` ispis — što je ono što i koristimo u gornjim primjerima.

Prevođenje

Direktorij dolazi s priloženim Makefile-om koji prati iste konvencije kao i Makefile u `osnove_programiranja/` (varijable `CC`, `CFLAGS`, `LDLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`). Detaljan opis strukture i korake gradnje Makefilea vidjeti u [../osnove_programiranja/README.md](#).

Tipična uporaba:

```
make          # gradi zadani cilj (perm_mask)
make all      # gradi sve primjere
make f_strip  # gradi samo zadani primjer
make clean    # briše izvršne, objektne i generirane datoteke
```


Pravilo `clean` briše sve izvršne datoteke, objektne datoteke, privremene `*~` datoteke, `a.out`, kao i datoteke koje primjeri sami stvaraju pri pokretanju — `file.strip`, `file.hole`, `datoteka1`, `datoteka2` — kako bi se radni direktorij vratio u čisto stanje.

Upravljanje datotekama

U ovom poglavlju upoznat ćemo dublji UNIX pogled na datoteke: njihove **atribute**, kako jezgra organizira informacije o njima, te kako iz programa dohvatiti i mijenjati ta svojstva. Dok smo se u prethodnom poglavlju bavili **sadržajem** datoteka — čitanjem i pisanjem bajtova — ovdje stojimo “stepenicu više” i gledamo **metapodatke**: tip datoteke, vlasnika, prava pristupa, vremena pristupa, broj linkova, fizičku organizaciju na disku.

Tipovi datoteka

UNIX-ova filozofija “*sve je datoteka*” znači da kroz datotečno sučelje pristupamo različitim vrstama resursa. Sustav podržava sljedeće tipove datoteka:

Tip	Opis
regularna datoteka	“obična” datoteka — niz bajtova: tekstualne datoteke, binarni programi, slike, archive
direktorij	datoteka koja sadrži popis imena drugih datoteka i pripadnih i-node brojeva
simbolički link	datoteka koja sadrži putanju do druge datoteke (UNIX-ov “prečac”)
blok specijalna datoteka	uređaj kojemu se pristupa u blokovima fiksne veličine (npr. disk: /dev/sda)
karakter specijalna datoteka	uređaj kojemu se pristupa znak po znak (npr. terminal: /dev/tty)
FIFO (imenovani cjevovod)	jednosmjerni komunikacijski kanal između procesa, vidljiv u datotečnom sustavu
socket	krajnja točka za lokalnu ili mrežnu međuprocensnu komunikaciju

Bez obzira na tip, svim ovim resursima upravlja se istim malim skupom sistemskih poziva (open, read, write, close, lseek) — što je ujedno temelj UNIX-ove jednostavnosti i moći.

Posebno se zaustavljamo na **simboličkim linkovima** jer ćemo ih sresti kroz cijelo poglavlje. Simbolički link je u osnovi obična datoteka koja sadrži tekst — doslovno putanju do druge datoteke. Pri tom datoteka na koju link pokazuje ne mora uopće postojati: link će svejedno čuvati string koji predstavlja tu putanju, čak i ako na toj putanji nema ničega. Ako se datoteka na koju link pokazuje stvori naknadno, link će na nju automatski pokazivati. Ovaj mehanizam temelji se isključivo na razrješavanju puta — jezgra svaki put kad pristupa linku iznova čita njegov sadržaj i koristi ga kao putanju.

Svojstva datoteka

Svaka datoteka u UNIX sustavu, neovisno o tipu, ima skup **atributa** (engl. *file attributes*) koje jezgra održava neovisno o samom sadržaju datoteke: tko je vlasnik, kakva su prava

pristupa, kolika je veličina, kada je posljednji put mijenjana, koliko ima linkova... Iz programa, ovi se atributi dohvaćaju jednim od tri sistema poziva iz stat obitelji:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *path, struct stat *buf);
int fstat(int fd, struct stat *buf);
int lstat(const char *path, struct stat *buf);
```

Povratna vrijednost (sve tri funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Tri funkcije razlikuju se po načinu na koji identificiraju datoteku i po tome kako se ponašaju ako je riječ o simboličkom linku:

- **stat()** — datoteka se zadaje putanjom; ako je putanja simbolički link, funkcija ga slijedi i vraća attribute datoteke na koju link pokazuje (umjesto atributa za simbolički link — datoteku koja je navedena kao argument funkcije).
- **fstat()** — atributi otvorene datoteke; umjesto putanje kao argument se zadaje deskriptor na kojem je datoteka otvorena.
- **lstat()** — kao stat(), ali ako je putanja simbolički link, ne slijedi ga nego vraća attribute samog linka (njegovu veličinu, vrijeme stvaranja itd.).

U svim trima slučajevima, drugi argument je pokazivač na strukturu struct stat u koju jezgra upisuje attribute. Strukturu prethodno alocira pozivatelj (najčešće kao lokalnu varijablu na stogu).

Struktura struct stat

```
struct stat {
    mode_t    st_mode;    // tip datoteke i prava pristupa
    ino_t      st_ino;     // i-node broj
    dev_t      st_dev;     // identifikator uređaja na kojem datoteka leži
    dev_t      st_rdev;    // identifikator uređaja (samo za specijalne datoteke)
    nlink_t    st_nlink;   // broj hard linkova
    uid_t      st_uid;     // user ID vlasnika
    gid_t      st_gid;     // group ID vlasnika
    off_t      st_size;    // velicina datoteke u bajtovima
    time_t     st_atime;    // vrijeme zadnjeg pristupa sadržaju
    time_t     st_mtime;    // vrijeme zadnje izmjene sadržaja
    time_t     st_ctime;    // vrijeme zadnje promjene statusa (atributa)
    blksize_t  st_blksize; // optimalna velicina I/O bloka
    blkcnt_t   st_blocks;  // broj alociranih blokova na disku
};
```

Pojašnjenje pojedinih polja:

- **st_mode** — sadrži dvije informacije u istom polju: tip datoteke (regularna, direktorij, link...) i prava pristupa (devet bitova rwxrwxrwx plus posebni bitovi). Tip se ne čita izravno iz bročane vrijednosti, nego pomoću posebnih makro funkcija opisanih u nastavku.
- **st_ino** — broj i-node zapisa, jedinstvene strukture u kojoj jezgra čuva sve attribute datoteke. Dvije datoteke s istim i-node brojem na istom datotečnom sustavu su zapravo **ista datoteka** (vidi hard linkove).
- **st_dev** — identifikator uređaja (datotečnog sustava) na kojem datoteka leži. Par (st_dev, st_ino) jedinstveno identificira datoteku na cijelom sustavu.
- **st_rdev** — koristi se samo za blok i karakter specijalne datoteke (uređaje); u njemu su kodirani tzv. *major* i *minor* brojevi uređaja, što izlazi izvan onoga što nas u ovom poglavlju zanima.

- **st_nlink** — broj hard linkova koji pokazuju na ovaj i-node. Datoteka može imati više imena u datotečnom sustavu i sva su ravnopravna — niti jedno nije “izvorno” ili “primarno”, svaki je obični zapis u nekom direktoriju koji upućuje na isti i-node. Kad brojač padne na 0 (kad se obriše i zadnje ime), datoteka se fizički briše s diska.
- **st_uid, st_gid** — brojčani identifikatori vlasnika i grupe vlasnika. Tekstualna imena (npr. dkrst, users) dohvaćaju se naknadno iz sistemskih datoteka /etc/passwd i /etc/group.
- **st_size** — veličina sadržaja u bajtovima. Za regularne datoteke je to broj bajtova, za direktorije veličina ovisi o implementaciji, a za simboličke linkove — sjetimo se ranije napomene da je link u osnovi tekstualna datoteka — to je broj znakova u stringu putanje koji link sadrži.
- **st_atime** — vrijeme zadnjeg **pristupa sadržaju** (čitanja). Neke UNIX implementacije ne ažuriraju ovo polje pri svakom čitanju zbog performansi.
- **st_mtime** — vrijeme zadnje **izmjene sadržaja** (pisanja u datoteku).
- **st_ctime** — vrijeme zadnje promjene **statusa** datoteke: ne sadržaja, nego nekog atributa (prava pristupa, vlasništvo, broj linkova). Razlika prema mtime je suptilna ali važna: chmod mijenja ctime ali ne mtime.
- **st_blksize** — optimalna veličina jedinice I/O prijenosa za ovu datoteku; često 4096 B na suvremenim sustavima.
- **st_blocks** — broj 512-bajtnih blokova fizički alociranih datoteci. Kod *rijetkih* datoteka (“rupa”) može biti znatno manji od $st_size / 512$.

Brojač linkova kod direktorija

Bilo bi prirodno očekivati da svaki novostvoreni direktorij ima `st_nlink` jednak 1 — ipak je riječ o samo jednom imenu u nadređenom direktoriju. Ipak, već u trenutku stvaranja, brojač linkova svakog direktorija je 2. Razlog je u dva posebna unosa koja jezgra automatski stvara unutar svakog direktorija: `.` (link na sam taj direktorij) i `..` (link na nadređeni). Tako svaki direktorij na samom početku ima dva imena koja na njega upućuju — jedno u direktoriju u kojem se promatrani direktorij nalazi (njegovo “stvarno” ime) i jedno unutar njega samoga (`.`).

Štoviše, svaki put kad se unutar nekog direktorija stvori novi poddirektorij, brojač linkova raste još za jedan jer poddirektorij sa sobom donosi i `..` koji pokazuje natrag. Pogledajmo to izravno u ljusci:

```
$ mkdir test_dir
$ cd test_dir
$ ls -al
total 8
drwxr-xr-x  2 dkrst users 4096 May  5 15:41 .
drwxr-xr-x  5 dkrst users 4096 May  5 15:41 ..
```

Tek što je stvoren, `test_dir` ima `nlink = 2` (drugi stupac u retku za `.`). Sad stvorimo poddirektorij:

```
$ mkdir poddir
$ ls -al
total 12
drwxr-xr-x  3 dkrst users 4096 May  5 15:41 .
drwxr-xr-x  5 dkrst users 4096 May  5 15:41 ..
drwxr-xr-x  2 dkrst users 4096 May  5 15:41 poddir
```

Brojač linkova za `test_dir` (vidljiv u retku za `.`) sad je 3, jer mu je `poddir/..` dodao novu referencu. Sam `poddir` također kreće od 2. Općenito vrijedi da brojač linkova direktorija jednak je $2 + \text{broj poddirektorija}$ — što ujedno objašnjava zašto naredba `find -type d -links 2` (ili slične formulacije) može poslužiti za pronalazak direktorija koji nemaju poddirektorija.

Otkrivanje tipa datoteke

Tip datoteke kodiran je u `st_mode` polju, ali ne kao broj — provjera se radi pomoću makro funkcija definiranih u `<sys/stat.h>`:

Makro	Vraća true ako je datoteka...
<code>S_ISREG(m)</code>	regularna
<code>S_ISDIR(m)</code>	direktorij
<code>S_ISLNK(m)</code>	simbolički link
<code>S_ISBLK(m)</code>	blok specijalna
<code>S_ISCHR(m)</code>	karakter specijalna
<code>S_ISFIFO(m)</code>	FIFO (imenovani cjevovod)
<code>S_ISSOCK(m)</code>	socket

Tipičan obrazac uporabe: `if (S_ISREG(st.st_mode)) { ... }`.

Ispis atributa datoteke

- **fileinfo.c** — prima ime datoteke kao argument naredbenog retka i ispisuje njene osnovne attribute pozivom `stat()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>

int main(int argc, char *argv[]) {
    struct stat st;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (stat(argv[1], &st) < 0) {
        perror("stat");
        return 1;
    }

    printf("Datoteka: %s\n", argv[1]);
    printf(" tip: ");
    if (S_ISREG(st.st_mode)) printf("regularna datoteka\n");
    else if (S_ISDIR(st.st_mode)) printf("direktorij\n");
    else if (S_ISLNK(st.st_mode)) printf("simbolicki link\n");
    else if (S_ISCHR(st.st_mode)) printf("karakter specijalna\n");
    else if (S_ISBLK(st.st_mode)) printf("blok specijalna\n");
    else if (S_ISFIFO(st.st_mode)) printf("FIFO\n");
    else if (S_ISSOCK(st.st_mode)) printf("socket\n");

    printf(" i-node broj: %ld\n", (long)st.st_ino);
    printf(" prava: %o\n", st.st_mode & 0777);
    printf(" vlasnik UID: %d\n", st.st_uid);
    printf(" grupa GID: %d\n", st.st_gid);
    printf(" velicina: %ld B\n", (long)st.st_size);
    printf(" broj linkova: %ld\n", (long)st.st_nlink);
    printf(" zadnji pristup: %s", ctime(&st.st_atime));
```

```

printf("  zadnja izmjena: %s", ctime(&st.st_mtime));
printf("  zadnja promjena: %s", ctime(&st.st_ctime));

return 0;
}

```

Konverzija %o ispisuje prava pristupa u oktalnom obliku (zajednički prikaz prava na UNIX sustavima — npr. 644 za rw-r--r--, 755 za rwxr-xr-x). Funkcija ctime() iz <time.h> pretvara time_t (sekunde od epohe, 1.1.1970) u čitljivi datum/vrijeme. Eksplicitne pretvorbe ((long)st.st_ino) nužne su jer su ino_t, off_t i drugi tipovi različiti na različitim sustavima — bez pretvorbe printf može ispisati pogrešne brojeve.

Pokretanje:

```

$ ./fileinfo fileinfo.c
Datoteka: fileinfo.c
tip:          regularna datoteka
i-node broj:  165
prava:        644
vlasnik UID:  1000
grupa GID:    1000
velicina:     1300 B
broj linkova: 1
zadnji pristup: Tue May  5 14:45:04 2026
zadnja izmjena: Tue May  5 14:44:43 2026
zadnja promjena: Tue May  5 14:44:43 2026

```

Pokušajte i s drugim vrstama datoteka:

```

$ ./fileinfo /etc          # direktorij
$ ./fileinfo /dev/null     # karakter specijalna
$ ./fileinfo /dev/sda      # blok specijalna (ako postoji)

```

Ponašanje na simboličkom linku

Stvorimo iz ljuske simbolički link drugoime.c koji pokazuje na fileinfo.c:

```

$ ln -s fileinfo.c drugoime.c
$ ls -l fileinfo.c drugoime.c
lrwxrwxrwx 1 dkrst users  10 May  5 16:14 drugoime.c -> fileinfo.c
-rw-r--r-- 1 dkrst users 1300 May  5 15:22 fileinfo.c

```

Iz ispisa ls -l vidimo da je drugoime.c zaista simbolički link (početno slovo l u prvom stupcu) koji pokazuje na fileinfo.c. Sad nad njim pokrenimo naš program:

```

$ ./fileinfo drugoime.c
Datoteka: drugoime.c
tip:          regularna datoteka
i-node broj:  63
prava:        644
vlasnik UID:  1000
grupa GID:    1000
velicina:     1300 B
broj linkova: 1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 15:22:01 2026
zadnja promjena: Tue May  5 16:14:38 2026

```

Iako smo u argumentu zadali drugoime.c, ispisani atributi pripadaju datoteci fileinfo.c — što vidimo po veličini od 1300 B (link sam ima samo 10 znakova) i tipu

“regularna datoteka”. To je posljedica činjenice da `stat()` **slijedi simbolički link** i vraća attribute datoteke na koju link pokazuje, ne samog linka.

Razlika `stat` i `lstat`

Kako vidjeti attribute samog linka, a ne datoteke na koju on pokazuje? Tu na scenu stupa funkcija `lstat()`. Da bismo ilustrirali razliku, modificirajmo `fileinfo.c` tako da koristi `lstat` umjesto `stat` — sve ostalo neka ostane isto.

- **fileinfo2.c** — identičan programu `fileinfo.c`, jedina razlika je u tome što umjesto `stat()` koristi `lstat()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>

int main(int argc, char *argv[]) {
    struct stat st;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (lstat(argv[1], &st) < 0) {
        perror("lstat");
        return 1;
    }

    printf("Datoteka: %s\n", argv[1]);
    printf(" tip:          ");
    if (S_ISREG(st.st_mode)) printf("regularna datoteka\n");
    else if (S_ISDIR(st.st_mode)) printf("direktorij\n");
    else if (S_ISLNK(st.st_mode)) printf("simbolički link\n");
    else if (S_ISCHR(st.st_mode)) printf("karakter specijalna\n");
    else if (S_ISBLK(st.st_mode)) printf("blok specijalna\n");
    else if (S_ISFIFO(st.st_mode)) printf("FIFO\n");
    else if (S_ISSOCK(st.st_mode)) printf("socket\n");

    printf(" i-node broj:   %ld\n", (long)st.st_ino);
    printf(" prava:         %o\n", st.st_mode & 0777);
    printf(" vlasnik UID:   %d\n", st.st_uid);
    printf(" grupa GID:    %d\n", st.st_gid);
    printf(" velicina:      %ld B\n", (long)st.st_size);
    printf(" broj linkova:  %ld\n", (long)st.st_nlink);
    printf(" zadnji pristup: %s", ctime(&st.st_atime));
    printf(" zadnja izmjena: %s", ctime(&st.st_mtime));
    printf(" zadnja promjena: %s", ctime(&st.st_ctime));

    return 0;
}
```

Usporedimo izlaz oba programa nad našim simboličkim linkom `drugoime.c`:

```
$ ./fileinfo drugoime.c
Datoteka: drugoime.c
tip:          regularna datoteka
i-node broj:  63
```

```

prava:          644
vlasnik UID:    1000
grupa GID:      1000
velicina:       1300 B
broj linkova:   1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 15:22:01 2026
zadnja promjena: Tue May  5 16:14:38 2026

$ ./fileinfo2 drugoime.c
Datoteka: drugoime.c
tip:          simbolicki link
i-node broj:  81
prava:        777
vlasnik UID:  1000
grupa GID:    1000
velicina:     10 B
broj linkova:  1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 16:14:39 2026
zadnja promjena: Tue May  5 16:14:39 2026

```

Razlika je sad očita. `fileinfo` (koristi `stat`) slijedi simbolički link i pokazuje attribute datoteke `fileinfo.c` — i-node 63, veličina 1300 B, tip “regularna datoteka”. `fileinfo2` (koristi `lstat`) ostaje na samom linku — i-node 81, veličina 10 B (koliko je dugačak string “`fileinfo.c`”), tip “simbolički link”.

Koju ćemo funkciju koristiti, `stat` ili `lstat`, ovisi o tome želimo li dobiti informaciju da je određena datoteka simbolički link te njezine attribute, ili informaciju o datoteci na koju simbolički link pokazuje. Odabir ovisi o namjeni našeg programa.

Međutim, potrebno je voditi računa o jednoj posebnoj situaciji koja se javlja kod programa koji pretražuju datotečni sustav (npr. `find` ili rekurzivni obilazak stabla direktorija). Ako u nekom direktoriju postoji simbolički link koji pokazuje natrag na taj isti direktorij — ili na bilo kojeg pretka u stablu — `stat()` će takav link slijediti i naš će program ponovno “ući” u već obišteni direktorij. Pri svakom novom ulasku link je opet tu, pa program ponovno ulazi, i tako u nedogled. Da bismo ovakvu beskonačnu petlju izbjegli, prilikom pretraživanja datotečnog sustava treba koristiti `lstat()` — koji će za simbolički link to upravo i prijaviti, pa naš program može odlučiti da ga ne slijedi.

Korisnici, grupe i prava pristupa

UNIX je višekorisnički sustav, što znači da na istom računalu istovremeno može raditi više korisnika. Svakom korisniku pridružen je jedinstveni cjelobrojni **UID** (*User ID*), a svaki korisnik pripada jednoj ili više **grupa**, od kojih svaka ima svoj **GID** (*Group ID*). Mapiranje imena u brojeve nalazi se u sistemskim datotekama `/etc/passwd` (korisnici) i `/etc/group` (grupe).

Kao što smo upoznali u prvom poglavlju, prava pristupa svakoj datoteci dijele se u tri razine — vlasnik (*user*), grupa (*group*), ostali (*others*) — i tri tipa prava: čitanje (*r*), pisanje (*w*), izvršavanje (*x*). Sveukupno **devet bitova** koji se zapisuju u `st_mode` polje strukture `stat`.

Procesi i njihovo vlasništvo

Vlasništvo nije samo svojstvo datoteka — nego i procesa. Uostalom, već smo naučili da je na UNIX-u sve datoteka. Vidjeli smo u prvom poglavlju da standardni izlaz jednog procesa možemo preusmjeriti na standardni ulaz drugog i tako efektivno “pisati” u proces. Stoga je sasvim prirodno da i procesi imaju svog vlasnika i grupu kojoj pripadaju.

Svaki proces u sustavu ima pridruženu skupinu identifikatora koji određuju “u čije ime” radi. Najvažnija dva su:

- **stvarni** (*real*) **UID i GID** — tko je doslovno pokrenuo proces; preuzima se od ljuske u kojoj je naredba upisana.
- **efektivni** (*effective*) **UID i GID** — koje ovlasti proces zapravo ima u trenutku provjere.

Stvarni i efektivni vlasnik, kao i stvarno i efektivno grupno vlasništvo, u najvećem se broju slučajeva ne razlikuju. Međutim, postoje određene situacije u kojima se efektivno i stvarno vlasništvo mogu razlikovati. Važno je zapamtiti da jezgra prilikom pristupa datotekama uvijek provjerava prava **efektivnog** vlasnika i efektivne grupe koji su vlasnici procesa.

Logika pridruživanja vlasnika novom procesu prirodno slijedi iz toga kako razmišljamo o korištenju alata općenito: kada koristimo neki alat, sve posljedice — dobre i loše — našeg rada s tim alatom idu nama, nikako ne proizvođaču alata. **Stvarni vlasnik procesa u memoriji stoga je uvijek onaj tko je proces pokrenuo**, ne onaj tko je napisao izvršnu datoteku. Efektivni vlasnik je gotovo uvijek također onaj tko je proces pokrenuo, ali postoje specifične situacije u kojima efektivni vlasnik (i/ili efektivna grupa) može biti onaj korisnik sustava koji je vlasnik izvršne datoteke. Često je to root (superuser) koji ima neograničena prava na sustavu.

Klasičan primjer je promjena lozinke. Kad korisnik utipka naredbu `passwd`, ona mora upisati novu kriptiranu lozinku u datoteku `/etc/shadow`. Iz očiglednih sigurnosnih razloga, čitanje i pisanje u datoteku `/etc/shadow` dozvoljeno je samo root-u — niti jedan običan korisnik ne smije ni pročitati tuđe lozinke ni napisati novu:

```
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 609 Apr 18 18:13 /etc/shadow
```

Ipak, svaki korisnik mora moći promijeniti **vlastitu** lozinku. Rješenje je **set-UID bit** — poseban bit u pravima izvršne datoteke koji jezgri kaže: “*kad se ovaj program pokrene, postavi efektivni UID procesa na UID vlasnika izvršne datoteke, ne na UID korisnika koji ga je pokrenuo*”. Set-UID bit u ispisu naredbe `ls -l` zauzima mjesto izvršnog bita za vlasnika i prikazuje se slovom `s`:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30 2024 /usr/bin/passwd
```

Datoteka `/usr/bin/passwd` u vlasništvu je root-a (četvrti i peti stupac) i ima postavljen set-UID bit (`s` umjesto `x` u korisničkim pravima). Posljedica je da se svaki put kad ju običan korisnik pokrene proces izvršava s **efektivnim UID-om root-a** (i može mijenjati `/etc/shadow`), dok **stvarni UID ostaje korisnikov** (pa program zna tko ga je pokrenuo i može mu mijenjati samo njegovu lozinku).

Set-UID bit moguće je postaviti i na izvršne datoteke koje sami napišemo — pomoću naredbe `chmod` (s kojom ćemo se pobliže upoznati malo kasnije u poglavlju). Pri tome treba biti **iznimno oprezan**: svaki nedostatak u programu s postavljenim set-UID bitom potencijalno se može iskoristiti tako da napadač izvrši proizvoljan kod s povišenim ovlastima. Zato se set-UID koristi samo na pažljivo provjerenim, minimalističkim alatima poput `passwd-a`, a u svakodnevnom programiranju se gotovo nikad ne koristi.

Kako će programi koji rade s povišenim ovlastima provjeriti smije li stvarni korisnik (onaj koji je proces pokrenuo) zaista raditi neku akciju? Tu na scenu stupa sistemski poziv `access`, kojeg upoznajemo u sljedećoj sekciji.

Provjera prava pristupa — `access()`

Za testiranje da li trenutni proces ima određena prava nad datotekom (čitanje, pisanje, izvršavanje, ili samo provjera postojanja), POSIX nudi sistemski poziv `access`.

```
#include <unistd.h>

int access(const char *pathname, int mode);
```

Povratna vrijednost: 0 ako su sva tražena prava ostvarena (ili datoteka postoji u slučaju F_OK); -1 ako barem jedno pravo nije dopušteno ili je došlo do greške.

Argumenti:

- **pathname** — putanja do datoteke koja se provjerava.
- **mode** — kombinacija (bitovni OR) jedne ili više konstanti:
 - R_OK — provjeri pravo čitanja,
 - W_OK — provjeri pravo pisanja,
 - X_OK — provjeri pravo izvršavanja,
 - F_OK — provjeri samo postojanje datoteke (zanemaruje prava).

Bitno je razumjeti da access provjerava prava pomoću **stvarnog** UID-a i GID-a procesa, ne efektivnog. Razlika postaje važna kod programa s postavljenim set-UID bitom (npr. passwd): access će vratiti odgovor kakav bi imao stvarni korisnik, ne onaj kojeg je program privremeno preuzeo. Ovo je namjerno ponašanje — daje set-UID programima način da provjere “smije li to korisnik koji me je pokrenuo zaista raditi”.

- **provjeri.c** — prima ime datoteke kao argument i ispisuje koja prava (čitanje, pisanje, izvršavanje) trenutni korisnik nad njom ima.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (access(argv[1], F_OK) < 0) {
        printf("Datoteka '%s' ne postoji.\n", argv[1]);
        return 1;
    }

    printf("Prava korisnika nad datotekom '%s':\n", argv[1]);
    printf("  citanje:      %s\n", access(argv[1], R_OK) == 0 ? "DA" : "NE");
    printf("  pisanje:      %s\n", access(argv[1], W_OK) == 0 ? "DA" : "NE");
    printf("  izvrsavanje: %s\n", access(argv[1], X_OK) == 0 ? "DA" : "NE");

    return 0;
}
```

Program prvo pozivom `access(argv[1], F_OK)` provjerava postoji li datoteka uopće — to je preporučena praksa jer `access` s drugim zastavicama ne razlikuje “datoteka ne postoji” od “datoteka postoji ali nemam prava”. Ako datoteka postoji, slijede tri zasebne provjere za svako od tri prava.

Pokretanje (kao običan korisnik):

```
$ ./provjeri /etc/passwd
Prava korisnika nad datotekom '/etc/passwd':
  citanje:      DA
  pisanje:      NE
  izvrsavanje:  NE

$ ./provjeri /etc/shadow
Prava korisnika nad datotekom '/etc/shadow':
  citanje:      NE
  pisanje:      NE
  izvrsavanje:  NE
```

```
$ ./provjeri ./provjeri
Prava korisnika nad datotekom './provjeri':
  citanje:      DA
  pisanje:      DA
  izvršavanje: DA
```

```
$ ./provjeri nepostoji.txt
Datoteka 'nepostoji.txt' ne postoji.
```

Iz primjera je vidljivo da običan korisnik može čitati /etc/passwd (koji sadrži korisnička imena i UID-ove), ali ne i /etc/shadow (koji sadrži kriptirane lozinke i kojem pristup ima samo root).

Promjena prava pristupa — chmod() i fchmod()

```
#include <sys/stat.h>
```

```
int chmod(const char *path, mode_t mode);
int fchmod(int fd, mode_t mode);
```

Povratna vrijednost (obje funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **path** — putanja do datoteke (chmod).
- **fd** — otvoreni file deskriptor (fchmod).
- **mode** — nova prava pristupa, zadana kao bitovni OR konstanti tipa mode_t (S_IRUSR | S_IWUSR | ...) ili kao oktalni broj (0644, 0755).

Da bi proces smio mijenjati prava pristupa neke datoteke, mora biti zadovoljen jedan od dva uvjeta: efektivni UID procesa mora biti jednak UID-u vlasnika datoteke, ili proces mora biti pokrenut kao root.

- **prava.c** — jednostavna inačica naredbe chmod iz ljuske: prima oktalni zapis prava pristupa i ime datoteke, te postavlja zadana prava.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    mode_t mode;

    if (argc != 3) {
        printf("koristenje: %s <oktalna_prava> <ime_datoteke>\n", argv[0]);
        printf("primjer: %s 644 dat.txt\n", argv[0]);
        return 1;
    }

    /* strtol s bazom 8 - korisnik zadaje oktalni zapis poput 644 */
    mode = (mode_t)strtol(argv[1], NULL, 8);

    if (chmod(argv[2], mode) < 0) {
        perror("chmod");
        return 1;
    }

    printf("Prava datoteke '%s' postavljena na %o.\n", argv[2], mode);
}
```

```
return 0;
}
```

Funkcija `strtol` iz standardne C biblioteke pretvara string u cijeli broj; treći argument (8) određuje broječnu bazu — za oktalne brojeve to je 8. (Mogli bismo koristiti i `strtol(..., 0)` koji automatski prepoznaje bazu prema prefiksu: 0 za oktalni, 0x za heksadekadski, ali smo se odlučili za eksplicitnu bazu radi jasnoće.)

Ekvivalent u ljusci je naredba `chmod` koja prihvaća iste oktalne brojeve:

```
$ ls -l dat.txt
-rw-r--r-- 1 dkrst users 5 May  5 15:00 dat.txt
```

```
$ ./prava 600 dat.txt
Prava datoteke 'dat.txt' postavljena na 600.
```

```
$ ls -l dat.txt
-rw----- 1 dkrst users 5 May  5 15:00 dat.txt
```

```
$ chmod 644 dat.txt          # ekvivalentno ./prava 644 dat.txt
$ ls -l dat.txt
-rw-r--r-- 1 dkrst users 5 May  5 15:00 dat.txt
```

Promjena vlasništva — `chown()`, `fchown()`, `lchown()`

```
#include <unistd.h>

int chown(const char *path, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
int lchown(const char *path, uid_t owner, gid_t group);
```

Povratna vrijednost (sve tri funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **path** — putanja do datoteke (`chown`, `lchown`).
- **fd** — otvoreni file deskriptor (`fchown`).
- **owner** — novi UID vlasnika; ako se zadaje -1, vlasnik se ne mijenja.
- **group** — novi GID grupe; ako se zadaje -1, grupa se ne mijenja.

Razlika između `chown` i `lchown` analogna je razlici između `stat` i `lstat`: `chown` slijedi simbolički link i mijenja vlasništvo ciljne datoteke, dok `lchown` mijenja vlasništvo samog linka.

Bitna napomena: na većini suvremenih UNIX sustava, **promjena vlasništva ograničena je na korisnika root**. Običan korisnik **ne može** “preuzeti” tuđu datoteku (jer bi to predstavljao sigurnosni rizik), niti svoju datoteku “predati” drugome (jer bi mogućnost da nekome “uvalimo” kukavičje jaje također predstavljala sigurnosni rizik, ali i omogućilo zaobilaženje kvota i drugih ograničenja). U praksi to znači da običan korisnik ove systemske pozive uglavnom ne koristi izravno; oni su prvenstveno alat administratora. Ekvivalent u ljusci je naredba `chown`. Pošto promjena vlasništva traži root ovlasti, naredba se najčešće poziva uz `sudo` — pomoćnu naredbu koja omogućuje da se ostatak naredbe izvrši s ovlastima root korisnika, pod uvjetom da korisnik ima takvo pravo (tipično definirano u datoteci `/etc/sudoers`):

```
$ sudo chown dkrst:users dat.txt # dodjeljuje datoteku korisniku dkrst, grupi users
```

Linkovi

UNIX podržava dva različita načina da jedna te ista datoteka bude dostupna pod više imena: **čvrste linkove** (engl. *hard link*) i **simboličke linkove** (engl. *symbolic link* ili *soft link*). Iako oba mehanizma pružaju “alias” za neku datoteku, oni rade na suštinski različite načine.

Čvrsti linkovi

Promislimo na trenutak o ulozi direktorija u datotečnom sustavu. Najjednostavnije ih možemo zamisliti kao registre neke velike arhive u kojima piše gdje se koji podatak nalazi: *“ako tražiš tu i tu mapu, pogledaj na trećoj polici lijevo, druga kutija od kraja”*. Nema nikakvog razloga da ista informacija o tome gdje se neki podatak nalazi ne piše u više različitih registara. U svakom od tih registara podatak je možda drugačije zaveden (recimo, računovodstvo fakulteta možda sistematizira studente na jedan, a referada na drugi način). Međutim, u svakom od njih zabilježena je ista lokacija mape na polici, i bez obzira u kojem registru tražimo, doći ćemo do istih podataka.

Na sličan način, iako datoteku čini jedan jedinstveni sadržaj pohranjen na disku, ne postoji nikakav razlog da informaciju o tome gdje se taj sadržaj nalazi ne pohranimo u više direktorija, pod istim ili različitim imenima. Bez obzira koje ime koristimo, sva ona u konačnici pokazuju na iste fizičke podatke na disku.

Tehnički, **čvrsti link** je upravo to: dodatni zapis u nekom direktoriju koji povezuje neko ime s istim i-node brojem kao već postojeća datoteka. Sjetimo se da u UNIX-u sve attribute datoteke (tip, prava, veličinu, vremena, lokaciju podataka na disku) jezgra čuva u i-node strukturi, a ime datoteke u nekom direktoriju je samo zapis koji to ime povezuje s odgovarajućim i-node brojem. Stvaranjem čvrstog linka jednostavno se dodaje novi takav zapis koji pokazuje na isti i-node — pa s gledišta jezgre, sva imena su jednako vrijedne reference na istu datoteku.

Iz ovog mehanizma slijedi nekoliko važnih svojstava čvrstih linkova:

- **Svi (čvrsti) linkovi pokazuju na isti i-node**, dijele identičan sadržaj i attribute. Promjena sadržaja kroz jedno ime vidljiva je odmah kroz drugo.
- **Svi (čvrsti) linkovi moraju biti na istom datotečnom sustavu** (istoj particiji), jer i-node brojevi vrijede samo unutar pojedinog datotečnog sustava.
- **Brisanjem nekog imena samo se uklanja njegov zapis u direktoriju** i smanjuje brojač linkova (`st_nlink`) na i-nodeu. Sama datoteka briše se s diska tek kad brojač padne na 0 — odnosno kad je obrisano i zadnje ime, jer više ništa ne “pokazuje” na te podatke.
- **Izvorno ime nije ničim “jače” od kasnije stvorenih linkova**. Brisanje izvornog imena je jednako brisanju bilo kojeg drugog linka.
- **Samo root smije stvarati čvrste linkove na direktorij**. Stvaranje takvog linka kreiralo bi cikluse u stablu direktorija — primjerice, link `/foo/bar` koji pokazuje natrag na `/foo` značio bi da iz direktorija `/foo` možemo unedogled “spuštati” se kroz `bar/bar/bar/...` i uvijek smo u istom direktoriju. Alati poput `find` ili `du` koji rekurzivno obilaze stablo zaglavili bi u beskonačnoj petlji. Slično vrijedi i za posebne unose `.` i `..` koji su u biti čvrsti linkovi, ali njih jezgra automatski stvara i njima upravlja sama.

```
#include <unistd.h>
```

```
int link(const char *oldpath, const char *newpath);  
int unlink(const char *pathname);
```

Povratna vrijednost (obje): 0 u slučaju uspjeha, -1 u slučaju greške.

`link()` stvara novo ime (`newpath`) za već postojeću datoteku (`oldpath`) — povećava brojač linkova na i-nodeu i atomski dodaje novi zapis u direktorij. `unlink()` radi obrnuto: uklanja zapis iz direktorija i smanjuje brojač. Ako brojač padne na 0 i nema otvorenih deskriptora na datoteku, jezgra fizički briše datoteku s diska.

Obje funkcije imaju izravne ekvivalente u ljusci: `link()` odgovara naredbi `ln`, a `unlink()` naredbi `unlink` ili — što je u praksi puno češće — naredbi `rm`. Naredba `rm` nudi sve što i `unlink`, ali i puno više: brisanje više datoteka odjednom, rekurzivno brisanje s opcijom `-r`, prisilno brisanje s `-f`. Ipak, s gledišta sustava, one rade istu temeljnu operaciju — uklanjaju zapis iz direktorija i smanjuju brojač linkova.

- **makelink.c** — stvara čvrsti link između dvije zadane putanje.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("koristenje: %s <postojeca_datoteka> <novi_link>\n", argv[0]);
        return 1;
    }

    if (link(argv[1], argv[2]) < 0) {
        perror("link");
        return 1;
    }

    printf("Stvoren hard link '%s' -> '%s'.\n", argv[2], argv[1]);
    return 0;
}
```

Pokrenimo cjeloviti scenarij koji ilustrira sva spomenuta svojstva. Stvorit ćemo datoteku, dodati joj dva čvrsta linka, pa pratiti brojač linkova dok ne dođe do nule:

```
$ echo "vrijedan sadrzaj" > orig.txt
$ ls -li orig.txt
1002 -rw-r--r-- 1 dkrst users 17 May  5 15:00 orig.txt
```

```
$ ./makelink orig.txt link1.txt
Stvoren hard link 'link1.txt' -> 'orig.txt'.
$ ln orig.txt link2.txt
```

```
$ ls -li orig.txt link1.txt link2.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 link1.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 link2.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 orig.txt
```

Drugi link je stvoren naredbom `ln` u ljusci — ekvivalentno pozivu `./makelink orig.txt link2.txt`.

Sva tri imena imaju **isti i-node 1002** (prvi stupac, koji se ispisuje opcijom `-li`) i **brojač linkova jednak 3** (četvrti stupac). Sad obrišimo izvorno ime:

```
$ rm orig.txt
$ ls -li link1.txt link2.txt
1002 -rw-r--r-- 2 dkrst users 17 May  5 15:00 link1.txt
1002 -rw-r--r-- 2 dkrst users 17 May  5 15:00 link2.txt
```

```
$ cat link1.txt
vrijedan sadrzaj
```

Brojač je pao na 2, ali i-node, sadržaj i ostali atributi nepromijenjeni su — datoteka još uvijek živi pod druga dva imena. Brisanje “izvornog” imena nije imalo nikakve posebne posljedice. Pokušajmo dalje:

```
$ rm link1.txt
```

```
$ ls -li link2.txt
1002 -rw-r--r-- 1 dkrst users 17 May  5 15:00 link2.txt

$ cat link2.txt
vrijedan sadržaj

$ rm link2.txt
$ cat link2.txt
cat: link2.txt: No such file or directory
```

Tek kad je brisano i zadnje ime — i brojač pao na 0 — jezgra je fizički obrisala datoteku s diska.

Simbolički linkovi

Simbolički link je datoteka koja sadrži tekst — putanju do druge datoteke. Ono što razlikuje simbolički link od bilo koje druge tekstualne datoteke jest njegov **tip**, zapisan u atributima datoteke (točnije, u `st_mode` polju i-noda). Po tom tipu jezgra zna da sadržaj ne treba čitati kao obične podatke — nego da tekst u datoteci treba interpretirati kao putanju do druge datoteke i operaciju nastaviti nad njom. Tako pri svakom otvaranju, čitanju ili pisanju kroz simbolički link, jezgra čita njegov sadržaj i operaciju izvršava nad datotekom čije je ime ondje pročitano.

Simbolički linkovi pružaju funkcionalnost sličnu čvrstim linkovima, ali rade na potpuno drugačiji način, što ima nekoliko važnih posljedica:

- **Simbolički linkovi mogu prelaziti granice datotečnih sustava.** Sadržaj je samo tekst (putanja), tako da nema problema kao kod čvrstih linkova.
- **Mogu pokazivati na nepostojeće datoteke** — u trenutku stvaranja jezgra ne provjerava da li ciljna datoteka postoji. Ako ciljna datoteka nikad nije stvorena, ili je obrisana, link postaje “razbijen” (engl. *dangling*).
- **Mogu pokazivati na direktorije** — bez ikakvih ograničenja koja vrijede za čvrste linkove na direktorije.
- **Brisanjem ciljne datoteke linkovi ostaju na mjestu**, ali postaju razbijeni. Obrnuto: brisanjem linka ciljnoj datoteci se ništa ne događa.
- **Imaju vlastite attribute** (vlasnika, vremena) odvojene od ciljne datoteke. Veličina linka je broj bajtova u stringu putanje koji sadrži.

```
#include <unistd.h>
```

```
int symlink(const char *target, const char *linkpath);
ssize_t readlink(const char *pathname, char *buf, size_t bufsize);
```

symlink() stvara simbolički link na putanji `linkpath` koji “pokazuje” na `target`. Vraća 0 u slučaju uspjeha, -1 u slučaju greške.

readlink() čita sadržaj simboličkog linka — odnosno **tekst (putanju) zapisan u datoteci tipa simbolički link** — i smješta ga u zadani međuspremnik. Vraća broj stvarno pročitanih bajtova, ili -1 u slučaju greške. Bitna posebnost: **readlink ne dodaje null-terminator** na kraj — pozivatelj ga mora dodati ručno.

- **makesymlink.c** — stvara simbolički link na zadanoj putanji koji “pokazuje” na zadanu metu.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("koristenje: %s <postojeca_datoteka> <novi_link>\n", argv[0]);
        return 1;
    }
}
```



```

    }

    if (symlink(argv[1], argv[2]) < 0) {
        perror("symlink");
        return 1;
    }

    printf("Stvoren simbolicki link '%s' -> '%s'.\n", argv[2], argv[1]);
    return 0;
}

```

Pokretanje:

```

$ echo "ovo je sadrzaj" > orig.txt
$ ./makesymlink orig.txt sym.txt
Stvoren simbolicki link 'sym.txt' -> 'orig.txt'.

```

```

$ ls -li orig.txt sym.txt
1003 -rw-r--r-- 1 dkrst users 15 May  5 15:00 orig.txt
1004 lrwxrwxrwx 1 dkrst users  8 May  5 15:00 sym.txt -> orig.txt

```

```

$ cat sym.txt
ovo je sadrzaj

```

Bitno: i-node simboličkog linka (1004) različit je od i-noda ciljne datoteke (1003); brojač linkova je 1 na svakom (nisu hard linkovi); tip prvog je - (regularna), drugog l (link); veličina linka je 8 bajtova (duljina stringa "orig.txt"). Pokušajmo obrisati izvornu datoteku:

```

$ rm orig.txt
$ ls -l sym.txt
lrwxrwxrwx 1 dkrst users 8 May  5 15:00 sym.txt -> orig.txt
$ cat sym.txt
cat: sym.txt: No such file or directory

```

Link je sam ostao netaknut — ali ciljna datoteka više ne postoji, pa je link sad razbijen. Moderne UNIX ljuške razbijene (*dangling*) simboličke linkove često prikazuju drugom bojom kako bi odmah bilo vidljivo da putanja u linku pokazuje na nepostojeću datoteku.

Ekvivalent u ljusci je opet naredba `ln`, ovaj put s opcijom `-s`:

```

$ ln -s orig.txt sym.txt

```

Naredba `ln` ima i niz drugih opcija — ovdje smo upoznali samo dvije najčešće (osnovni oblik za hard linkove i `-s` za simboličke). Za potpunu uputu i popis svih opcija, kao i kod svih ostalih UNIX naredbi, čitatelj se upućuje na priručnik koji se otvara naredbom `man ln`.

- **readsymlink.c** — čita sadržaj simboličkog linka, odnosno doslovni tekst (putanju) koji je u njemu zapisan, pomoću funkcije `readlink`.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    char buf[256];
    ssize_t n;

    if (argc != 2) {
        printf("koristenje: %s <simbolicki_link>\n", argv[0]);
    }
}

```



```

    return 1;
}

/* readlink ne dodaje null terminator, pa ga moramo sami dodati */
n = readlink(argv[1], buf, sizeof(buf) - 1);
if (n < 0) {
    perror("readlink");
    return 1;
}
buf[n] = '\0';

printf("Sadržaj linka '%s': \"%s\"\n", argv[1], buf);
return 0;
}

```

Da bismo program isprobali, stvorimo jednu datoteku u poddirektoriju (kako bi imala nešto složeniju putanju), pa na nju usmjerimo dva simbolička linka pisana s različito formuliranom putanjom — jednog napravimo pozivom `makesymlink-a`, a drugog izravno iz ljuške naredbom `ln -s`:

```

$ mkdir -p dokumenti
$ echo "ovo je sadržaj" > dokumenti/orig.txt

$ ./makesymlink dokumenti/orig.txt sym1
Stvoren simbolički link 'sym1' -> 'dokumenti/orig.txt'.

$ ln -s ./dokumenti/orig.txt sym2

```

```

$ ls -l sym1 sym2
lrwxrwxrwx 1 dkrst users 19 May  5 17:23 sym1 -> dokumenti/orig.txt
lrwxrwxrwx 1 dkrst users 21 May  5 17:23 sym2 -> ./dokumenti/orig.txt

```

Već iz `ls -l` ispisa vidimo da oba linka pokazuju na istu datoteku, ali se njihove veličine razlikuju — `sym1` zauzima 19 bajtova (duljina niza `"dokumenti/orig.txt"`), a `sym2` 21 bajt (duljina niza `"./dokumenti/orig.txt"` — dva bajta više za uvodno `./`). Time je vidljivo da svaki link doslovno čuva onaj tekst kojim je stvoren.

Sad pročitajmo sadržaj oba linka pozivom `readsymlink`:

```

$ ./readsymlink sym1
Sadržaj linka 'sym1': "dokumenti/orig.txt"

$ ./readsymlink sym2
Sadržaj linka 'sym2': "./dokumenti/orig.txt"

```

Vidimo da nam program ispisuje upravo onaj tekst koji smo prosljedili kao putanju pri stvaranju linka — nezavisno o tome koristi li se `symlink()` u C-u ili `ln -s` u ljusci. Različita formulacija putanje (`dokumenti/orig.txt` vs `./dokumenti/orig.txt`) doslovno se čuva u sadržaju linka — jezgra niti normalizira putanju niti provjerava simboličke linkove pri stvaranju. Provjera se događa tek pri svakom pristupu kroz link.

Vremena pristupa

Već smo upoznali tri vremenska polja u struct `stat`: `st_atime`, `st_mtime`, `st_ctime`. Jezgra ih ažurira automatski — pri svakom čitanju, pisanju ili promjeni atributa datoteke. Ipak, ponekad želimo eksplicitno postaviti `atime` i `mtime` za određenu datoteku na neku zadanu vrijednost — najčešće da bismo “potvrdili” da je datoteka svjež, ili da Build sustavi (`make`) misle da je novija od neke druge.

```
#include <sys/types.h>
#include <utime.h>

int utime(const char *pathname, const struct utimbuf *times);

struct utimbuf {
    time_t actime;    // novo atime
    time_t modtime;   // novo mtime
};
```

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — datoteka kojoj mijenjamo vremena.
- **times** — pokazivač na strukturu s novim vremenima. Ako je NULL, oba vremena postavljaju se na **trenutno** vrijeme (što je zapravo ekvivalent pozivu naredbe touch bez argumenata koji zadaju vrijeme).

Pogledajmo kako se ova funkcija koristi u praksi:

- **dotakni.c** — pojednostavljena inačica naredbe touch: ako datoteka ne postoji, stvara je praznu; potom postavlja njena vremena na trenutno vrijeme.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <utime.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    int fd;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    /* Ako datoteka ne postoji, stvori je praznu */
    fd = open(argv[1], O_WRONLY | O_CREAT, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    close(fd);

    /* Postavi atime i mtime na trenutno vrijeme.
     * NULL kao drugi argument znaci "koristi trenutno vrijeme". */
    if (utime(argv[1], NULL) < 0) {
        perror("utime");
        return 1;
    }

    printf("Datoteka '%s' azurirana.\n", argv[1]);
    return 0;
}
```

Pokretanje:

```
$ ls nova.txt
```

```
ls: cannot access 'nova.txt': No such file or directory
```

```
$ ./dotakni nova.txt  
Datoteka 'nova.txt' azurirana.
```

```
$ ls -l nova.txt  
-rw-r--r-- 1 dkrst users 0 May  5 15:00 nova.txt
```

```
$ ./dotakni nova.txt          # vec postoji - samo azurira vremena  
Datoteka 'nova.txt' azurirana.
```

Ekvivalent u ljusci je naredba touch:

```
$ touch nova.txt          # ekvivalent gornjeg poziva
```

Naredba touch u praksi je nešto bogatija od našeg programa. Po zadanom postavlja **oba vremena** (atime i mtime) na trenutni trenutak, ali nudi i niz opcija za finiju kontrolu:

- **a** mijenja samo atime, dok mtime ostaje netaknut,
- **m** mijenja samo mtime, dok atime ostaje netaknut,
- **d** ili **-t** omogućuju zadavanje **proizvoljnog** datuma i vremena umjesto trenutnog (npr. touch -d "2024-01-15 10:30:00" dat.txt),
- **r** preslikava vremena s neke druge datoteke kao referentne (touch -r referenca.txt cilj.txt),
- **c** sprečava stvaranje datoteke ako ne postoji.

Funkcija utime u svojoj punoj verziji već ima podršku za sve ovo: drugi argument može biti pokazivač na struct utimbuf s eksplicitno zadanim vrijednostima atime i mtime (umjesto NULL koji znači "trenutno vrijeme"). Vrijednosti tipa time_t mogu se dobiti iz čovjeku čitljivog datuma pomoću funkcija mktime() ili strptime(). Naš dotakni.c dakle pokriva samo najjednostavniji slučaj; proširenje s navedenim opcijama može biti korisna **vježba za čitatelja** ako želi bolje razumjeti rad s vremenom u UNIX sustavima.

Tipična uporaba touch (i dotakni) — "natjerati" make da iznova prevede neki fajl, čak i ako mu sadržaj nije promijenjen, samo postavljanjem mtime na sadašnji trenutak.

Rad s direktorijima

U posljednjem dijelu poglavlja dat ćemo pregled funkcija koje u našim programima možemo koristiti za upravljanje datotečnim stablom — stvaranje i brisanje direktorija, kretanje kroz njih, te čitanje njihova sadržaja. Završit ćemo i s odgovarajućim primjerom korištenja kojim ćemo ilustrirati kako radi još jedna od standardnih UNIX naredbi koju iznimno često koristimo u svakodnevnom radu.

```
#include <unistd.h>  
#include <sys/stat.h>  
  
int mkdir(const char *pathname, mode_t mode);  
int rmdir(const char *pathname);  
  
char *getcwd(char *buf, size_t size);  
int chdir(const char *pathname);  
int fchdir(int fd);
```

Funkcija mkdir()

Stvara novi prazan direktorij. U njemu jezgra automatski stvara dva posebna unosa: . (pokazivač na sam taj direktorij) i .. (pokazivač na nadređeni direktorij).

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja na kojoj se stvara novi direktorij.
- **mode** — prava pristupa za novi direktorij (npr. 0755); konačna prava dobiju se kombinacijom s maskom procesa (umask).

Funkcija rmdir()

Briše direktorij — ali samo ako je **prazan** (sadrži samo . i ..). Ukoliko želimo obrisati direktorij u kojem se nalaze druge datoteke i direktoriji, potrebno je ući u njega i rekurzivno obrisati cijeli njegov sadržaj, datoteku po datoteku (sistemskim pozivom unlink) te tek nakon toga obrisati i sam direktorij.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija koji se briše.

Funkcija getcwd()

Vraća putanju do trenutnog radnog direktorija procesa (CWD - *Current Working Directory*). Bitno je razumjeti da je radni direktorij **svojstvo procesa**, ne korisnika; svaki proces ima svoj vlastiti CWD koji je naslijedio od svog roditelja u trenutku stvaranja (sistemskim pozivom fork()).

Povratna vrijednost: pokazivač na buf u slučaju uspjeha, NULL u slučaju greške (npr. ako je međuspremnik premali da primi punu putanju).

Argumenti:

- **buf** — međuspremnik u koji se upisuje putanja do trenutnog direktorija (kao null-terminirani string).
- **size** — veličina međuspremnika buf u bajtovima.

Funkcije chdir() i fchdir()

Mijenjaju radni direktorij procesa. chdir direktorij identificira putanjom, dok fchdir koristi otvoreni file deskriptor.

Bitan suptilan detalj: chdir u programu mijenja radni direktorij **samo tom procesu** — ne i ljusci koja ga je pokrenula. Zato naredba cd **nije implementirana kao “vanjska” naredba**, tj. ne postoji izvršna datoteka koja se poziva kad napišemo cd (za razliku od drugih naredbi, kao npr. ls, touch i brojnih drugih koje smo koristili). Ovako implementirana naredba bila bi beskorisna: pokretanje novog programa podrazumijeva i stvaranje novog procesa u kojem će se program izvršavati, dok ljuska čeka da on završi i prihvati sljedeću naredbu korisnika. Pozivanje funkcije chdir() u takvom novom procesu zapravo bi promijenilo trenutni radni direktorij (CWD) samo tog procesa, ne i ljuske koja je naredbu pozvala. Stoga je naredba cd realizirana kao ugrađena (*built-in*) naredba ljuske — ne izvršava se u novom procesu, nego mijenja CWD same ljuske u kojoj je pozvana, pozivom chdir() “iznutra”.

Povratna vrijednost (obje funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija (chdir).
- **fd** — otvoreni file deskriptor (fchdir).

Čitanje sadržaja direktorija

Direktoriji su zapravo posebne datoteke koje sadrže popis imena i pripadnih i-node brojeva. Jezgra dopušta samo sebi da piše u njih — korisnički procesi smiju ih čitati, ali ne i izravno mijenjati. Najčešći oblik rada s direktorijem — sekvencijalno čitanje svih zapisa u njemu — obavlja se trojkom funkcija iz `<dirent.h>`:

```
#include <dirent.h>

DIR      *opendir(const char *pathname);
struct dirent *readdir(DIR *dp);
int      closedir(DIR *dp);
```

Funkcija `opendir()`

Otvora direktorij za čitanje i vraća pokazivač na DIR strukturu — **neprozirni** (*opaque*) tip, što znači da programer ne zna i ne treba znati kako struktura iznutra izgleda; samo je prosljeđuje ostalim funkcijama obitelji kao apstraktnu referencu, dok upravljanje sadržajem strukture u potpunosti preuzima biblioteka.

Povratna vrijednost: pokazivač na DIR strukturu u slučaju uspjeha, NULL u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija koji se otvara.

Funkcija `readdir()`

Vraća zapis koji se nalazi na **trenutnoj poziciji** u direktoriju, a zatim tu poziciju pomiče na sljedeći zapis. Pozivom `opendir()` pozicija se postavlja na **prvi zapis** u direktoriju, pa prvi poziv `readdir`-a vraća upravo njega. Svaki sljedeći poziv vraća sljedeći zapis i pomiče poziciju za jedno mjesto naprijed; kad više nema zapisa, `readdir` vraća NULL. Time u jednoj `while` petlji možemo prirodno pročitati cijeli direktorij od početka do kraja.

Mehanizam je analogan onomu što smo već vidjeli kod običnih datoteka: tamo `read()` čita podatke od trenutnog **offseta** u datoteci i pomiče ga za broj pročitanih bajtova; ovdje `readdir()` čita zapis s trenutne pozicije u direktoriju i pomiče tu poziciju na sljedeći zapis. Razlika je u tome što kod regularnih datoteka offsetom programer može slobodno upravljati funkcijom `lseek()`, dok je kod direktorija interna pozicija dio neprozirne DIR strukture i nije izravno dostupna programeru — on je može samo vratiti na početak pozivom `rewinddir()`, ili koristiti `telldir()` i `seekdir()` o kojima ćemo govoriti malo kasnije.

Povratna vrijednost: pokazivač na `struct dirent` strukturu sa zapisom, ili NULL na kraju direktorija ili u slučaju greške.

POSIX standard zahtijeva da `struct dirent` sadrži samo **dva polja**:

```
struct dirent {
    ino_t  d_ino;      // i-node broj zapisa
    char   d_name[];   // ime datoteke (null-terminirano)
};
```

Polje `d_name` ima **neodređenu veličinu** — POSIX kaže samo “polje znakova koje sadrži ime od najviše `NAME_MAX` bajtova plus null-terminator”. Različite implementacije strukturu dopunjuju vlastitim poljima (`d_off`, `d_reclen`, `d_type` na Linuxu/glibc), ali ta polja **nisu prenosiva**. Zbog ovoga vrijedi nekoliko praktičnih napomena:

- **`sizeof(d_name)` ne radi pouzdano** — neke implementacije deklariraju polje kao `char d_name[1]`, druge kao `char d_name[256]`. Za pravu duljinu uvijek koristimo `strlen(d_name)`.
- **`sizeof(struct dirent)` ne predstavlja nužno stvarnu veličinu zapisa** — iz istog razloga kao gore.

- **Polje `d_type`** (regularna/direktorij/link/...) postoji na Linuxu, ali nije dio POSIX standarda. Ukoliko želimo pisati prenosivi kod, sigurnije je koristiti `lstat` sa imenom datoteke (koje možemo dobiti iz `d_name` polja) i nakon toga iz `st_mode` saznati tip datoteke.

Vraćeni pokazivač pokazuje na strukturu koju je sama biblioteka prethodno alocirala u svojoj internoj memoriji (najčešće u statičkom međuspremniku); pozivatelj ju nije ni alocirao, pa ju ni ne smije osloboditi pozivom `free()`. Sljedeći poziv `readdir`-a nad istim direktorijem prepíše taj međuspremnik novim zapisom — povratni pokazivač iz prethodnog poziva nakon toga još uvijek vrijedi, ali sadržaj na koji pokazuje više nije isti. Ako iz nekog razloga trebamo zadržati podatke iz zapisa duže (npr. spremiti listu svih imena u direktoriju), moramo ih kopirati u vlastitu memoriju.

Argumenti:

- **`dp`** — pokazivač na otvorenu DIR strukturu, dobiven prethodnim pozivom `opendir`.

Funkcija `closedir()`

Zatvara direktorij i oslobađa pripadne resurse.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **`dp`** — pokazivač na otvorenu DIR strukturu.

Veza sa standardnom C bibliotekom: uočite da je kombinacija `opendir` → `readdir` → `closedir` za direktorije analogna kombinaciji `fopen` → `fread` → `fclose` za regularne datoteke. U oba slučaja imamo “stream” apstrakciju — neprozirni objekt (DIR * odnosno FILE *) koji čuva interne podatke o napretku čitanja, plus funkcije za otvaranje, sekvencijalno čitanje i zatvaranje.

Pozicioniranje unutar direktorija

Pored osnovne trojke koju smo upravo opisali, `<dirent.h>` nudi i tri dodatne funkcije za izravno upravljanje internom pozicijom u direktoriju:

```
#include <dirent.h>

void rewinddir(DIR *dp);
long telldir(DIR *dp);
void seekdir(DIR *dp, long loc);
```

Funkcija `rewinddir()`

Vraća čitanje na početak direktorija — sljedeći poziv `readdir`-a opet će vratiti prvi zapis.

Povratna vrijednost: nema (funkcija je tipa void).

Argumenti:

- **`dp`** — pokazivač na otvorenu DIR strukturu.

Funkcije `telldir()` i `seekdir()`

`telldir` vraća trenutnu poziciju u direktoriju kao neprozirnu vrijednost; `seekdir` postavlja čitanje natrag na poziciju ranije zapamćenu `telldir`-om. Ove dvije funkcije rijetko se koriste u praksi.

Povratna vrijednost:

- **`telldir`** — trenutna pozicija (kao long vrijednost), ili -1 u slučaju greške.
- **`seekdir`** — nema povratne vrijednosti (void).

Argumenti:

- **dp** — pokazivač na otvorenu DIR strukturu.
- **loc** (samo seekdir) — pozicija ranije dobivena pozivom telldir.

Sada kad smo upoznali sve funkcije za rad s direktorijima, iskoristit ćemo ih u jednom konkretnom primjeru — implementaciji **pojednostavljene inačice standardne UNIX naredbe ls**, koja kao i izvorna naredba ispisuje sadržaj zadanog direktorija.

- **mojls.c** — pojednostavljena inačica naredbe ls. Bez argumenta lista trenutni direktorij; s argumentom lista zadani. Za svaki zapis ispisuje veličinu datoteke u bajtovima, te ime.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <dirent.h>

int main(int argc, char *argv[]) {
    DIR *dp;
    struct dirent *entry;
    struct stat st;
    char path[1024];
    const char *dir;

    /* bez argumenta, listamo trenutni direktorij */
    dir = (argc < 2) ? "." : argv[1];

    dp = opendir(dir);
    if (dp == NULL) {
        perror("opendir");
        return 1;
    }

    /* readdir vraca po jedan zapis u svakom pozivu;
     * NULL znaci kraj direktorija (ili greska) */
    while ((entry = readdir(dp)) != NULL) {
        /* preskoci skrivene datoteke (ukljucujuci . i ..) */
        if (entry->d_name[0] == '.')
            continue;

        /* sastavi punu putanju "dir/ime" za poziv lstat-u */
        snprintf(path, sizeof(path), "%s/%s", dir, entry->d_name);

        if (lstat(path, &st) < 0) {
            perror(path);
            continue;
        }

        printf("%10ld  %s\n", (long)st.st_size, entry->d_name);
    }

    closedir(dp);
    return 0;
}
```

Glavna petlja je tipična UNIX-ova “while-readdir” konstrukcija — funkcija u istom pozivu javlja i podatke (kroz povratnu vrijednost) i kraj pretrage (vraćanjem NULL-a). Ovakvu petlju, koja prolazi kroz niz podataka ili poziva zaključenih NULL pokazivačem,

često ćete sresti u programima.

Bitno: entry->d_name sadrži samo **ime zapisa unutar otvorenog direktorija**, ne njegovu punu putanju. lstat() međutim treba putanju — pa prije svakog poziva snprintf-om sastavljamo string oblika "<dir>/<ime>". Bez ovog koraka lstat bi tražio entry->d_name u trenutnom radnom direktoriju procesa, što bi naravno bilo pogrešno čim listamo neki drugi direktorij osim ".". Format "%10ld" u printf-u poravnava broj u stupac širine 10 znakova radi urednijeg ispisa.

Pokretanje:

```
$ ./mojls
      564 prava.c
      373 makesymlink.c
      595 provjeri.c
      807 Makefile
      484 readsymlink.c
     1302 fileinfo2.c
      942 mojls.c
   58904 README.md
      677 dotakni.c
     1300 fileinfo.c
      361 makelink.c
```

```
$ ./mojls /etc
     4096 cron.d
     4096 default
     4096 X11
     1111 passwd
      597 group
      609 shadow
      219 hosts
       11 hostname
    12813 services
      582 profile
      656 fstab
      122 resolv.conf
    ...
```

Direktorij /etc u praksi sadrži znatno više datoteka i poddirektorija nego što je prikazano (na tipičnom sustavu desetke i stotine zapisa); ostatak ispisa koji ovdje nije prikazan iz prostornih razloga označen je s ... na dnu.

Direktoriji (cron.d, default, X11) uobičajeno imaju veličinu 4096 bajtova, što je tipična veličina bloka datotečnog sustava na Linuxu — jedan blok dovoljan je za pohranjivanje popisa imena i pripadnih i-node brojeva uobičajenog direktorija. Bitno je razumjeti da ova vrijednost **ne predstavlja zbroj veličina datoteka unutar direktorija**, kao što čitatelj možda intuitivno očekuje, nego veličinu prostora koji direktorij sam zauzima na disku radi vlastite organizacije. Većina direktorija stoga u lstat() ispisu pokazuje istih 4096 B bez obzira što se u njima nalazi (vrijednost može biti i veća kod direktorija s vrlo mnogo unosa, kad jedan blok više nije dovoljan). Za regularne datoteke (passwd, services, ...) st_size pokazuje stvarnu veličinu sadržaja u bajtovima.

Vježba za čitatelja: ispisivanje pravog ls -l sadrži znatno više informacija — tip datoteke, prava pristupa, vlasnika i grupu, vrijeme zadnje izmjene. Sve te informacije su nam već dostupne u struct stat strukturi koju smo popunili pozivom lstat(). Pokušajte doraditi mojls tako da reproducira potpun ls -l ispis. Korisne funkcije iz standardne C biblioteke: getpwuid() (pretvara UID u ime korisnika, definirana u <pwd.h>), getgrgid() (pretvara GID u ime grupe, definirana u <grp.h>), te ctime() ili strftime() za pretvorbu time_t vrijednosti u tekstualni datum.

Što smo zapravo radili

Vrijedi se na kraju ovog poglavlja na trenutak osvrnuti na ono što smo zapravo radili kroz dane primjere. Mali UNIX programi poput `dotakni`, `mojls`, `prava`, `makelink`, `makesymlink`, `readsymlink` — kao i `f_cat`, `f_strip` ili `f_write` iz prethodnog poglavlja — nisu tek umjetni vježbeni primjeri. Svaki od njih implementira temeljnu funkcionalnost neke od standardnih UNIX naredbi: `touch`, `ls`, `chmod`, `ln`, `ln -s`, `readlink`, `cat`. Time ilustriramo i jednu od najvažnijih osobina UNIX-a: skupina alata koje koristimo svaki dan zapravo je **tanki sloj iznad sistemskih poziva**. Kad ih sami napišemo u nekoliko desetaka linija C koda, vidimo iz prve ruke da iza naredbi koje na prvi pogled izgledaju “magično” stoje sasvim razumljivi pozivi `stat`, `chmod`, `link`, `symlink`, `utime` — a temeljne UNIX rutine i sistemski pozivi koji ih implementiraju isti su već gotovo pola stoljeća, što pokazuje koliko je dobro osmišljen UNIX sustav.

Ovo nam ukazuje na još jednu osobinu UNIX-a: koncepti na kojima UNIX počiva su složeni, ali kad ih jednom usvojite, naizgled kompleksne stvari možete realizirati na relativno jednostavan način, sa svega nekoliko sistemskih poziva. Ova osobina utkana je u temeljnu filozofiju izrade UNIX programa: *radi jednu stvar, ali radi je dobro!* Takav pristup potiče pisanje malih, modularnih programa specijaliziranih za obavljanje jedne konkretne zadaće, uz mogućnost njihova povezivanja u veće i složenije cjeline koristeći osnovno načelo UNIX-a — *sve je datoteka*.

Prevođenje

Direktorij dolazi s priloženim Makefile-om koji prati iste konvencije kao i u ostalim poglavljima (varijable `CC`, `CFLAGS`, `LDFLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`).

```
make all           # gradi sve primjere
make fileinfo      # gradi pojedinačni primjer
make clean         # čisti generirane datoteke
```

Okruženje procesa

U ovom poglavlju dani su primjeri koji demonstriraju upravljanje procesima na UNIX-u: dohvaćanje argumenata naredbenog retka i okruženja UNIX procesa, stvaranje procesa, pokretanje programa i upravljanje limitima. Svaki od ovih mehanizama vezan je uz jedan ili više sistemskih poziva — `fork()`, `exec` obitelj, `wait()`, `dup2()`, `setrlimit()` — koji zajedno tvore jezgru UNIX-ove filozofije upravljanja procesima. Primjeri su poredani tako da se teme grade postupno — od najjednostavnijih (ispis primljenih argumenata) prema složenijima (kombinacija `fork`, `exec`, `dup2` i `wait` u jednom programu).

Argumenti naredbenog retka

Funkcija `main` prema ISO C standardu može imati dva osnovna oblika:

```
int main(void)
int main(int argc, char *argv[])
```

Prvi oblik koristimo kada program ne očekuje dodatne opcije i argumente koje korisnik može zadati prilikom pokretanja programa. Međutim, često je korisniku ostavljena mogućnost da putem dodatnih argumenata, koji se zadaju kao stringovi u naredbenom retku iza same naredbe kojom je program pozvan, usmjerava način izvršavanja našeg programa. U ovom slučaju koristimo drugi oblik funkcije `main`, kako bi mogli pristupiti svim argumentima koji su u naredbenom retku zadani.

Prilikom pozivanja bilo koje naredbe u UNIX ljusci, ljuska analizira niz znakova kojim korisnik želi pokrenuti naredbu i dijeli ga na podnizove odvojene razmacima. Ovaj postupak nazivamo **tokenizacija**, a rezultat je niz **tokena** — stringova koji redom sadrže pozvanu naredbu i sve argumente koje je korisnik naveo. Ukoliko ovim stringovima želimo pristupiti iz našeg programa, koristimo drugi oblik funkcije `main`, kod kojeg funkcija prima dva argumenta: cjelobrojni `argc` u kojem je pohranjen ukupan broj stringova navedenih u naredbenom retku (uključujući i naredbu kojom je program pozvan), i polje pokazivača na znakovni niz `argv`, u kojem su ovi stringovi redom pohranjeni.

Argumenti naredbenog retka nalaze se na samom vrhu adresnog prostora procesa, kako je prikazano na slici:

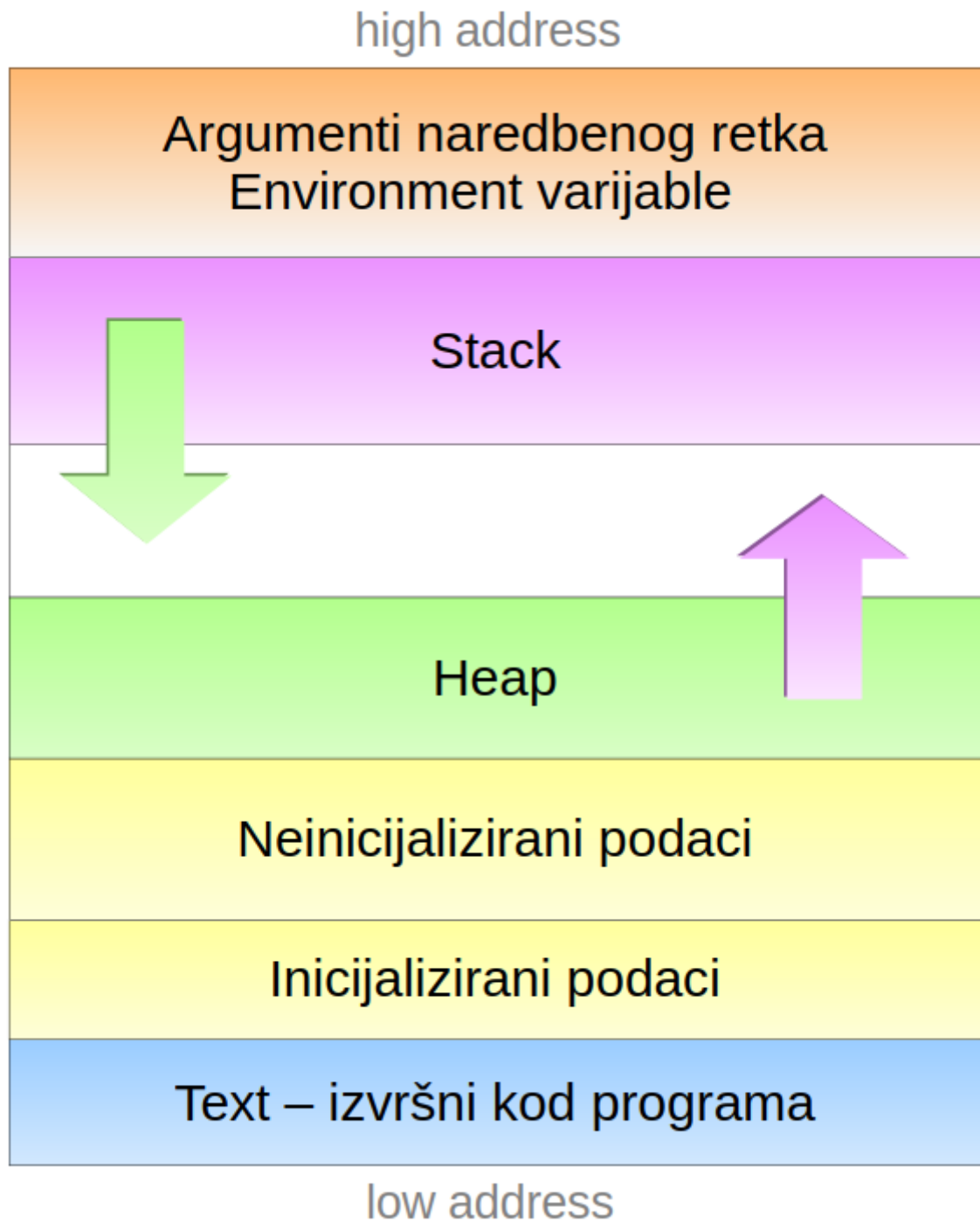
Najjednostavniji način pristupa je iteriranje kroz polje pokazivača `argv`, od prvog elementa (indeks 0) koji pokazuje na samu naredbu, do posljednjeg s indeksom `argc-1`.

Uzmimo za primjer hipotetski program `mojprogram` kod kojeg korisnik može prilikom pokretanja zadati argumente naredbenog retka. Neka je naš program pozvan s:

```
./mojprogram prvi argument      drugi_argument
```

Vrijednost argumenta `argc` bila bi 4 i odgovarala bi broju tokena koji čine naredbeni redak. Polje pokazivača `argv` redom bi pokazivalo na adrese pri vrhu adresnog prostora procesa, na kojima bi bile sljedeće vrijednosti:

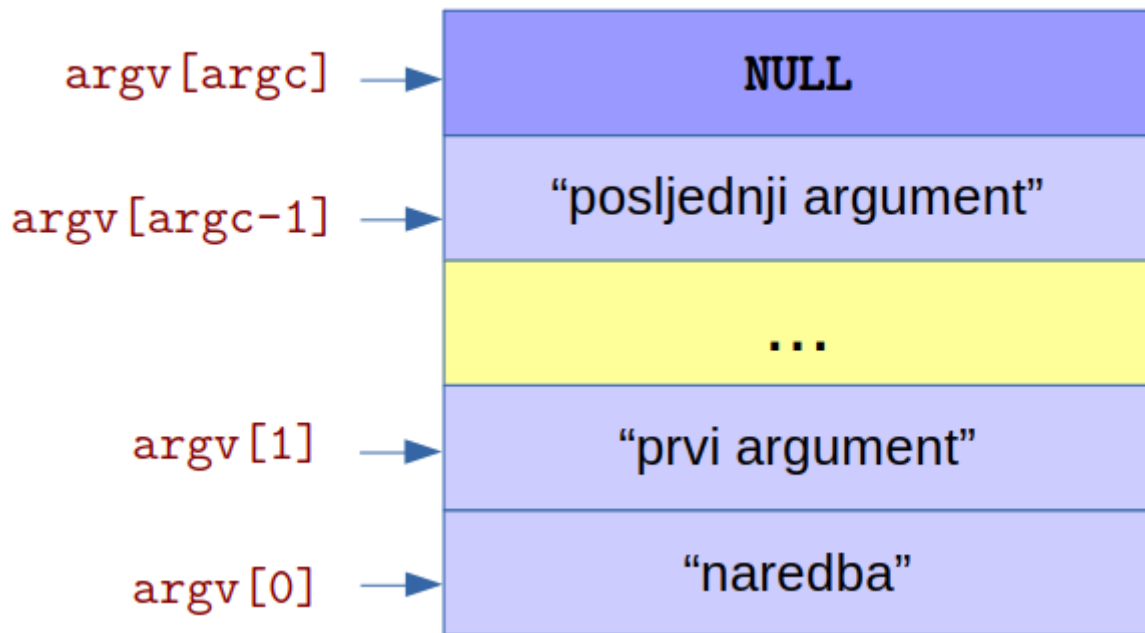
```
argv[0] = "./mojprogram"
argv[1] = "prvi"
argv[2] = "argument"
argv[3] = "drugi_argument"
```



Slika 5: Memorijska slika UNIX procesa

`argv[4] = NULL`

Uočimo dva detalja: ljska tokenizira naredbeni redak tako da tokene razdvaja temeljem razmaka (*space*) — praznog prostora između njih. Pri tom je svejedno koristimo li jedan ili više razmaka (više puta pritisnuta tipka *space* prilikom unosa). Dodatno, pored pokazivača `argv[0]` do `argv[argc-1]`, uvijek postoji i posljednji pokazivač u nizu s indeksom `argc`, koji pokazuje na vrijednost `NULL`, tj. `(void*)0`. Organizacija polja `argv` u memoriji shematski je prikazana na sljedećoj slici:



Slika 6: Organizacija argumenata naredbenog retka u memoriji procesa

- **argumenti.c** — najjednostavniji mogući primjer rada s argumentima naredbenog retka. Program u petlji prolazi kroz polje `argv[0]`, ..., `argv[argc-1]` i ispisuje indeks i vrijednost svakog argumenta. Koristi se za vizualnu provjeru kako ljska prenosi naredbeni redak programu — posebno korisno za razumijevanje razdvajanja riječi po razmacima, ponašanja navodnika, ili kako `argv[0]` uvijek nosi ime kojim je program pokrenut.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int k;

    for (k = 0; k < argc; k++)
        printf("%d:\t %s\n", k, argv[k]);

    return 0;
}
```

```
$ ./argumenti jedan dva tri
0:      ./argumenti
1:      jedan
2:      dva
3:      tri
```

- **zbroji.c** — program koji dva argumenta naredbenog retka tumači kao cijele brojeve i ispisuje njihov zbroj. Uvodi praksu **provjere broja argumenata** na početku programa (`argc < 3` → ispis upute za korištenje i izlaz) te funkciju `atoi()` iz standardne C

biblioteke za konverziju stringa u int. Uobičajeni UNIX obrazac: poruka o korištenju uvijek koristi argv[0] kako bi točno odražavala naziv pod kojim je program pozvan.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int a, b, zbroj;

    if (argc < 3) {
        printf("koristenje: %s <1.broj> <2.broj>\n", argv[0]);
        exit(0);
    }

    a = atoi(argv[1]);
    b = atoi(argv[2]);
    zbroj = a + b;

    printf("%d + %d = %d\n", a, b, zbroj);

    exit(0);
}
```

```
$ ./zbroji
koristenje: ./zbroji <1.broj> <2.broj>
$ ./zbroji 17 25
17 + 25 = 42
```

Varijable okruženja

Pored argumenata naredbenog retka, svaki UNIX proces u svom memorijskom prostoru sadrži i drugu vrstu konteksta — **varijable okruženja** (engl. *environment variables*). Ovaj skup vrijednosti proces nasljeđuje od svog roditelja — procesa koji je inicirao njegovo stvaranje korištenjem sistemskog poziva fork(), a može se mijenjati tijekom izvršavanja procesa.

Varijable okruženja nalaze se u memoriji procesa, pri samom vrhu adresnog prostora (odmah "ispod" argumenata naredbenog retka), a zadane su u formi niza parova oblika "IME=vrijednost". Ove vrijednosti procesu prenose informacije o sistemskoj konfiguraciji i korisničkim postavkama, bez potrebe da se eksplicitno zadaju kao argumenti naredbenog retka. Vrijednosti pojedinih varijabli postavljaju se u inicijalizacijskim skriptama ljuste, a sve naredbe pokrenute iz iste sesije ih dijele kao zajednički kontekst.

Najčešće varijable okruženja na svakom UNIX sustavu su:

Varijabla	Značenje
HOME	Apsolutna putanja korisničkog <i>home</i> direktorija.
PATH	Popis direktorija (odvojenih dvotočkom) u kojima ljusta traži izvršne datoteke.
USER	Korisničko ime trenutno prijavljenog korisnika.
SHELL	Apsolutna putanja korisnikove zadane ljuste.
LANG	Lokalizacijske postavke (jezik, kodna stranica).
PWD	Trenutno radni direktorij.
TERM	Tip terminala u kojem se sesija odvija.

Vrijednostima varijabli okruženja možemo pristupati i mijenjati ih direktno iz UNIX ljuške. Vrijednost varijable okruženja čitamo korištenjem prefiksa \$ ispred imena. Na primjer, ukoliko želimo saznati koji je naš korisnički direktorij, dovoljno je izvršiti:

```
$ echo $HOME
/home/dkrst
```

Za promjenu vrijednosti postojeće, ili dodavanje nove varijable okruženja koristimo naredbu export:

```
$ export MOJA_VAR="neka vrijednost"
$ echo $MOJA_VAR
neka vrijednost
```

Naredba env (bez argumenata) ispisuje sve varijable okruženja trenutne sesije.

Iz programa pisanih u C-u, varijablama okruženja pristupa se kroz funkcije iz standardne C biblioteke deklarirane u <stdlib.h>:

```
#include <stdlib.h>

char *getenv(const char *name);
int  setenv(const char *name, const char *value, int overwrite);
int  unsetenv(const char *name);
int  putenv(char *string);
```

Funkcija getenv()

Dohvaća vrijednost varijable okruženja prema njezinom imenu.

Povratna vrijednost: pokazivač na string s vrijednošću tražene varijable, ili NULL ako varijabla nije postavljena.

Argumenti:

- **name** — ime varijable okruženja čija se vrijednost dohvaća.

Vraćeni string ne smije se modificirati — pripada okruženju procesa kojim upravlja jezgra. Ako želimo zadržati vrijednost duže vremena, najsigurnije ju je odmah kopirati u vlastiti međuspremnik (npr. pozivom strdup() ili strncpy()).

Funkcija setenv()

Postavlja vrijednost varijable okruženja, eventualno prepisujući postojeću.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške (pri čemu se postavlja errno).

Argumenti:

- **name** — ime varijable koja se postavlja.
- **value** — nova vrijednost varijable.
- **overwrite** — ponašanje kad varijabla već postoji: ako je 0, postojeća vrijednost se zadržava; inače se prepisuje.

Funkcija unsetenv()

Briše varijablu iz okruženja.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **name** — ime varijable koju brišemo. Ako varijabla ne postoji, poziv se smatra uspješnim (vraća 0).

Funkcija putenv()

Postavlja varijablu zadanu u obliku "ime=vrijednost".

Povratna vrijednost: 0 u slučaju uspjeha, ne-nula vrijednost u slučaju greške.

Argumenti:

- **string** — pokazivač na niz oblika "ime=vrijednost". Bitno je naglasiti da putenv() **ne kopira** zadani string nego pohranjuje sam pokazivač u tablicu okruženja. Stoga string mora biti trajno dostupan — najčešće se koriste statički ili dinamički alocirani nizovi, a **nikad lokalne varijable** funkcije koja poziva putenv(). Zbog ovog ponašanja, setenv() je u modernom kodu preferiran jer kopira vrijednost u svoje vlasništvo.

Funkcije setenv, unsetenv i putenv koriste se za izmjenu okruženja samog procesa — promjene se **ne** propagiraju natrag u roditeljski proces (ljusku), nego ostaju lokalne tekućem procesu i njegovim potomcima.

- **readenv.c** — čita vrijednost jedne varijable okruženja čije se ime zadaje kao argument naredbenog retka, pozivom getenv() iz standardne C biblioteke. getenv() vraća pokazivač na string s vrijednošću varijable, ili NULL ako varijabla nije postavljena. Program razlikuje ta dva slučaja i prikazuje odgovarajuću poruku. Ilustrira temeljni način na koji program pristupa okolini koju je naslijedio od ljuske — varijable poput HOME, PATH, USER ili vlastite varijable postavljene naredbom export.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    char *vrijednost;

    if (argc < 2) {
        printf("koristenje: %s <ime varijable>\n", argv[0]);
        return 0;
    }

    vrijednost = getenv(argv[1]);
    if (!vrijednost)
        printf("%s: environment varijabla ne postoji\n", argv[1]);
    else
        printf("%s = %s\n", argv[1], vrijednost);

    return 0;
}
```

```
$ ./readenv HOME
HOME = /home/dkrst
$ ./readenv NEPOSTOJECA
NEPOSTOJECA: environment varijabla ne postoji
```

Treći argument funkcije main — envp

Pored dva standardna oblika funkcije main opisana ranije, na UNIX sustavima česta je i sljedeća, proširena varijanta:

```
int main(int argc, char *argv[], char *envp[]) {
    // envp je ovdje lokalni parametar funkcije
}
```

Treći argument envp je polje pokazivača na stringove oblika "IME=vrijednost", završeno NULL pokazivačem — (void *)0 — i sadrži kompletno okruženje koje je proces naslijedio pri pokretanju.

Treba imati na umu da **envp nije dio ISO C standarda** — ISO/IEC 9899:2018 u Annexu J.5.1 (*Common extensions*) spominje ovaj treći argument samo kao uobičajenu implementacijsku ekstenziju, što znači da je implementacije smiju ali nisu dužne podržavati. Ni POSIX.1-2017 ne propisuje envp kao standardni oblik — naprotiv, izričito preporučuje korištenje varijable environ (opisane u nastavku). U praksi je ipak treći argument main-a podržan na praktički svim modernim UNIX i Linux distribucijama, kao i u Microsoft C kompajleru.

Vanjska varijabla environ

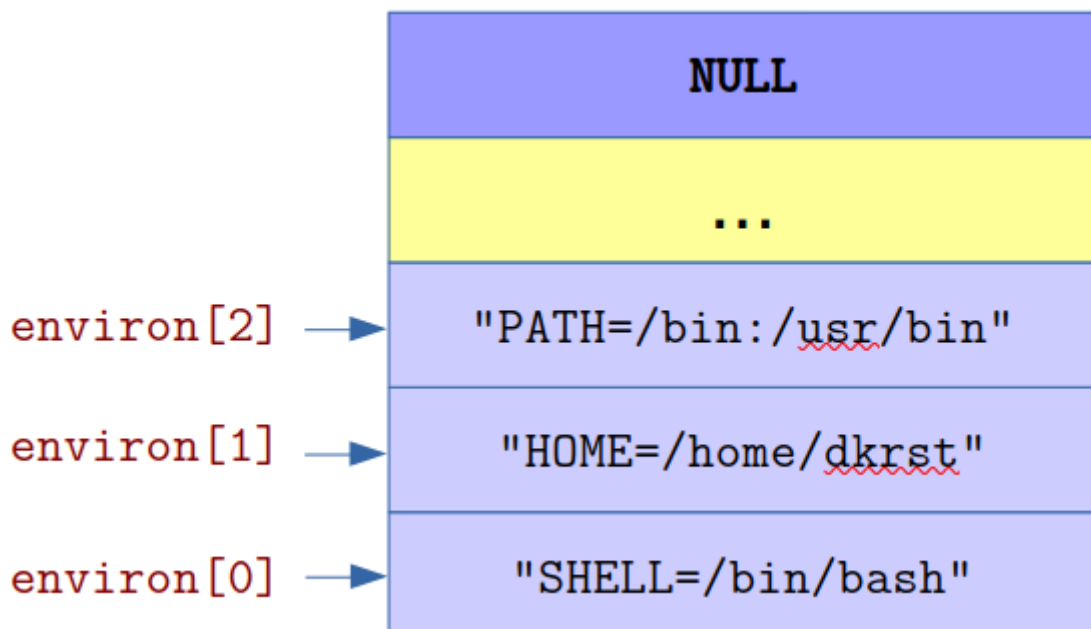
Drugi način pristupa okolini iz programa pisanog u jeziku C jest preko vanjske globalne varijable environ. Za razliku od envp, koji je lokalni parametar funkcije main, environ je dostupna iz bilo koje funkcije programa nakon što se na nju prethodno deklarira:

```
#include <unistd.h>
extern char **environ;    // deklaracija vanjske varijable

int main(int argc, char *argv[]) {
    // environ je dostupan ovdje iako nije u zagradama main-a
}
```

Pokazivač environ pokazuje na isti niz pokazivača kao i envp u trenutku pokretanja procesa — drugim riječima, oba mehanizma daju početno isti pogled na okruženje. Razlika postaje vidljiva tek nakon poziva funkcija setenv(), putenv() ili unsetenv(): te funkcije ažuriraju varijablu environ (eventualno realocirajući memoriju), dok envp ostaje pokazivati na izvornu, sada zastarjelu kopiju. Drugim riječima, envp je “snapshot” okoline u trenutku ulaska u main, a environ je “živa” referenca.

Organizacija varijabli okruženja u memoriji procesa shematski je prikazana na sljedećoj slici. Bez obzira pristupa li se okruženju kroz environ ili kroz envp, sama struktura u memoriji uvijek je ista — niz pokazivača na stringove oblika “IME=vrijednost” završen NULL-om. Razlikuje se samo ime varijable kroz koju mu pristupamo.



Slika 7: Organizacija varijabli okruženja u memoriji procesa

Za razliku od envp, varijabla environ **jest** standardizirana — propisuje je POSIX.1-2017. Zanimljivo, to je jedini objekt u POSIX-u kojeg ne deklarira nijedna sistemska zaglavna datoteka, pa korisnik mora sam navesti deklaraciju `extern char **environ;` u svom

programu (kao u primjeru iznad). ISO C, ponovo, ovu varijablu uopće ne spominje.

Koji oblik koristiti?

Preferirani oblik je `environ`, iz nekoliko razloga:

- **Standardiziran je u POSIX-u**, dok je `envp` samo implementacijska ekstenzija u Annexu ISO C standarda.
- **Uvijek odražava trenutno stanje okoline** — promjene napravljene pozivima `setenv()/putenv()/unsetenv()` reflektiraju se u `environ`-u, ali ne i u `envp`-u, pa korištenje `envp`-a nakon takvih izmjena vodi u nedefinirano ponašanje.
- **Dostupna je iz bilo koje funkcije programa**, ne samo iz `main`. Predaja `envp`-a niz funkcijske pozive zahtijeva da ga svaka funkcija koja ga treba primi kao argument.

U sljedećem ćemo paru primjera (`listenv1.c` i `listenv2.c`) prikazati oba mehanizma — funkcionalno daju isti rezultat, ali ilustriraju razliku u načinu pristupa okruženju iz koda.

- **`listenv1.c`** — ispisuje sve varijable okruženja procesa korištenjem trećeg argumenta funkcije `main` (`envp`):

```
int main(int argc, char *argv[], char *envp[])
```

Petlja iterira kroz polje `envp` sve dok ne naiđe na završni `NULL`. Funkcionalno je ekvivalentan UNIX naredbi `env`:

```
$ ./listenv1
SHELL=/bin/bash
HOME=/home/dkrst
PATH=/usr/local/bin:/usr/bin:/bin
LANG=hr_HR.UTF-8
...
```

- **`listenv2.c`** — ista funkcionalnost kao `listenv1.c`, ali umjesto trećeg argumenta `main`-a koristi vanjsku varijablu `environ`:

```
extern char **environ;
```

```
int main(int argc, char *argv[])
```

Funkcija `main` koristi standardni oblik s dva argumenta, a okruženju se pristupa preko globalne varijable. Ispis je identičan onom iz `listenv1`. Ovaj oblik je preporučen za stvarne programe iz razloga navedenih u prethodnom odjeljku.

U `listenv2.c` smo se uz to malo poigrali s pokazivačkom aritmetikom — umjesto da koristimo brojač i indeksiramo polje kao u `listenv1.c` (`envp[k]`), ovdje izravno pomičemo sam pokazivač `environ` izrazom `*environ++` u svakoj iteraciji petlje. Funkcionalno je rezultat potpuno isti — petlja redom prolazi kroz sve elemente niza dok ne naiđe na završni `NULL` — samo što sada uloga “indeksa” više nije zasebna varijabla `k`, nego sam pokazivač koji se pomiče po nizu.

Životni ciklus procesa

Životni ciklus UNIX procesa započinje sistemskim pozivom `fork`, što je jedini način za stvaranje novog procesa na UNIX-u. U ovom trenutku proces dobija **PID**, koji predstavlja jedinstveni identitet procesa i nepromjenjiv je za cijelo vrijeme njegovog postojanja, te memorijski prostor u kojem se proces izvršava — nakon čega je spreman za izvršavanje.

```
#include <unistd.h>
```

```
pid_t fork(void);
```

Povratna vrijednost: u roditeljskom procesu `fork()` vraća PID novostvorenog djeteta (pozitivna cjelobrojna vrijednost); u djetetu vraća `0`; u slučaju greške (npr. kada je

Sistemski poziv `fork()` nema argumenata i jednostavno kopira memorijsku sliku postojećeg procesa — procesa roditelja koji je pozvao `fork()`. Ova funkcija **poziva se jednom, a vraća dva puta**: u postojećem procesu (proces roditelj, *parent process*) i u novonastalom procesu (proces dijete, *child process*), koji je identična kopija roditeljskog procesa — uključujući stanje stoga, hrpe i svih varijabli, ali i **brojača instrukcija** (engl. *program counter*, PC) — posebnog registra u UNIX procesu koji pokazuje na sljedeću instrukciju u strojnom kodu koja se treba izvršiti. Ovo efektivno znači da proces dijete nastavlja točno tamo gdje se proces roditelj nalazio u trenutku poziva funkcije `fork()` — kao da su se do tog trenutka oba procesa izvršavala na potpuno jednak način i nalaze se u identičnom stanju, iako je zapravo postojao samo jedan proces. Od tog trenutka dva procesa počinju živjeti odvojene živote — promjena bilo koje varijable u jednom od njih ne utječe na vrijednost varijabli u drugom procesu, jer se svaki proces izvršava u svom memorijskom prostoru.

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

- **nproc.c** — ilustrira upravo opisano svojstvo `fork()`-a: novi proces dobiva vlastitu kopiju memorijskog prostora roditelja, uključujući sve varijable na stogu i hrpi, te nastavlja izvršavanje točno tamo gdje se roditelj nalazio u trenutku poziva. Program u petlji poziva `fork()` tri puta. Nakon završetka petlje program ispisuje svoj jedinstveni process ID (PID) i proces ID procesa roditelja koristeći funkcije `getpid()` i `getppid()`. Ova informacija ispisuje se **8 puta** — što znači da izvršavanje petlje rezultira s ukupno 8 procesa, iako bi na prvu mogli pomisliti da je logičan broj procesa nakon izvršavanja petlje 4 (jedan izvorni i tri kopije, stvorene u tri iteracije petlje).

Ovo je posljedica upravo gore opisanog načina funkcioniranja sistemskog poziva `fork()` — kopiranjem memorijske slike procesa kopira se i varijabla `k`, ali i brojač instrukcija (PC) koji pokazuje na sljedeću instrukciju koja se treba izvršiti. U svakoj iteraciji petlje iz jednog procesa nastaju dva potpuno identična procesa, koja se od te točke izvršavaju nezavisno — oba procesa nalaze se usred petlje i nastavljaju dalje kroz istu petlju. Razvoj kroz iteracije, uz broj procesa i vrijednost varijable `k` koja nakon `fork`-a postoji nezavisno u svakome:

```
iteracija k=0:
    fork() klonira postojeći proces → 2 procesa, k=0 u svakom
    k++ → 2 procesa, k=1 u svakom
```

iteracija k=1:
fork() klonira sva 2 procesa → 4 procesa, k=1 u svakom
k++ → 4 procesa, k=2 u svakom

iteracija k=2:
fork() klonira sva 4 procesa → 8 procesa, k=2 u svakom
k++ → 8 procesa, k=3 u svakom

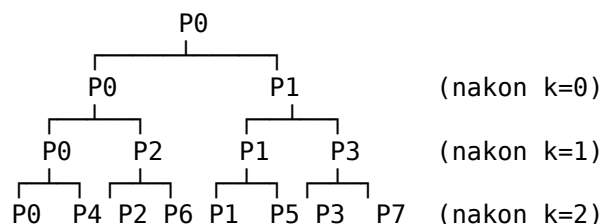
uvjet k<3 neistinit u svima → izlazak iz petlje
printf() izvršava se u svih 8

Stablo procesa nakon tri iteracije, s oznakama P0 (izvorni proces) do P7 (posljednje stvoreno dijete). Svaki fork() poziv udvostručuje skup aktivnih procesa — u svakoj iteraciji svi postojeći procesi paralelno kloniraju sebe, tako da nakon tri iteracije imamo 8 procesa:

prije petlje: P0

nakon k=0: P0 P1 (svaki se u k=1 forka)
↓ ↓
nakon k=1: P0 P1 P2 P3 (svaki se u k=2 forka)
↓ ↓ ↓ ↓
nakon k=2: P0 P1 P2 P3 P4 P5 P6 P7

Odnos roditelj–dijete u stablu:



Formula za broj procesa nakon n uzastopnih fork() poziva je 2^n — svaki fork() udvostručuje broj aktivnih procesa. U ovom primjeru u petlju ulazi 1 proces, a iz nje izlazi 8. Iz ispisanih PPID-ova moguće je rekonstruirati cijelo stablo — svaki proces zna tko mu je neposredni roditelj.

- **novi.c** — korištenje povratne vrijednosti fork()-a za razlikovanje uloge roditelja i djeteta. Iako su oba procesa identične kopije jedan drugog, njih dva trebaju se u nastavku ponašati različito: roditelj obično nastavlja svoj prethodni posao, a dijete izvršava neki novi zadatak. Jedini mehanizam po kojem dva procesa mogu razaznati tko je tko jest upravo povratna vrijednost fork()-a — roditelj dobiva PID djeteta, a dijete dobiva 0. Program to iskorištava za granjanje logike — istim printf-om ispisuje različit tekst ovisno o tome tko ga izvršava:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int pid;

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        printf("CHILD (%d)\t PID: %d\t PPID: %d\n",
            pid, getpid(), getppid());
    } else {
        printf("PARENT (%d)\t PID: %d\t PPID: %d\n",
            pid, getpid(), getppid());
    }
}

```

```

        pid, getpid(), getppid());
    }

    return 0;
}

```

```

$ ./novi
PARENT (25017)      PID: 25016  PPID: 14567
CHILD  (0)         PID: 25017  PPID: 25016

```

PID djeteta (25017) koji je roditelj dobio kao povratnu vrijednost `fork()`-a točno odgovara PID-u koji dijete prijavljuje pozivom `getpid()`; istovremeno dijete vidi PPID jednak PID-u roditelja, što potvrđuje odnos roditelj-dijete u procesnom stablu. Redoslijed ispisa između roditelja i djeteta nije određen — ovisi o raspoređivaču koji odlučuje koji će od dva spremna procesa dobiti procesor prvi.

Izlazni status procesa — sistemski poziv `wait`

Nakon stvaranja novog procesa sistemskim pozivom `fork`, jezgra zapisuje informacije o novonastalom procesu u **tablicu procesa** — istu onu u kojoj se čuva i informacija o svim otvorenim deskriptorima datoteka. Ovaj zapis u tablici procesa čuva se sve dok ima potrebe za tim — sve dok proces postoji, ali ponekad i duže. Sjetimo se da funkcija `main` ima povratnu vrijednost `int`. Vjerojatno ste se već zapitali čemu ova vrijednost služi i kome se uopće vraća, s obzirom da je `main` prva funkcija koja se poziva u novopokrenutom programu i ne postoji funkcija koja bi ju pozivala.

Upravo ova vrijednost osigurava mehanizam putem kojeg proces roditelj može saznati što se dogodilo s njegovim procesom djetetom: je li izvršio svoju zadaću ili je došlo do greške, i što je tu grešku uzrokovalo? Naime, poziv `return` u funkciji `main`, isto kao i poziv sistemskog poziva `exit` u bilo kojoj drugoj funkciji, prekidaju izvršavanje procesa i predaju jezgri **izlazni status procesa** — cjelobrojnu vrijednost od 0 do 255. Jezgra ovu vrijednost pohranjuje u slog u tablici procesa koji odgovara procesu koji je upravo završio. Zadatak procesa roditelja je da ovu vrijednost pročita i na temelju nje sazna sudbinu svog djeteta, a to može napraviti sistemskim pozivom `wait`:

```

#include <sys/wait.h>

pid_t wait(int *wstatus);

```

Povratna vrijednost: PID djeteta koje je završilo, ili -1 u slučaju greške (npr. proces nema djece koja bi mogla završiti). Izlazni status `child` procesa upisuje se u cjelobrojnu varijablu na koju pokazuje argument `wstatus`.

`wait()` blokira pozivajući proces sve dok jedno od njegove djece ne završi izvršavanje. Kad se to dogodi, jezgra puni varijablu na koju pokazuje `wstatus` informacijama o tome kako je dijete završilo: je li završilo normalno (npr. pozivom `return` ili `exit()`), je li prekinuto signalom, te koja je njegova povratna vrijednost odnosno koji je signal uzrokovao prekid. Ako pozivatelja zanima samo to da pričekava dijete, ali ne i sam status, kao argument se može proslijediti `NULL`.

Ovaj statusni broj **nije izravno** povratna vrijednost djeteta — riječ je o pakiranom cjelobrojnem polju u kojem su različite informacije o završetku procesa smještene u različite skupove bitova. Konkretno, izlazni status (vrijednost koju je dijete vratilo iz `main`-a ili predalo `exit()`-u, u rasponu **0 do 255**) zapisuje se u bitove 8-15 statusne riječi. Da bi se ta vrijednost izvukla, koristi se makro `WEXITSTATUS()` iz iste zaglavne datoteke `<sys/wait.h>`.

Posljedica je da bezuvjetan ispis statusnog broja kao cijele vrijednosti ne daje povratnu vrijednost djeteta, nego njezino “pomaknuto” prikazivanje: ako dijete vrati 5, `raw status bit` će `5 << 8 = 1280`; ako vrati 255, `raw status bit` će `65280`.

- **ret_stat.c** — minimalan primjer koji ilustrira upravo opisani odnos između raw statusnog broja i WEXITSTATUS-a. Roditelj pozivom `fork()` stvara dijete koje jednostavno vrati cjelobrojnu vrijednost zadanu kao argument naredbenog retka:

```
} else if (pid == 0) {
    return atoi(argv[1]); // child: vrati zadanu vrijednost
} else {
    wait(&rv);
    printf("CHILD EXIT STATUS: %d\n", rv);
    // printf("CHILD EXIT STATUS: %d\n", WEXITSTATUS(rv));
}
```

Pokrenimo program s nekoliko različitih vrijednosti u rasponu 0–255 (sve dopuštene povratne vrijednosti procesa):

```
$ ./ret_stat 0
CHILD EXIT STATUS: 0
$ ./ret_stat 1
CHILD EXIT STATUS: 256
$ ./ret_stat 5
CHILD EXIT STATUS: 1280
$ ./ret_stat 255
CHILD EXIT STATUS: 65280
```

Samo prva vrijednost (0) odgovara onome što je dijete uistinu vratilo. Ostali brojevi su pomnoženi sa 256 — odnosno, gledano kao niz bitova, izvorna vrijednost je pomaknuta osam mjesta ulijevo. To su upravo bitovi 8–15 statusne riječi, gdje jezgra po POSIX konvenciji pohranjuje izlazni status. Ostali bitovi statusnog broja rezervirani su za druge informacije (signal koji je prekinuo proces, oznaka core-dumpa itd.) — te ćemo informacije iskoristiti u sekciji **Pokretanje novog programa**.

Da bismo dobili izvornu povratnu vrijednost, koristi se makro `WEXITSTATUS()`. U primjeru `ret_stat.c` zakomentirajte prvi `printf` i otkomentirajte drugi:

```
// printf("CHILD EXIT STATUS: %d\n", rv);
printf("CHILD EXIT STATUS: %d\n", WEXITSTATUS(rv));
```

Nakon ponovnog prevođenja, ispis je upravo onakav kakav očekujemo:

```
$ ./ret_stat 5
CHILD EXIT STATUS: 5
$ ./ret_stat 255
CHILD EXIT STATUS: 255
```

Makro `WEXITSTATUS(s)` ne radi ništa “magično” — interno samo izvuče bitove 8–15 iz statusnog broja, što je ekvivalentno izrazu `(s >> 8) & 0xFF`. Korištenje makroa, međutim, čini kod jasnijim, neovisnim o konkretnoj implementaciji statusne riječi i prenosivim na različite UNIX sustave.

Zombi procesi

Slijed događaja koji smo upravo opisali — proces završi pozivom `exit()` ili `return` iz funkcije `main`, a njegov proces roditelj njegov izlazni status preuzme pozivom `wait()` — otvara jedno suptilno pitanje: **kada točno jezgra može osloboditi zapis o procesu u tablici procesa?** Resursi koje je proces koristio (memorija, otvoreni file deskriptori, itd.) oslobađaju se odmah po prekidu izvršavanja, ali to se ne odnosi na sam zapis u tablici procesa: u njemu je pohranjen izlazni status koji jezgra još mora moći predati procesu roditelju kad on (u nekom budućem trenutku) pozove `wait()`. Ako bi taj zapis bio izbrisan u trenutku prekida izvršavanja procesa djeteta, poziv `wait()` ne bi imao odakle pročitati izlazni status.

Rješenje koje UNIX koristi je sljedeće: kad dijete završi, jezgra **zadržava** njegov zapis u tablici procesa, ali u posebnom stanju u kojem proces više ne troši resurse — postoji samo

zapis o tome koji PID je imao i koji je njegov izlazni status. U uredno napisanim programima, ovaj period je vrlo kratak. Problem nastaje kad roditelj iz nekog razloga ne pozove `wait()` za svoju djecu. U ovom slučaju, imamo proces koji više nije živ, ali je još uvijek tu negdje i jezgra mora čuvati odgovarajući slog u tablici procesa. Štoviše, procesa u ovom stanju ne možemo se ni prisilno riješiti, npr. slanjem signala KILL koji bezuvjetno ubija proces kojem je poslan — jer ionako više nije živ. U UNIX terminologiji ovakvi procesi nazivaju se (prigodno) **zombi procesima** (engl. *zombie process*).

Zombi proces moguće je vidjeti naredbom `ps`: u stupcu STAT (status) prikazana je oznaka **Z**, a uz ime procesa stoji oznaka `<defunct>`. Problem nastaje ako se ovakvi procesi počnu gomilati: svaki od njih troši jedan slog u tablici procesa i koristi jedan PID — iako više nije aktivan. Kako tablica procesa ima ograničenu veličinu, kontinuirano stvaranje zombi procesa može dovesti do iscrpljenja sistemskih resursa — sustav neće moći pokrenuti nove procese sve dok netko ne “pokupi” zombije.

Stoga je dobra programerska praksa voditi računa o procesima koje ste stvorili: obavezno se pobrinite da po završetku izvršavanja vašeg child procesa pozovete `wait`, makar s argumentom `NULL` ukoliko vas njegov izlazni status ne zanima — ovo je jezgri potrebno da bi znala da ne mora više čuvati izlazni status procesa koji je završio s izvršavanjem.

Što se događa ako roditelj završi s izvršavanjem prije nego pozove `wait()`, ili jednostavno prekine izvršavanje dok je child još uvijek aktivan? Tu na scenu stupa mehanizam koji ćemo upoznati u sljedećem poglavlju, kod **osirotjelih procesa** (engl. *orphan processes*). Jezgra preuzima ulogu roditelja svim procesima čiji proces roditelj je završio izvršavanje — uključujući i zombije — i prebacuje ih procesu `init` (PID 1, ili u modernim Linux distribucijama `systemd`). `init` putem signala dobiva obavijest kada bilo koji od njegovih child procesa završi s izvršavanjem, uključujući i “usvojene” procese koji su ostali bez svojih roditelja. Svaki put kada primi takav signal, `init` poziva `wait()` te “čisti” iz tablice procesa sve zombije i pušta jezgru da oslobodi njihove zapise.

Pokretanje novog programa

Proces i program su dva povezana ali različita koncepta. **Proces** je aktivna instanca izvršavanja — entitet koji ima svoj PID, memorijski prostor i resurse. **Program** je statičan zapis: strojni kod pohranjen u izvršnoj datoteci, niz instrukcija koje proces izvršava.

Odnos između programa i procesa možemo usporediti s receptima i kuhanjem ručka. Svaki recept u osnovi predstavlja “program” za kuhanje: niz precizno definiranih koraka i postupaka s naredbama kontrole toka (`if (nedovoljno_slano): dodaj_soli`), petljama (`while(meso_tvrdo): nastavi_kuhanje`) i svim ostalim instrukcijama potrebnim da dobijemo očekivani rezultat u vidu ukusnog ručka. Sam postupak kuhanja u osnovi je pokretanje procesa koji izvršava program, tj. prati zadani recept korak po korak. Pri tom ne postoji nikakva prepreka da više kuhara istovremeno ne pokrene proces kuhanja po istom receptu. Tada svaki od kuhara zapravo nezavisno izvršava svoj “proces”, ali svi kuhaju po istom receptu, tj. u svim procesima izvršava se isti “program”.

Na UNIX-u proces “kuhanja” započinje s pripremom: stvaranjem novog procesa i pripadajućeg memorijskog prostora u kojem će se proces izvršavati — pozivom funkcije `fork()`, kao što smo vidjeli u prethodnoj sekciji. Nakon toga krećemo s kuhanjem po odgovarajućem receptu — programu zapisanom u izvršnoj datoteci — korištenjem sistemskog poziva `exec`, koji kao argument uzima izvršnu datoteku koju želimo pokrenuti. Valja napomenuti da je `exec` zapravo **obitelj funkcija**, koje imaju istu funkcionalnost (učitavanje programa-recepta), a razlikuju se u načinu kako se zadaju argumenti naredbenog retka i put do izvršne datoteke:

```
#include <unistd.h>

int execl (const char *path, const char *arg0, ... /*, NULL */);
int execlp(const char *file, const char *arg0, ... /*, NULL */);
int execlx(const char *path, const char *arg0, ... /*, NULL, char *const envp[] */);
int execv (const char *path, char *const argv[]);
```



```
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
```

Razlike među varijantama kodirane su u sufiksima iza exec:

- **l** (*list*) — argumenti nove naredbe prenose se kao redom navedena **lista** pojedinačnih stringova, završena NULL-om (npr. `execlp("ls", "ls", "-al", NULL)`). Sjetimo se da polje pokazivača `argv` u funkciji `main` uvijek završava NULL pokazivačem — upravo zato i u pozivu `execl` NULL moramo navesti kao posljednji argument.
- **v** (*vector*) — argumenti se prenose kao **polje pokazivača** na stringove, završeno NULL-om (npr. `execvp("ls", argv)`). Prikladno kad broj argumenata nije poznat u trenutku pisanja koda, odnosno kada se argumenti dobivaju u istom obliku (kao u `argv`) od samog pozivatelja.
- **p** (*path*) — izvršna datoteka pronalazi se pretragom direktorija iz varijable okoline `PATH`, pa se umjesto pune putanje može navesti samo ime (npr. "ls" umjesto `"/bin/ls"`). Varijante bez `p` očekuju punu putanju.
- **e** (*environment*) — poziv prima dodatni posljednji argument `envp`, niz pokazivača na stringove "IME=vrijednost" završen NULL-om, čime se novoj naredbi može eksplicitno zadati drugačije okruženje od onog koje ima trenutni proces. Varijante bez `e` ne mijenjaju trenutno okruženje procesa.

Svih šest funkcija u pozadini završava u sistemskom pozivu `execve()` — ostalih pet je omotač koji pogodnije pakira argumente. Zajedničko svim varijantama je da **ne stvaraju novi proces**: PID ostaje isti, mijenja se samo kod koji se izvršava. Ako `exec` uspije, poziv se nikad ne vraća — izvorni kod programa zamijenjen je novim, učitanim iz izvršne datoteke, varijable na stogu i hrpi se brišu, a brojač instrukcija (PC) postavlja se na prvu instrukciju u novom programu. Vraćanje iz `exec`-a znači grešku (npr. izvršna datoteka nije pronađena ili nemamo pravo izvršavanja).

- **myls.c** — prvi primjer funkcija iz `exec` obitelji: poziv `execlp()` zamjenjuje trenutni programski kod procesa kodom nekog drugog programa — u ovom slučaju UNIX naredbe `ls`. Za razliku od `fork()`, koji stvara novi proces, `exec()` **ne stvara novi proces** — PID ostaje isti, promijeni se samo programski kod koji se izvršava.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2)
        execlp("ls", "ls", "-al", NULL);
    else
        execlp("ls", "ls", "-al", argv[1], NULL);

    perror("execlp");
    return 1;
}
```

Bez argumenata, program pokreće `ls -al` nad trenutnim direktorijem; s jednim argumentom, pokreće `ls -al <argument>` kako bi nam ispisao sadržaj zadanog direktorija. Primijetite da su **prva dva argumenta poziva `execlp` identična**: prvi ("ls") je ime izvršne datoteke koju treba naći i pokrenuti, a drugi ("ls") je ono što će nova naredba dobiti kao svoj `argv[0]`. Podsjetimo se da po konvenciji svaka naredba u svom `argv[0]` očekuje vlastito ime (upravo iz tog razloga poruke o korištenju koriste `argv[0]`). Teoretski je moguće proslijediti i različito ime — tako bi program vidio da je pokrenut pod drugim imenom — ali u normalnoj uporabi oba se argumenta podudaraju.

Ako `execlp()` uspije, poziv se nikad ne vraća — zato će se `perror("execlp")` izvršiti samo u slučaju greške (npr. naredba `ls` nije pronađena u `PATH`-u). Stoga nije potrebno provjeravati povratnu vrijednost funkcije `execlp`. Za razliku od poziva `fork()`, koji se poziva jednom a vraća dva puta, `execlp` (kao i ostale funkcije iz obitelji `exec`) ne vraća

se nikad — osim u slučaju greške.

```
$ ./mysls
ukupno 24
drwxr-xr-x 2 dkrst users 4096 Apr 23 15:10 .
drwxr-xr-x 8 dkrst users 4096 Apr 23 15:09 ..
-rwxr-xr-x 1 dkrst users 8760 Apr 23 15:10 myls
-rw-r--r-- 1 dkrst users 256 Apr 23 15:09 myls.c
...
```

- **pokreni.c** — uopćena verzija prethodnog primjera: program prima proizvoljnu naredbu s argumentima kao svoje argumente naredbenog retka i pokreće je pozivom `execvp()`.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ...\n", argv[0]);
        return 0;
    }

    execvp(argv[1], &argv[1]);
    perror("execvp");

    return 1;
}
```

Kako funkcija `main` već dobiva argumente u obliku polja pokazivača (`argv`), upravo takav oblik očekuje i `execvp` — dovoljno je proslijediti pokazivač na `argv[1]`. Izraz `&argv[1]` je pokazivačka aritmetika — `argv` je polje pokazivača na stringove, a `&argv[1]` daje adresu **drugog elementa** tog polja, tj. pokazivač na pod-polje koje počinje od `argv[1]` i ide dalje. Funkcija `execvp` će to pod-polje tretirati kao svoje vlastito `argv`: `argv[1]` iz perspektive `pokreni`-ja postaje `argv[0]` nove naredbe (njezino ime), `argv[2]` postaje `argv[1]`, i tako redom. Ovako jednostavnim obrascem dobivamo zametak vlastite ljuske — program koji može pokrenuti bilo koju drugu UNIX naredbu.

Preporuka je pokušati razne kombinacije i promotriti ponašanje:

```
$ ./pokreni ls
$ ./pokreni ls -al
$ ./pokreni ls -al /root
$ ./pokreni asdhasd
```

Prva dva poziva rade očekivano. Treći poziv (`ls -al /root`) ovisi o tome je li korisnik `root` — ako nije, `ls` će ispisati poruku o greški jer nema pravo pristupa direktoriju `/root`. Zanimljivo je odgovoriti na pitanje: **tko ispisuje tu poruku?** U ovom slučaju to nije `pokreni` — `pokreni` je svoj posao obavio kad je pozvao `execvp`, koji je uspješno zamijenio njegov kod kodom naredbe `ls`. `pokreni` u doslovnom smislu više ne postoji kao program u tom procesu; poruku o grešci ispisuje sama naredba `ls`, nad svojim otvorenim standardnim izlazom za greške. U četvrtom pozivu (`./pokreni asdhasd`), poruku o grešci ispisuje `pokreni` sam — `execvp` je neuspješno pokušao pronaći izvršnu datoteku `asdhasd`, vratio se u `pokreni`, i idući redak koda je upravo `perror("execvp")`:

```
$ ./pokreni asdhasd
execvp: No such file or directory
```

Razlika je suptilna ali ključna: kad `exec` uspije, program koji je pozvao `exec` prestaje postojati; kad `exec` ne uspije, izvršavanje se nastavlja s naredbom iza njega.

Zabavan primjer rekurzivne zamjene:


```
$ ./pokreni ./pokreni ./pokreni ls -al
```

Jedan isti proces redom mijenja svoj sadržaj tri puta: prvi pokreni kod je zamijenjen drugim pozivom pokreni, taj pozivom trećeg pokreni-a, i on na kraju pozivom ls -al. PID ostaje isti kroz sve četiri metamorfoze, a ono što korisnik vidi kao rezultat — ispis sadržaja direktorija — potpuno je isto kao da smo ls -al pozvali direktno. Ova rekurzija ilustrira što exec zapravo jest: ne pokretanje novog procesa, nego **zamjena tijela postojećeg procesa drugim kodom**.

- **pokreni2.c** — pokretanje programa u novom procesu: **kombinacija fork() i exec()**.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char **argv) {
    int pid, s;

    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ...\n", argv[0]);
        return 0;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) { /* dijete */
        execvp(argv[1], &argv[1]);
        perror("execvp");
        return 127;
    } else { /* roditelj */
        wait(&s);
        if (WIFEXITED(s))
            printf("Normalan izlaz, izlazni status: %d\n",
                  WEXITSTATUS(s));
        else if (WIFSIGNALED(s))
            printf("Prekid signalom, signal: %d\n", WTERMSIG(s));
    }

    return 0;
}
```

U prethodnom primjeru pokreni je nakon execvp() prestao postojati kao pokreni — njegov je kod zamijenjen kodom pokrenute naredbe, pa nakon završetka naredbe nema ničeg na što bi se vratilo. U praksi često želimo zadržati roditelja živim kako bi nastavio izvršavati svoju primarnu zadaću, ili čak pokrenuo novu naredbu nakon što se program pokrenut s exec u child procesu završi. Ovaj obrazac slijedi UNIX ljuska svaki put kada korisnik utipka naredbu: ljuska pozivom fork stvori novi proces (kopiju same sebe), potom u tom novom procesu pozivom exec pokrene traženu naredbu, a u roditeljskom procesu čeka njen završetak pozivom wait. Ljuska ovo radi u petlji — nakon svakog wait-a korisniku daje novu ljuskinu oznaku (*prompt*) i mogućnost da unese sljedeću naredbu.

U prethodnom odjeljku, kod primjera ret_stat.c, već smo se susreli s makroom WEXITSTATUS() koji iz statusne riječi izvlači izlazni status djeteta. U pokreni2 koristimo dva dodatna makroa iz <sys/wait.h> koji nam pomažu razlikovati **na koji način** je dijete završilo:

```
WIFEXITED(status) // istina ako je dijete završilo normalno (return / exit)
WIFSIGNALED(status) // istina ako je dijete prekinuto signalom
```

```
WTERMSIG(status)    // broj signala koji je prekinuo dijete
```

Ovi makroi ispituju različite skupove bitova iste statusne riječi: WIFEXITED provjerava bit koji označava “uredan izlazak”, WIFSIGNALED ispituje bit “prekinut signalom”, a WTERMSIG izvlači broj signala iz dijela rezerviranog za signale (bitovi 0–6). Za bilo koji statusni broj točno jedan od WIFEXITED i WIFSIGNALED vratit će istinu — proces ne može istovremeno završiti i normalno i biti prekinut signalom. pokreni2 na temelju toga ispisuje različitu poruku:

```
if (WIFEXITED(s))
    printf("Normaln izlaz, izlazni status: %d\n", WEXITSTATUS(s));
else if (WIFSIGNALED(s))
    printf("Prekid signalom, signal: %d\n", WTERMSIG(s));
```

Povratna vrijednost 127, koju pokreni2 koristi u grani kad `execvp()` ne uspije, je UNIX konvencija za “naredba nije pronađena”.

Isprobajmo s različitim naredbama koje **uredno završavaju**:

```
$ ./pokreni2 ls
argumenti argumenti.c listenv1 listenv1.c ...
Normaln izlaz, izlazni status: 0

$ ./pokreni2 asdasdasd
execvp: No such file or directory
Normaln izlaz, izlazni status: 127

$ ./pokreni2 ls sdfsd fsdf
ls: cannot access 'sdfsd fsdf': No such file or directory
Normaln izlaz, izlazni status: 2
```

Zašto se izlazni status razlikuje među tri slučaja? U prvom slučaju `ls` je uspješno obavio svoj posao i `exit`-ao s 0 — to je UNIX konvencija “sve je prošlo dobro”. U drugom slučaju `execvp` u djetetu nije uspio (naredba `asdasdasd` ne postoji u `PATH`-u), pa se dijete vratilo na idući redak koda — `perror` i zatim `return 127`. U trećem slučaju `ls` se uspješno pokrenuo, ali je pri otvaranju datoteke `sdfsd fsdf` utvrdio da ne postoji, ispisao poruku o grešci na standardni izlaz za greške i vratio 2 — što je specifična konvencija same naredbe `ls` za “ozbiljnija greška”. Sva tri scenarija su uredan izlazak; razlikuje se samo vrijednost koju je dijete vratilo.

- **ptr_err.c** — primjer koji namjerno izaziva grešku u rukovanju memorijom. Sav posao programa svodi se na nekoliko redaka koda:

```
int main() {
    int *val = (int*)99;
    *val = 3;
    return 0;
}
```

Pokazivač `val` postavlja se na potpuno proizvoljnu numeričku vrijednost (99) koja sigurno nije valjana adresa u memorijskom prostoru procesa, a zatim se kroz njega pokušava upisati. Jezgra to detektira i procesu šalje signal `SIGSEGV` (signal broj 11). Ako `ptr_err` pokrenemo korištenjem našeg `pokreni2`, dobit ćemo sljedeći rezultat:

```
$ ./pokreni2 ./ptr_err
Prekid signalom, signal: 11
```

Za razliku od svih prethodnih primjera, `WIFEXITED` ovdje vraća laž — dijete nije završilo normalno — pa se izvršava `else if (WIFSIGNALED(s))` grana, a `WTERMSIG(s)` vraća 11, što je broj signala `SIGSEGV`.

Ako `ptr_err` pokušate pokrenuti direktno iz ljuske, dobit ćete ispis `Segmentation fault` — ljuska na isti način kao i naš primjer sazna razlog prekida izvršavanja

naredbe koju smo zadali, provjeri koji je signal uzrok prekida te, temeljem te informacije, ispisuje odgovarajuću poruku.

- **preusmjeri.c** — povezuje koncepte iz ovog poglavlja s mehanizmom preusmjeravanja ulaza/izlaza obrađenim u poglavlju o ulazno-izlaznim operacijama. Program prima dva ili više argumenata: prvi je ime datoteke u koju želimo preusmjeriti standardni izlaz, a ostatak je naredba koju želimo pokrenuti.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <fcntl.h>

int main(int argc, char **argv) {
    int fd;

    if (argc < 3) {
        printf("koristenje: %s <datoteka> <naredba> [arg1] [arg2] ...\n",
            argv[0]);
        return 0;
    }

    fd = creat(argv[1], (mode_t)0644);
    if (fd < 0) {
        perror("creat");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) < 0) {
        perror("dup2");
        return 1;
    }
    if (fd != STDOUT_FILENO)
        close(fd);

    execvp(argv[2], &argv[2]);
    perror("execvp");

    return 1;
}
```

Redoslijed koraka je:

1. Pozivom `creat()` otvori se izlazna datoteka s pravima 0644.
2. Pozivom `dup2(fd, STDOUT_FILENO)` file deskriptor datoteke se duplicira na deskriptor 1 — upravo onaj koji proces koristi za standardni izlaz. Od tog trenutka svaki `write` ili `printf` kojemu je odredište `STDOUT_FILENO` efektivno piše u otvorenu datoteku.
3. Stari deskriptor `fd` više nije potreban pa se zatvara, uz uobičajenu zaštitnu provjeru (`fd != STDOUT_FILENO`) za rijedak slučaj kad je `creat()` slučajno vratio baš 1.
4. Pozivom `execvp()` program se zamjenjuje traženom naredbom. Ključno zapažanje: **otvoreni file deskriptori nasljeđuju se kroz `exec()`** — nova naredba naslijedi preusmjerenje na datoteku, iako ona sama nema pojma o tome.

Dobivamo funkcionalni ekvivalent ljuškinog operatora `>`:

```
$ ./preusmjeri izlaz.txt ls -la
$ cat izlaz.txt
ukupno 24
drwxr-xr-x 2 dkrst users 4096 Apr 23 15:10 .
```

...

Na istom principu radi i sama UNIX ljuska kad obrađuje naredba `>` datoteka: prije `exec-a` naredbe, ljuska otvori datoteku i pozivom `dup2` podmetne je na deskriptor 1.

Ograničavanje resursa (`setrlimit`)

- `limitraj.c` i `potrosac.c` — ilustriraju sistemski poziv `setrlimit()`, kojim proces može postaviti ograničenje na različite resurse koje mu sustav dopušta koristiti. `limitraj` postavlja `RLIMIT_CPU` na 3 sekunde (i tvrdo i meko ograničenje), što znači da sljedeći proces smije potrošiti najviše 3 sekunde procesorskog vremena; nakon toga jezgra mu šalje signal koji ga prekida. Nakon postavljanja limita, `limitraj` pozivom `execvp()` pokreće naredbu zadanu argumentima. Kako se limiti resursa — kao i file deskriptori — **nasljeđuju kroz `exec()`**, pokrenuta naredba nasljeđuje i ograničenje. Ovo je osnovna ideja iza mehanizma ugrađene naredbe `ulimit` u ljusci, kao i alata poput `timeout` i različitih “sandbox” okruženja.

```
/* limitraj.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/resource.h>

int main(int argc, char **argv) {
    struct rlimit l;
    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ...\n", argv[0]);
        return 0;
    }

    l.rlim_cur = 3;
    l.rlim_max = 3;
    setrlimit(RLIMIT_CPU, &l);

    execvp(argv[1], &argv[1]);
    perror("execvp");

    return 1;
}
```

Program `potrosac.c` je pratilac koji služi samo za demonstraciju djelovanja `limitraj-a` — sastoji se od jedne jedine beskonačne petlje `while(1);`, bez ijednog sistemskog poziva ili I/O operacije.

```
/* potrosac.c */
int main() {
    while (1);
    return 0;
}
```

Kad se `potrosac` pokrene samostalno, trošio bi procesor neograničeno dugo. Pokrenut kroz `limitraj`, nasljeđuje njegov CPU limit:

```
$ ./limitraj ./potrosac
Killed
```

Ljuska nakon prekida procesa signalom ispisuje `Killed` (na Linux sustavima s engleskom lokalizacijom; na nekim drugim sustavima ispis može biti `CPU time limit exceeded` ili izostati sasvim). Bitno je zapaziti koliko dugo program traje — oko **tri sekunde**, a ne više.

Procesorsko i stvarno vrijeme. Limit `RLIMIT_CPU` broji isključivo **procesorsko vrijeme** (*CPU time*) — ukupno vrijeme tijekom kojeg je proces zaista izvršavao

instrukcije na nekom procesoru. To nije isto što i **stvarno vrijeme** (*wall-clock time* ili *real time*) koje mjeri koliko je proteklo vremena od pokretanja do kraja procesa, uključujući sve periode kad je proces bio pauziran od strane raspoređivača, kad je čekao na I/O, ili kad je sustav izvršavao druge procese. Ova dva vremena razlikuju se po tome da **procesorsko vrijeme nikad nije duže od stvarnog** — proces ne može utrošiti više procesorskih sekundi nego što je sekundi proteklo u stvarnosti. Najčešće je procesorsko vrijeme **kraće** od stvarnog: čim proces počne čekati na nešto (pritisak tipke, čitanje s diska, poruku preko mreže), stvarni sat nastavlja kucati, a procesorski stoji jer proces ne radi. U slučaju potrosac-a razlika između ta dva vremena je minimalna jer proces zaista cijelo vrijeme samo okreće procesor; kod realnih programa koji komuniciraju s korisnikom ili diskom razlika zna biti vrlo velika. Razlika je vidljiva i korisniku — UNIX naredba `time` ispisuje sva tri vremena (stvarno/*real*, korisničko procesorsko/*user*, sistemsko procesorsko/*sys*):

```
$ time ./limitraj ./potrosac
Killed
```

```
real    0m3.012s
user    0m3.000s
sys     0m0.001s
```

Osirotljeli procesi (engl. *orphan processes*)

- **noparent.c** — detaljniji primjer `fork()` mehanizma koji prikazuje što se dogodi kada **roditelj završi prije djeteta**. Program stvara stablo od tri procesa — PARENT 1, CHILD 1 i CHILD 2 — pri čemu CHILD 2 (unuk po odnosu prema PARENT 1) namjerno spava duže od svog direktnog roditelja (CHILD 1). Zbog toga CHILD 1 završi dok njegovo dijete još radi, a time CHILD 2 postaje **osirotljeli proces** (*orphan*). U takvoj situaciji jezgra automatski preuzima ulogu novog roditelja — tradicionalno je to proces `init` s PID-om 1.

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pid;

    pid = fork();
    if (pid == 0) { /* CHILD 1 */
        pid = fork();
        if (pid == 0) { /* CHILD 2 */
            printf("CHILD 2:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
            sleep(3);
            printf("CHILD 2:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
        } else { /* CHILD 1 - PARENT 2 */
            sleep(1);
            printf("CHILD 1:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
            exit(0);
        }
    } else { /* PARENT 1 */
        pid = wait(NULL);
        printf("PARENT 1:\t PID: %d\t izlazi: %d\n",
            getpid(), pid);
        printf("PARENT 1:\t PID: %d\t PPID: %d\n",
```

```

        getpid(), getppid());
    sleep(5);
}

return 0;
}

```

Očekivani ispis s konkretnim PID-ovima (skraćeno; SH je PID ljuske iz koje je program pokrenut, npr. 14567; PARENT 1 dobiva PID 25016, CHILD 1 25017, CHILD 2 25018):

```

CHILD 2:  PID: 25018  PPID: 25017
CHILD 1:  PID: 25017  PPID: 25016
PARENT 1: PID: 25016  izlazi: 25017
PARENT 1: PID: 25016  PPID: 14567
CHILD 2:  PID: 25018  PPID: 1

```

U prvom ispisu CHILD 2 vidi svog pravog roditelja CHILD 1 (PPID = 25017); nakon tri sekunde spavanja, njegov drugi ispis prijavljuje **PPID = 1** — u međuvremenu je CHILD 1 završio, a jezgra je posvojenje prenijela na init.

Napomena o modernim sustavima. Na suvremenim Linux distribucijama koje koriste systemd i korisničke sesije (systemd --user), posljednji PPID često neće biti 1 nego PID korisničkog systemd procesa. Razlog je mehanizam **subreapera** uveden u Linux jezgri 3.4 (2012.): pozivom prctl(PR_SET_CHILD_SUBREAPER, 1) proces se može registrirati kao “zamjenski init” za sve svoje potomke. Kad sirotan nastane, jezgra traži najbližeg pretka-subreapera umjesto da ga automatski pošalje na PID 1. Na desktop Linuxu sa systemd-om upravo je systemd --user takav subreaper, pa se tamo vidi PPID jednak njegovom PID-u. Konkretni iznos može se provjeriti naredbom `ps -p <ppid> -o pid,ppid,comm`. Klasično pravilo (*sirotan* → *PID 1*) i dalje vrijedi u odsutnosti subreapera — primjerice pri pokretanju iz SSH sesije na minimalnom serveru.

Prevođenje

Direktorij dolazi s priloženim **Makefile**-om koji prati iste konvencije kao i Makefile datoteke u direktorijima P02-Osnove_programiranja/ i P03-Ulazno_izlazne_operacije/ (varijable CC, CFLAGS, LDFLAGS, TARGETS; implicitno pravilo .c.o; pravila default, all, clean). Detaljan opis strukture i korake gradnje Makefilea vidjeti u [../P02-Osnove_programiranja/README.md](#).

Tipična uporaba:

```

make          # gradi zadani cilj (novi)
make all      # gradi sve primjere
make pokreni2 # gradi samo zadani primjer
make clean    # briše izvršne i objektne datoteke

```

Signali

U ovom poglavlju upoznat ćemo **signale** — UNIX-ov primarni mehanizam za asinkronu komunikaciju s procesom. Signal je kratka poruka koju jezgra šalje procesu kao obavijest da se dogodio neki događaj: korisnik je pritisnuo Ctrl+C, proces je pokušao pristupiti nedopuštenoj memorijskoj adresi, istekao je timer, neki drugi proces je eksplicitno zatražio prekid, i tako dalje. Iz perspektive procesa, signal može stići u **bilo kojem trenutku** između dvije strojne instrukcije — proces nema mogućnost predvidjeti kada će se to dogoditi.

Proces na primljeni signal može reagirati na nekoliko načina: prepustiti zadanu reakciju jezgri (što za većinu signala znači prekid procesa), eksplicitno ignorirati signal, ili registrirati vlastitu funkciju — **rukovatelj signala** (engl. *signal handler*) — koja će se izvršiti svaki put kad takav signal stigne. U ovom poglavlju kroz nekoliko primjera ilustriramo kako registriamo rukovatelj signala, kako se signali hvataju, kako se koriste za komunikaciju među procesima i o čemu je potrebno voditi računa pri pisanju rukovatelja.

Najčešći signali

Signali su, gledano iz perspektive operacijskog sustava i programa, jednostavno **mali cjelobrojni identifikatori** — svaki signal ima svoj broj. Zbog čitljivosti i prenosivosti koda, u programima nikad ne baratamo izravno tim brojevima, nego koristimo **simboličke konstante** definirane u standardnoj zaglavnoj datoteci <signal.h> (npr. SIGINT, SIGTERM, SIGKILL...).

POSIX standard definira tridesetak signala, a u praksi se najčešće susrećemo sa skupinom njih desetak. Tablica niže daje pregled onih s kojima ćemo raditi u ovom poglavlju i u tipičnim programima:

Signal	Broj	Predefinirana akcija	Značenje
SIGHUP	1	prekid procesa	terminal je zatvoren ili je sesija prekinuta; često se koristi i kao “ponovno učitaj konfiguraciju”
SIGINT	2	prekid procesa	korisnik je pritisnuo Ctrl+C u terminalu; programi ga često hvataju kako bi od korisnika zatražili potvrdu izlaska — “čisti” izlaz
SIGQUIT	3	prekid procesa + core file	korisnik je pritisnuo Ctrl+ — kao SIGINT, ali generira i core file

Signal	Broj	Predefinirana akcija	Značenje
SIGILL	4	prekid procesa + core file	proces je pokušao izvršiti nevažeću strojnu instrukciju
SIGABRT	6	prekid procesa + core file	proces je sam sebe prekinuo pozivom <code>abort()</code> (npr. zbog <code>assert</code> greške)
SIGFPE	8	prekid procesa + core file	aritmetička greška (dijeljenje s nulom, prelijevanje)
SIGKILL	9	prekid procesa	bezuvjetni prekid — ne može se uhvatiti niti ignorirati ; jezgra ga koristi kao zadnju mjeru kad se proces ogлуši o SIGTERM
SIGSEGV	11	prekid procesa + core file	proces je pokušao pristupiti memorijskoj adresi kojoj nema pravo pristupa (<i>segmentation fault</i>)
SIGPIPE	13	prekid procesa	pokušaj pisanja u cijevovod (<i>pipe</i>) ili socket čiji je drugi kraj zatvoren
SIGALRM	14	prekid procesa	istekao je timer postavljen pozivom <code>alarm()</code>
SIGTERM	15	prekid procesa	pristojan zahtjev za prekid; programi ga često koriste za uredno spremanje stanja i "čisti" izlaz. Ako se program ogлуši o SIGTERM, jezgra ga može bezuvjetno prekinuti signalom SIGKILL
SIGUSR1	10	prekid procesa	korisnički signal br. 1 — bez unaprijed definirane semantike, slobodan za vlastite svrhe
SIGUSR2	12	prekid procesa	korisnički signal br. 2 — bez unaprijed definirane semantike, slobodan za vlastite svrhe
SIGCHLD	17	ignorira se	dijete procesa je promijenilo stanje (završilo, zaustavljeno itd.)

Signal	Broj	Predefinirana akcija	Značenje
SIGSTOP	19	zaustavi proces	bezuovjetno zaustavljanje — ne može se uhvatiti niti ignorirati
SIGCONT	18	nastavi proces	nastavi izvršavanje zaustavljenog procesa
SIGTSTP	20	zaustavi proces	korisnik je pritisnuo Ctrl+Z u terminalu — svojevrsna “pauza” za pokrenuti program (zaustavlja ga privremeno; nastavak slanjem SIGCONT)
SIGXCPU	24	prekid procesa + core file	premašen je CPU limit postavljen <code>setrlimit()</code> -om (vidi Limiti)
SIGXFSZ	25	prekid procesa + core file	premašen je file size limit postavljen <code>setrlimit()</code> -om (vidi Limiti)

Brojevi signala u tablici odgovaraju POSIX-u za osnovne signale i podudaraju se s vrijednostima na većini modernih UNIX sustava. Ipak, najsigurniji pristup je u kodu uvijek koristiti **simbolička imena** (SIGINT, SIGTERM...) umjesto numeričkih vrijednosti — time se izbjegavaju zabune i osigurava prenosivost koda na sustave gdje neki manje uobičajeni signali mogu imati drukčije brojeve. Brojeve navodimo samo radi reference (npr. uz ispis “Prekid signalom, signal: 11” iz `pokreni2-a`). Potpuni popis dostupan je u priručniku: `man 7 signal`.

Posebnu pažnju zaslužuju **SIGKILL** i **SIGSTOP** — to su jedina dva signala koja se ne mogu uhvatiti niti ignorirati. Razlog je praktičan: bez ova dva signala sustav ne bi imao apsolutni mehanizam za bezuvjetan prekid ili zaustavljanje “neposlušnog” procesa. Svaki drugi signal proces može uhvatiti i odlučiti kako reagirati — uključujući i SIGTERM, što neki programi iskorištavaju za uredno spremanje stanja prije izlaska.

Što je core file? Za neke signale predefinirana akcija nije samo prekid procesa, nego i stvaranje *core file*-a — datoteke koja sadrži snimku kompletnog memorijskog prostora procesa u trenutku kad je signal primljen (varijable na stogu, hrpi, registri procesora, otvoreni file deskriptori i drugi metapodatci). Datoteka se obično zove `core` ili `core.<pid>` i nastaje u trenutnom radnom direktoriju procesa. Ovo ponašanje vezano je uz signale koji uglavnom znače da smo u programu *nešto zabrljali* — SIGSEGV (pristup nevažećoj memoriji), SIGFPE (aritmetička greška), SIGILL (nevažeća instrukcija), SIGABRT (eksplicitan prekid pomoću `abort()`). Core file omogućuje *post-mortem* analizu: alatima poput gdb programer može učitati core file zajedno s izvršnom datotekom, vidjeti gdje je proces bio u trenutku pada, koje su vrijednosti varijabli imale, kakav je bio stack trace itd. Naravno, ako za to imamo volje i znanja — u suprotnom core file se može jednostavno obrisati.

Hvatanje signala

- **potvrđi.c** — minimalan primjer hvatanja signala koji uvodi sve ključne koncepte rada s njima. Program registrira vlastiti rukovatelj za signal SIGINT (signal koji se procesu šalje kad korisnik u terminalu pritisne Ctrl+C); kad korisnik pritisne Ctrl+C prvi put, program ne završi, nego ispiše poruku da treba pritisnuti Ctrl+C još jednom ako se

zaista želi izaći. Ovo je obrazac kakav vidamo u stvarnim alatima (npr. htop, vim) i u skriptama koje se ne smiju nehotice prekinuti.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;

void uhvati(int signum) {
    brojac++;
}

int main() {
    signal(SIGINT, uhvati);

    while (brojac < 2) {
        pause();
        if (brojac == 1)
            printf("Pritisnite ponovo CTRL - C ukoliko zelite izaci\n");
    }

    return 0;
}
```

Za registraciju rukovatelja koristi se sistemski poziv signal:

```
#include <signal.h>

void (*signal(int signum, void (*handler)(int)))(int);
```

Na prvi pogled deklaracija djeluje zastrašujuće, ali zapravo je riječ o funkciji koja prima dva argumenta i vraća pokazivač:

- **signum** — broj signala koji želimo hvatati (npr. SIGINT)
- **handler** — rukovatelj signala; pokazivač na funkciju koja će biti pozvana kada signal signum stigne. Rukovatelj signala kao argument prima jednu cjelobrojnu vrijednost (int) koja označava broj signala koji je pozvao rukovatelj (isti rukovatelj može se pozivati za više različitih signala). Rukovatelj signala ima povratni tip void — nema povratnu vrijednost.
- **Povratna vrijednost** — pokazivač na *prethodno* registriranu funkciju, ili SIG_ERR u slučaju greške.

Ovdje se prvi put susrećemo s **pokazivačem na funkciju** kao argumentom drugoj funkciji. Ideja je jednostavna: kako svaka funkcija, baš kao i svaka varijabla, u izvršnoj datoteci ima svoju adresu u memoriji, tako i njezino ime u kodu (bez zagrada) jednostavno označava upravo tu adresu. Drugim riječima, **ime funkcije je njezin pokazivač**. U našem programu funkcija uhvati definirana je tako da prima int i ne vraća ništa — što točno odgovara tipu argumenta handler — pa je dovoljno predati samo njezino ime: signal(SIGINT, uhvati).

Mehanizam je sljedeći. Funkcija signal(SIGINT, uhvati) jezgri kaže: “od ovog trenutka, kad god mom procesu stigne signal SIGINT, ne primjenjuj zadanu reakciju (prekid procesa) nego pozovi funkciju uhvati.” Kad korisnik pritisne Ctrl+C, jezgra prekine trenutno izvršavanje glavnog programa točno tamo gdje je bilo, pozove uhvati, a kad se ona vrati, glavni program nastavlja izvršavanje od mjesta gdje je bio prekinut. Sav posao koji rukovatelj napravi — u ovom slučaju jednostavno povećanje varijable brojac — vidljiv je glavnom programu kroz globalnu varijablu, što je standardni način “razgovora” između main-a i rukovatelja.

Glavni program koristi sistemski poziv pause(), koji uspava proces sve dok ne stigne bilo koji signal. Kad signal stigne i njegov rukovatelj završi izvršavanje, pause() se

vрати i petlja nastavlja: provjeri trenutnu vrijednost brojac-a, ako je on 1 ispiše uputu, a u sljedećem prolazu petlje opet uđe u pause() čekajući novi signal. Tek kad brojac dosegne 2, izlazi iz petlje i program uredno završava.

Pokrenimo program i isprobajmo:

```
$ ./potvrdi
^C
Pritisnite ponovo CTRL - C ukoliko zelite izaci
^C
Korisnik je potvrdio odlazak - kraj programa!
$
```

Da bismo bolje uočili koliki je doprinos rukovatelja, predlažemo i sljedeći eksperiment: zakomentirajte redak `signal(SIGINT, uhvati);`, ponovno prevedite program i pokrenite ga. Pritisak `Ctrl+C` sada će dovesti do trenutnog prekida programa — nećete vidjeti ni poruku iz petlje, ni završnu poruku. Bez registriranog rukovatelja primjenjuje se **zadana reakcija** jezgre na `SIGINT`, a ona za ovaj signal podrazumijeva prekid procesa.

Funkcionalno je program gotovo trivijalan, ali pokriva nekoliko važnih koncepata vrijednih da se odmah istaknu:

- **Rukovatelj je vrlo kratak** — samo inkrementira brojč i ne pokušava ništa složenije od toga (npr. nema poziva `printf`-a). Ovo nije slučajno: rukovatelj signala se izvršava u posebnom kontekstu — može prekinuti glavni program u doslovno bilo kojem trenutku, uključujući i sredinu poziva drugih funkcija. Preporuka je iz rukovatelja pozivati samo tzv. **async-signal-safe** funkcije — funkcije za koje POSIX standard jamči da ih je sigurno pozvati iz signal handlera. U taj skup ne spada i `printf` — njegovo korištenje u rukovatelju može u rijetkim slučajevima dovesti do iznenađujućih grešaka. Detaljnije ćemo o ovome u kasnijem primjeru, ali već sad uvodimo dobru praksu: rukovatelj postavlja zastavicu, glavni program reagira.
- **Komunikacija kroz globalnu varijablu** — rukovatelj i glavni program “razgovaraju” kroz brojč. Strogo gledano, takve dijeljene varijable trebale bi biti deklarirane s tipom `volatile sig_atomic_t` umjesto običnog `int`-a:
 - * Ključna riječ `volatile` govori prevoditelju da vrijednost varijable može biti promijenjena “iza leđa” glavnog programa (od strane rukovatelja), pa optimizator ne smije njezinu vrijednost cache-irati u registru kroz iteracije petlje.
 - * Tip `sig_atomic_t` jamči da se čitanje i pisanje varijable obavlja u jednoj nedjeljivoj operaciji — rukovatelj ne može uhvatiti glavni program “u sredini” upisa.

Na većini modernih arhitektura (uključujući x86) u praksi će raditi i obični `int`.

Napomena. U nekim povijesnim verzijama UNIX-a rukovatelj signala resetirao bi se svaki put kada bi signal bio primljen, pa ga je bilo potrebno ponovno registrirati. U modernim verzijama UNIX-a ovo ponašanje gotovo sigurno nećete susresti.

Vlastiti alarm — `SIGALRM` i `alarm()`

- **alarm_clock.c** — primjer u kojem proces zakaže slanje signala samome sebi. Izvor signala najčešće dolazi izvan samog procesa, npr. ukoliko korisnik pritisne `Ctrl+C`, ili eksplicitno zatražimo slanje signala pozivom `kill` iz ljuške. UNIX, međutim, omogućuje i da proces zatraži od jezgre da mu nakon određenog broja sekundi pošalje signal `SIGALRM`. Tu funkcionalnost pruža sistemski poziv `alarm()`:

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

Ovaj poziv jezgri kaže: “za točno *seconds* sekundi pošalji mi *SIGALRM*.” Ako je već postojao prethodni alarm, on se zamjenjuje novim, a poziv vraća broj sekundi koje su preostale do isporuke prethodnog alarma. Pozivom `alarm(0)` aktivni alarm se otkazuje. Bitno je primijetiti da je `alarm` **jednokrat**an — ako želimo periodičko “tikanje” svakih *N* sekundi, novi alarm moramo zakazati svaki put iznova.

Upravo to radi naš primjer:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;

void alrm_handler(int signum) {
    brojac++;
    alarm(1);
}

int main() {
    signal(SIGALRM, alrm_handler);
    alarm(1);

    while (brojac < 5) {
        pause();
        printf("tik %d\n", brojac);
    }

    printf("kraj!\n");
    return 0;
}
```

Glavni program registrira rukovatelj za *SIGALRM*, postavlja prvi alarm na jednu sekundu i ulazi u petlju koja čeka pet “tikanja”. Rukovatelj je vrlo kratak — samo poveća brojač i odmah zakaže sljedeći alarm — pa se ovaj ciklus odvija jednom u sekundi. Ispis “tik *N*” obavlja sam glavni program nakon što se vrati iz `pause()`-a, sljedeći obrazac koji smo uveli kod prethodnog primjera (*rukovatelj postavlja zastavicu, glavni program reagira*).

Pokrenimo program:

```
$ ./alarm_clock
tik 1
tik 2
tik 3
tik 4
tik 5
kraj!
```

Cijeli ispis traje točno pet sekundi.

Primijetite da smo rukovatelj imenovali `alrm_handler` umjesto naprosto uhvati, kao u prethodnom primjeru. Pri imenovanju funkcija dobra je praksa odabrati intuitivna imena koja odmah daju naslutiti što funkcija radi. U slučaju rukovatelja signala nije loše u ime ugraditi i ime samog signala — npr. `alrm_handler` za *SIGALRM*, ili `int_handler` za *SIGINT*, što bi u prethodnom primjeru bilo prikladnije ime od općenitog uhvati. Ovo je naravno samo sugestija autora; čitatelj ima punu slobodu

imenovati svoje funkcije kako god mu odgovara.

- **stoperica.c** — primjer koji kombinira tehnike iz prethodna dva: koristi SIGALRM za odbrojavanje sekundi, a SIGINT (Ctrl+C) za zaustavljanje. Funkcionalno se ponaša kao jednostavna stoperica: nakon pokretanja svake sekunde ispiše “tik N”, a kad korisnik pritisne Ctrl+C ispiše ukupno proteklo vrijeme i uredno završi.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;
int broji = 1;

void alm_handler(int signum) {
    brojac++;
    alarm(1);
}

void int_handler(int signum) {
    broji = 0;
}

int main() {
    signal(SIGALRM, alm_handler);
    signal(SIGINT, int_handler);

    printf("Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.\n");
    alarm(1);

    while (broji) {
        pause();
        if (broji)
            printf("tik %d\n", brojac);
    }

    printf("Proteklo: %d sekundi\n", brojac);
    return 0;
}
```

Program ima dvije globalne varijable kroz koje glavni program i rukovatelji “razgovaraju”: brojac koji broji koliko je sekundi prošlo i broji koji upravlja izvršavanjem glavne petlje. Registrirana su dva rukovatelja s različitim ulogama: alm_handler poveća brojač i ponovno zakaže alarm (točno kao u prethodnom primjeru), dok int_handler postavi broji = 0 čime signalizira glavnoj petlji da treba završiti.

Glavna petlja while (broji) u svakom prolazu pasivno čeka signal pomoću pause(). Kad signal stigne i odgovarajući rukovatelj završi, pause() se vrati i petlja nastavlja. Slijedi mali ali važan detalj: prije ispisa “tik N” provjeravamo da je broji i dalje istinit. Razlog je u tome što se pause() vraća za **bilo koji** primljeni signal, uključujući i SIGINT. Bez te provjere, posljednji ispis bio bi suvišan “tik N” prije završne poruke “Proteklo: N sekundi”. Ovaj obrazac — *nakon pause(), ponovo provjeri stanje pa tek onda reagiraj* — tipičan je kad više signala utječe na istu petlju.

Pokrenimo stopericu i nakon nekoliko sekundi pritisnimo Ctrl+C:

```
$ ./stoperica
Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.
tik 1
tik 2
```

tik 3

^CProteklo: 3 sekundi

Imenovanjem rukovatelja prema signalu na koji odgovaraju (alarm_handler i int_handler), kod postaje samodokumentirajuć — već iz naziva je jasno koji rukovatelj reagira na koji signal, što je posebno korisno kad u programu imamo više signala koje hvatamo.

- **stoperica2.c** — funkcionalno identičan prethodnom programu, ali strukturno drukčiji: koristi **jedan zajednički rukovatelj** za oba signala umjesto dva odvojena. Na ovom primjeru ilustrirano je korištenje argumenta signum koji rukovatelj prima — broj signala koji je upravo isporučen procesu.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;
int broji = 1;

void rukovatelj(int signum) {
    switch (signum) {
        case SIGALRM:
            brojac++;
            alarm(1);
            break;
        case SIGINT:
            broji = 0;
            break;
    }
}

int main() {
    signal(SIGALRM, rukovatelj);
    signal(SIGINT, rukovatelj);

    printf("Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.\n");
    alarm(1);

    while (broji) {
        pause();
        if (broji)
            printf("tik %d\n", brojac);
    }

    printf("Proteklo: %d sekundi\n", brojac);
    return 0;
}
```

Do sada smo argument rukovatelja, iako ga je deklaracija zahtijevala, jednostavno ignorirali. U ovom primjeru njegova svrha postaje jasna: kad jedna funkcija obrađuje više različitih signala, jezgra joj kao argument predaje broj signala koji je upravo isporučen, pa funkcija na temelju toga može odlučiti što dalje. switch je u takvim slučajevima prirodniji od lanca if/else if jer se lakše proširuje dodavanjem novih case grana za nove signale.

Pokretanje i ispis su identični prethodnom primjeru:

```
$ ./stoperica2
```

```
Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.
```

```
tik 1
```

```
tik 2
tik 3
^CProteklo: 3 sekundi
```

Koji je pristup bolji — razdvojeni rukovatelji kao u `stoperica.c` ili zajednički kao u `stoperica2.c`? Stvar je ukusa i konteksta. Razdvojeni rukovatelji su pregledniji kad je logika za svaki signal značajno različita i kad u svakom rukovatelju ima više od nekoliko redaka koda. Zajednički rukovatelj je prikladan kad signali dijele zajedničke resurse ili pomoćne varijable, ili kad očekujemo da će se broj obrađivanih signala s vremenom povećavati. U realnim programima često su zastupljena oba pristupa istovremeno: na primjer, jedan zajednički rukovatelj za sve signale koji označavaju zahtjev za prekidom (SIGINT, SIGTERM, SIGHUP) i poseban rukovatelj za vremenske signale.

Korištenje signala za komunikaciju između procesa

U dosadašnjim primjerima signali su dolazili iz dva izvora: izvana, od korisnika preko Ctrl+C, ili iznutra, kad ih je proces sam sebi zakazao pozivom `alarm()`. UNIX, međutim, dopušta i da jedan proces eksplicitno **pošalje signal drugom procesu** — što čini signale jednim od najjednostavnijih oblika međuprocenjske komunikacije (engl. *Inter-Process Communication*, IPC). Za to služi sistemski poziv `kill`:

```
#include <signal.h>
#include <sys/types.h>

int kill(pid_t pid, int sig);
```

Povratna vrijednost: 0 u slučaju uspjeha; -1 u slučaju greške (npr. ne postoji proces s navedenim PID-om, ili nemamo dovoljnu razinu ovlasti za slanje signala tom procesu).

Ime `kill` je povijesno — prvotno je sistemski poziv služio isključivo za prekid drugog procesa pomoću SIGKILL ili SIGTERM. Vremenom se njegova uloga proširila i danas se koristi za slanje **bilo kojeg** signala, ali ime je ostalo. Istog imena je i istoimena ugrađena naredba ljske (`kill -<sig> <pid>`, npr. `kill -USR1 12345`), kojom korisnik može poslati proizvoljni signal nekom procesu iz terminala.

- **razgovor.c** — primjer u kojem dva procesa komuniciraju isključivo putem signala. Roditelj fork-a dijete, zatim mu u petlji svake sekunde šalje SIGUSR1. Dijete na svaki primljeni signal ispiše poruku i broji koliko ih je primilo. Nakon pet “dojava”, roditelj djetetu pošalje SIGTERM da uredno završi, pa pričekava njegov završetak pozivom `wait()`.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int radi = 1;
int prosao = 0;

void usr1_handler(int signum) {
    prosao++;
}

void term_handler(int signum) {
    radi = 0;
}

int main() {
    pid_t pid = fork();
```

```

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    /* CHILD */
    signal(SIGUSR1, usr1_handler);
    signal(SIGTERM, term_handler);

    while (radi) {
        pause();
        if (radi)
            printf("[child] primio sam SIGUSR1, prolaz %d\n", prosao);
    }

    printf("[child] završavam, ukupno prolaza: %d\n", prosao);
    return 0;
}

/* PARENT */
printf("[parent] pokrenuo dijete s PID-om %d\n", pid);

for (int k = 0; k < 5; k++) {
    sleep(1);
    printf("[parent] šaljem SIGUSR1 djetetu\n");
    kill(pid, SIGUSR1);
}

sleep(1);
printf("[parent] šaljem SIGTERM djetetu\n");
kill(pid, SIGTERM);

wait(NULL);
printf("[parent] dijete je završilo, izlazim\n");
return 0;
}

```

Struktura programa razdvaja se odmah nakon `fork()`-a u dvije grane. **Dijete** registrira dva rukovatelja: `usr1_handler` koji broji primljene `SIGUSR1` signale, i `term_handler` koji postavlja `radi = 0` čime signalizira glavnoj petlji da treba završiti. Petlja je standardna `while (radi) pause()` konstrukcija s provjerom `if (radi)` prije `printf`-a, kao u `stoperica.c`-u, da se izbjegne suvišan ispis nakon `SIGTERM`-a.

Bitno je primijetiti da se rukovatelji **registriraju samo u dječjoj grani**, a ne i u roditelju. Razlog je u tome što roditelj ove signale uopće ne prima — on je **pošiljatelj**, a ne primatelj. Roditelj eksplicitno šalje `SIGUSR1` i `SIGTERM` djetetu pozivom `kill(pid, ...)`, a sam ne treba reagirati na te signale. Da smo registraciju rukovatelja stavili **prije** `fork()`-a, oba bi je procesa naslijedila — pa bi i roditelj imao registrirane rukovatelje koji se nikad ne bi izvršili, što ne bi bila greška, ali je nepotrebno. Stavljanjem registracije unutar dječje grane jasno odvajamo uloge: **dijete reagira, roditelj šalje**.

Roditelj zna PID djeteta jer mu ga je `fork()` vratio, pa može pozivati `kill(pid, SIGUSR1)` da mu šalje signale u željenim trenucima. Po završetku radne petlje šalje `SIGTERM`, a zatim `wait(NULL)` čeka da dijete uredno završi (čime se ono prestaje voditi kao zombi proces u tablici procesa, kao što smo objasnili u prethodnom poglavlju).

Pokrenimo program:


```

$ ./razgovor
[parent] pokrenuo dijete s PID-om 12345
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 1
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 2
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 3
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 4
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 5
[parent] saljem SIGTERM djetetu
[child] završavam, ukupno prolaza: 5
[parent] dijete je završilo, izlazim

```

Iz ispisa je vidljiv pravilan redoslijed događaja: roditelj svake sekunde signalizira djetetu, dijete trenutno reagira, i tako pet puta zaredom; konačno SIGTERM zatvara petlju u djetetu, a roditelj nakon `wait()`-a izlazi.

Bitno je naglasiti da `kill(pid, sig)` samo “isporuči” signal djetetu — ne čeka da rukovatelj završi izvršavanje, a nema nikakve garancije da će signal biti obrađen prije nego roditelj nastavi sa svojim radom. Ovo je primjer **asinkronog** mehanizma: pošiljalac i primatelj nisu usklađeni vremenski. Za pravu sinkroniziranu komunikaciju (gdje pošiljalac čeka primateljev odgovor) postoje drugi UNIX IPC mehanizmi — cijevi, redovi poruka, dijeljena memorija — kojima ćemo se baviti u kasnijim poglavljima.

Značenje signala.

Većina UNIX signala ima predefinirano značenje. Međutim, kako programer može napisati vlastiti rukovatelj za većinu signala, time praktički može promijeniti njihovo predefinirano značenje — npr. iskoristiti SIGSEGV (kojim nas jezgra upozorava da smo s pokazivačem izašli izvan svog memorijskog prostora) za razmjenu poruka između roditelja i djeteta, kao u našem primjeru. Iako ovo nije nikakav problem implementirati, ideja je vrlo loša: što ako stvarno u programu napravimo grešku u rukovanju memorijom i jezgra nas pokuša na to upozoriti, a mi taj signal protumačimo kao poruku od drugog procesa?

Ideja da signalima pridijelimo posve drugačije značenje otprilike je jednako dobra kao da se studenti koji slušaju kolegij *Programiranje za UNIX* na FESB-u dogovore da crveno svjetlo na semaforu za njih znači “kreni”, a zeleno “stani”: dok su sami na cesti, sustav može funkcionirati, ali ako se pojavi bilo koji drugi vozač, posljedice su potencijalno katastrofalne.

Iz istog razloga treba poštivati predefinirana značenja signala, a za vlastite komunikacijske protokole između grupe procesa koristiti slobodne signale SIGUSR1 i SIGUSR2, kao u našem primjeru.

Sistemska poziva `sigaction`

U svim dosadašnjim primjerima rukovatelj smo registrirali korištenjem funkcije `signal()`. Iako je `signal()` jednostavan i intuitivan, njegova povijest puna je nedosljednosti između UNIX implementacija. U starijim System V verzijama rukovatelj se nakon prve isporuke signala automatski poništavao (resetirao na zadanu reakciju), pa je sljedeća instanca istog signala — koja stigne prije nego rukovatelj stigne ponovno registrirati samog sebe — uzrokovala prekid procesa. Na BSD sustavima rukovatelj je ostajao registriran. Drugi izvor razlika bilo je ponašanje **prekinutih sistemskih poziva**: kad signal stigne procesu koji je u sporom sistemskom pozivu (`read`, `accept`, `pause`, `wait`...), BSD je sistemski poziv automatski nastavljao, dok je System V vraćao grešku s `errno = EINTR`. Treći problem je

“atomarnost” registracije — bilo je moguće da signal stigne usred zamjene rukovatelja i pozove pogrešnu funkciju.

Da bi se ove razlike uklonile i ponašanje precizno definiralo, POSIX uvodi noviju funkciju `sigaction()`, koja u potpunosti zamjenjuje `signal()` i nudi dodatne mogućnosti: blokiranje drugih signala tijekom obrade rukovatelja, kontrolu nad nastavkom prekinutih sistemskih poziva, te alternativnu formu rukovatelja koja prima detaljne informacije o izvoru signala.

```
#include <signal.h>

int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);

struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void); /* obsolete, ne dirati */
};
```

Argumenti `sigaction()`-a su:

- **signum** — broj signala koji se hvata (kao i kod `signal()`-a),
- **act** — pokazivač na strukturu koja opisuje novu akciju za taj signal,
- **oldact** — pokazivač na strukturu u koju će se upisati **prethodna** akcija; korisno ako želimo privremeno preuzeti signal pa kasnije vratiti staro ponašanje. Ako nas prethodna akcija ne zanima, kao treći argument možemo koristiti `NULL` pokazivač.

Najvažnija polja strukture `struct sigaction`:

- **sa_handler** — pokazivač na funkciju koja će biti pozvana kad signal stigne, isto kao drugi argument funkcije `signal()`. Umjesto pokazivača na funkciju, polju se mogu dodijeliti i posebne vrijednosti `SIG_DFL` (vraća zadanu reakciju jezgre) ili `SIG_IGN` (ignoriraj signal).
- **sa_mask** — skup signala koje treba blokirati **dok se rukovatelj izvršava**. Po POSIX defaultu, dok se izvršava rukovatelj za neki signal `S`, taj isti signal `S` automatski je blokirani — ako tijekom obrade `S`-a stigne nova instanca, ona čeka u redu (engl. *pending*) i isporučuje se tek nakon što rukovatelj završi. Polje `sa_mask` proširuje ovo blokiranje: tu možemo navesti **dodatne** signale koji će biti blokirani tijekom izvršavanja rukovatelja. Ovo je iznimno korisno kad više signala dijele isti rukovatelj ili manipuliraju istim podacima — postavljanjem cross-maske između njih sprječavamo da jedan rukovatelj prekine drugog usred kritične sekcije.
- **sa_flags** — bit-mask za zastavicu koje fino podešavaju ponašanje. Najčešće korištene su:
 - `SA_RESTART` — automatski nastavi prekinute sistemske pozive (BSD ponašanje); bez ove zastavice, sistemski poziv prekinut signalom vraća grešku `EINTR`.
 - `SA_NOCLDWAIT` — kad se postavi za `SIGCHLD`, jezgra automatski uklanja zombije bez potrebe za `wait()`-om.
 - `SA_SIGINFO` — proširuje rukovatelj dodatnim informacijama o signalu (vidi opis polja `sa_sigaction` niže).

Polje `sa_sigaction` je alternativa polju `sa_handler` — koristi se uz zastavicu `SA_SIGINFO` i daje rukovatelju prošireni potpis `void f(int signum, siginfo_t *info, void *context)`. Drugi argument `info` je struktura s detaljnim podacima o signalu (PID procesa pošiljatelja, UID njegovog vlasnika, razlog isporuke...). Treći argument `context` daje pristup CPU registrima u trenutku prekida (rijetko se koristi izravno). Dva polja, `sa_handler` i `sa_sigaction`, na nekim su sustavima implementirana kao `union` — pa je dobra praksa koristiti samo jedno od njih i nikad oba istovremeno. Polje `sa_restorer` je interni Linux mehanizam i ne smije se eksplicitno postavljati. Zato je dobra praksa cijelu strukturu prije korištenja inicijalizirati `memset`-om, čime osiguravamo da neiskorištena polja imaju nulte

vrijednosti.

Atomarna zamjena rukovatelja. Bitno je razumjeti da `sigaction()` postavlja novu akciju atomski: nije moguće da signal stigne usred izmjene, pronade pola-staropola-novo stanje, i pozove pogrešan rukovatelj. To je važna garancija koju System V `signal()` nije pružao.

Postupak za registraciju rukovatelja `sigaction()`-om uvijek slijedi isti obrazac:

1. Deklarirati `struct sigaction sa`,
2. Nulirati strukturu pomoću `memset(&sa, 0, sizeof(sa))`,
3. Postaviti `sa.sa_handler` na pokazivač na rukovatelj,
4. Inicijalizirati masku pomoću `sigemptyset(&sa.sa_mask)` (ili dodati signale koje treba blokirati),
5. Postaviti `sa.sa_flags` na željenu kombinaciju zastavica (najčešće 0),
6. Pozvati `sigaction(signum, &sa, NULL)`.

Za sav novi kod preporučuje se `sigaction()` umjesto `signal()`. U ostatku ovog poglavlja ipak smo se služili objema funkcijama: `signal()` u jednostavnijim primjerima gdje smo htjeli zadržati pregledan kod, a `sigaction()` ondje gdje su nam potrebne njegove dodatne mogućnosti.

- **potvrđi2.c** — funkcionalno identičan primjeru `potvrđi.c` s početka poglavlja, ali rukovatelj se registrira pomoću `sigaction()`-a:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

int brojac = 0;

void int_handler(int signum) {
    brojac++;
}

int main() {
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = int_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);

    while (brojac < 2) {
        pause();
        if (brojac == 1)
            printf("Pritisnite ponovo CTRL - C ukoliko zelite izaci\n");
    }

    printf("Korisnik je potvrdio izlazak - kraj programa!\n");
    return 0;
}
```

Glavna petlja, rukovatelj i logika su nepromijenjeni u odnosu na `potvrđi.c` — razlika je samo u načinu registracije. Iako je ovo nešto više koda od jednostavnog `signal(SIGINT, int_handler)` poziva, zauzvrat dobivamo precizno definirano ponašanje koje vrijedi na svim POSIX sustavima.

Pokretanje i ispis su identični prethodnom primjeru:

```

$ ./potvrđi2
^C
Pritisnite ponovo CTRL - C ukoliko želite izaci
^C
Korisnik je potvrdio izlazak - kraj programa!
$

```

Blokiranje i ignoriranje signala

Do sada smo signale uvijek “hvatali” — registrirali rukovatelja koji bi se pozvao kad signal stigne. UNIX, međutim, nudi i druge načine da odredimo što se događa kad signal stigne procesu. Dva najvažnija su **blokiranje** i **ignoriranje**, i između njih postoji važna razlika.

Blokiranje ne uklanja signal — samo odgađa njegovu isporuku. Svaki proces ima takozvanu **masku signala** (engl. *signal mask*): skup signala koji su trenutno blokirani. Maska signala je još jedan od podataka o procesu koje jezgra čuva u tablici procesa — uz sve drugo što smo dosad spominjali (PID, PPID, tablica otvorenih datoteka, izlazni status, limiti resursa). Postupno se vidi koliko različitih informacija jezgra mora držati za svaki proces, sve uredno upakirano u jedan slog tablice procesa. Kad signal stigne procesu, a taj signal je u njegovoj maski, jezgra ga pohrani u **red čekanja** (engl. *pending*). Tamo ostaje sve dok proces ne ukloni signal iz maske — tek tada se isporučuje, i tek tada se pokreće rukovatelj (ili zadana akcija). Blokiranje koristimo kad imamo dio koda koji ne smije biti prekinut signalom, ali ne želimo trajno izgubiti signal.

Ignoriranje je kvalitativno drugačije: signal se isporučuje, ali se odmah odbacuje. Proces ne saznaje da je signal stigao i nikad ne reagira na njega. Za ignoriranje koristimo specijalnu vrijednost SIG_IGN koju postavimo kao “rukovatelj” pomoću sigaction()-a (ili signal()-a). Ignoriranje koristimo kad nas određeni signal jednostavno ne zanima.

Sistemska poziv sigprocmask

Maska signala glavnog programa mijenja se sistemskim pozivom sigprocmask:

```

#include <signal.h>

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);

```

Argumenti:

- **how** — što se radi s argumentom set:
 - SIG_BLOCK — dodaj signale iz set trenutnoj maski,
 - SIG_UNBLOCK — ukloni signale iz set iz trenutne maske,
 - SIG_SETMASK — postavi masku točno na set.
- **set** — skup signala kojim mijenjamo masku; ako je NULL, maska se ne mijenja (poziv samo dohvaća staru u oldset).
- **oldset** — pokazivač u koji se upisuje **prethodna** maska, korisno za kasnije vraćanje stare vrijednosti; ako nas to ne zanima, predaje se NULL.

Argumente tipa sigset_t koriste i drugi pozivi koje smo već vidjeli (polje sa_mask u struct sigaction). To je apstraktni skup signala kojim ne baratamo izravno, nego pomoću pomoćnih funkcija:

```

int sigemptyset(sigset_t *set);           /* prazan skup */
int sigfillset(sigset_t *set);           /* svi signali */
int sigaddset(sigset_t *set, int signum); /* dodaj signal u skup */
int sigdelset(sigset_t *set, int signum); /* makni signal iz skupa */
int sigismember(const sigset_t *set, int signum); /* je li u skupu? */

```

Funkcija sigemptyset inicijalizira set kao prazan skup, na koji potom novim signalima dodajemo pojedinačno pomoću sigaddset. Ovaj pristup je prikladan kada treba sastaviti masku s relativno malim brojem signala — npr. samo SIGINT u našem maska.c primjeru.

Obrnuto, kada nam treba maska koja sadrži većinu signala, prirodnije je krenuti od potpunog skupa i ukloniti samo one koje ne želimo: `sigfillset` inicijalizira set kao skup koji sadrži sve signale, a `sigdelset` zatim uklanja pojedinačne signale po potrebi. Funkcija `sigismember` provjerava sadrži li set zadani signal — koristimo je npr. nakon poziva `sigpending` da bismo provjerili je li određeni signal u redu čekanja.

Postoji i poziv `sigpending(sigset_t *set)` koji u set upisuje signale koji su upravo **u redu čekanja** — stigli su procesu, ali su blokirani pa se još nisu isporučili. Korisno kad prije skidanja maske želimo provjeriti hoće li nešto biti isporučeno.

- **maska.c** — kratak primjer blokiranja SIGINT-a tijekom kratke “kritične sekcije”. Program registrira jednostavan rukovatelj koji ispiše poruku, blokira SIGINT, “radi” pet sekundi (`sleep`), pa skida masku.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

void int_handler(int signum) {
    printf("SIGINT obraden\n");
}

int main() {
    struct sigaction sa;
    sigset_t blok, prethodna;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = int_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    sigemptyset(&blok);
    sigaddset(&blok, SIGINT);

    sigprocmask(SIG_BLOCK, &blok, &prethodna);
    sleep(5);
    sigprocmask(SIG_SETMASK, &prethodna, NULL);

    return 0;
}
```

Pokrenimo program i tijekom njegovog rada iz druge ljuške pošaljimo SIGINT (npr. `kill -INT <pid>`). Iako signal stiže odmah, proces neće reagirati dok ne završi `sleep` — rukovatelj se izvršava tek nakon poziva `sigprocmask(SIG_SETMASK, &prethodna, NULL)` koji vraća masku na prethodnu vrijednost (bez SIGINT-a). Iz ovoga je jasno da je signal sve to vrijeme čekao u redu.

Bitan detalj: koliko god SIGINT-a pošaljemo tijekom blokade, rukovatelj će se izvršiti **samo jednom**. Razlog je u tome što jezgra za “klasične” UNIX signale ne vodi brojač pojavljivanja, samo zastavicu “u redu čekanja je / nije”. Više instanci istog signala koje stignu tijekom blokade spojavu se u jednu jedinu isporuku. (POSIX-realtime signali, brojevi 32–64, imaju pravi red s brojačem, ali to je tema za posebnu raspravu.)

Pažljivi čitatelj će uočiti suptilan problem s gornjim kodom. Redoslijed operacija je: prvo se pozivom `sigaction()` registrira rukovatelj, a tek zatim pozivom `sigprocmask()` blokira SIGINT. Što se događa ako SIGINT stigne u kratkom vremenskom prozoru između ova dva poziva? Signal će se isporučiti — rukovatelj će raditi svoj posao prije nego što program uopće uđe u “kritičnu sekciju”. U našem benignom primjeru posljedica je samo blago netočan tajming, ali u stvarnom

programu u kojem kritična sekcija mijenja zajedničke podatke, ovakav race condition može dovesti do korupcije podataka i neočekivanog ponašanja programa.

Možda biste pomislili da problem rješavamo postavljanjem SIGINT-a u polje `sa.sa_mask` strukture `struct sigaction`. Međutim, **sa_mask blokira signale samo dok se rukovatelj izvršava** — kad se rukovatelj vrati, blokada nestaje, pa to ne pomaže za zaštitu naše kritične sekcije.

Ostavljamo ovu situaciju kao **vježbu za čitatelja**: pokušajte preraditi `maska.c` tako da garantirano nijedan SIGINT ne može biti isporučen prije nego što program uđe u kritičnu sekciju, čak ni ako signal stigne u “neugodnom” trenutku. (Savjet: razmislite o redoslijedu poziva `sigaction()` i `sigprocmask()`.)

- **maska2.c** — funkcionalno drugačiji primjer, ali strukturom vrlo blizak prethodnom: umjesto da blokiramo SIGINT, ovdje ga **ignoriramo**. Rukovatelj nije potreban — kao “akciju” za signal postavljamo specijalnu vrijednost `SIG_IGN`:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

int main() {
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    sleep(5);
    printf("kraj programa - SIGINT-i su tijekom rada bili ignorirani\n");

    return 0;
}
```

Pokretanje izgleda slično `maska.c`-u — pet sekundi spavanja tijekom kojih program ne reagira na Ctrl+C, a na kraju ispiše završnu poruku koja nam jasno pokazuje da je program došao do kraja, tj. da SIGINT nije prekinuo izvršavanje. Kvalitativna razlika u odnosu na `maska.c` dolazi do izražaja kad usporedimo izlaze: `maska.c` će prije završne poruke iz `main`-a ispisati SIGINT obraden (ako smo poslali signal tijekom spavanja), dok `maska2.c` to neće ispisati nikad — signal je tiho odbačen u trenutku isporuke.

Razlika između blokiranja i ignoriranja, sažeto:

	Blokiranje (sigprocmask)	Ignoriranje (SIG_IGN)
Što se događa kada signal stigne	jezgra ga pohrani u red čekanja	jezgra ga odbaci
Kad se uvjet ukloni	signal se isporučuje, rukovatelj radi	ništa se ne događa, signal više ne postoji
Više instanci istog signala	spajaju se u jednu isporuku	sve odbačene
Tipičan use case	“ne sad, ali ne želim izgubiti signal”	“ovaj signal me zaista ne zanima”

Razlika postaje važna kad nas zanima da na signal ipak reagiramo, samo ne odmah. Klasičan primjer: tijekom kritične transakcije s bazom podataka, ne želimo prekid Ctrl+C-om, ali nakon završene transakcije želimo provjeriti je li korisnik tražio izlaz pa se uredno zatvoriti. Tu pomaže blokiranje; ignoriranje bi sve takve zahtjeve trajno izgubilo.

Pokupljanje djece — SIGCHLD

U poglavlju o procesima (P04) susreli smo se s pojmom **zombi procesa**: proces koji je završio izvršavanje, ali čiji zapis u tablici procesa jezgra još uvijek čuva, jer roditelj nije pozvao `wait()` da pokupi izlazni status. Tamo smo zaključili da je dobra programerska praksa za svako dijete obavezno pozvati `wait()`. Sada se postavlja pitanje: kako to napraviti elegantno ako roditelj u međuvremenu treba raditi nešto drugo, a ne samo blokirati u `wait()`-u?

Odgovor leži u signalu `SIGCHLD`. Svaki put kad proces dijete promijeni stanje (završi izvršavanje, bude zaustavljeno signalom, ili bude nastavljeno), jezgra šalje roditelju signal `SIGCHLD`. Po defaultu se ovaj signal ignorira — što je razlog zašto smo dosad mogli “zaboraviti” na njega. Međutim, ako registriramo rukovatelj za `SIGCHLD`, dobivamo elegantan obrazac u kojem roditelj pokuplja djecu **asinkrono**, kad god ona završe, dok glavni program nesmetano nastavlja sa svojim radom.

Ovo je jedan od najčešćih obrazaca u stvarnom UNIX programiranju — koriste ga UNIX ljuške, web serveri (npr. Apache, nginx za radne procese), baze podataka i mnogi drugi sustavi koji upravljaju većim brojem podređenih procesa.

- **nozombie.c** — primjer u kojem roditelj forka tri djeteta s različitim trajanjem, a sam u glavnoj petlji broji sekunde. Pokupljanje djece obavlja se u `SIGCHLD` rukovatelju, asinkrono u odnosu na glavnu petlju.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

void chld_handler(int signum) {
    int status;
    pid_t pid;

    pid = wait(&status);
    printf("[parent] dijete %d pokupljeno (status %d)\n",
           pid, WEXITSTATUS(status));
}

int main() {
    struct sigaction sa;
    int trajanje[] = {3, 1, 2};
    int i, sek;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = chld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa, NULL);

    for (i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("[child %d] PID %d, spavam %d s\n",
                   i+1, getpid(), trajanje[i]);
            sleep(trajanje[i]);
            return i+1;
        }
    }
}
```



```

    for (sek = 1; sek <= 5; sek++) {
        sleep(1);
        printf("[parent] sekunda %d\n", sek);
    }

    return 0;
}

```

Roditelj registrira rukovatelj `chld_handler` za signal `SIGCHLD`, a zatim u petlji forka tri djeteta. Svako dijete spava drukčiji broj sekundi (3, 1 ili 2) i pri završetku vraća svoj redni broj kao izlazni status. Glavna petlja roditelja jednostavno broji sekunde — pet puta odspava sekundu i ispiše broj. Dok roditelj broji, djeca jedno po jedno završavaju; svaki put kad proces dijete završi, jezgra roditelju isporuči `SIGCHLD`, rukovatelj se izvrši i pozove `wait(&status)` da pokupi gotovo dijete.

Pokrenimo program:

```

$ ./nozombie
[child 2] PID 40, spavam 1 s
[child 3] PID 41, spavam 2 s
[child 1] PID 39, spavam 3 s
[parent] sekunda 1
[parent] dijete 40 pokupljeno (status 2)
[parent] sekunda 2
[parent] dijete 41 pokupljeno (status 3)
[parent] sekunda 3
[parent] dijete 39 pokupljeno (status 1)
[parent] sekunda 4
[parent] sekunda 5

```

Iz ispisa se vidi vremenski tijek: nakon prve sekunde završava dijete 2 (spavalo je 1 sekundu), pa rukovatelj odmah javlja da je pokupljeno. Nakon druge sekunde isto se dogodi za dijete 3, a nakon treće za dijete 1. Glavni program ovo ne primjećuje — uredno nastavlja brojati sekunde do pet.

Zašto SA_RESTART? Glavni program u svojoj petlji koristi `sleep(1)`. Kad `SIGCHLD` stigne tijekom `sleep`-a, jezgra prekida sistemski poziv i poziva rukovatelj. Po defaultu, kad se rukovatelj vrati, prekinuti sistemski poziv vraća grešku s `errno = EINTR`. Za `sleep` to znači: vraća se prije isteka tražene sekunde — brojač sekundi bio bi neispravan. Postavljanjem zastavice `SA_RESTART` u `sa_flags` jezgri kažemo: *“nakon obrade signala automatski nastavi prekinuti sistemski poziv”*. Tako naš `sleep` spava punu sekundu čak i ako tijekom toga stigne `SIGCHLD`.

Napomena o printf-u u rukovatelju. Pažljivi čitatelj zapazit će da naš `chld_handler` poziva `printf` — što je u suprotnosti s preporukom koju smo uveli na samom početku poglavlja: u rukovatelju treba pozivati samo `async-signal-safe` funkcije, a `printf` to nije. U ovom primjeru smo se ipak odlučili za `printf` radi jasnoće — svrha je pokazati u kojem se trenutku rukovatelj zaista izvrši i koje dijete pokuplja, što ispis čini ključnim za razumijevanje primjera. U produkcijskom kodu ovo nije prihvatljivo: rukovatelj bi tipično samo postavio zastavicu ili upisao podatke u red iz kojeg ih glavni program kasnije izvuče i ispiše. Imajte to na umu kad ovaj uzorak budete prilagođavali za vlastite programe.

Pokupljanje više djece odjednom. U ovom primjeru djeca završavaju jedno po jedno, s razmakom od jedne sekunde, pa svaka instanca `SIGCHLD`-a dovodi do pokupljanja točno jednog djeteta. Međutim, ako bi više djece završilo gotovo istovremeno, mogla bi se dogoditi situacija u kojoj je rukovatelj pozvan jednom, a u međuvremenu su dvije ili više djece spremne za pokupljanje. Razlog leži u tome što su signali “klasične” UNIX vrste — više instanci istog signala koje stignu blizu jedne drugoj spojavu se u jednu isporuku (kao što smo vidjeli kod blokiranja). U produkcijskom kodu rukovatelj zato obično poziva `waitpid(-1, &status, WNOHANG)` u petlji, sve dok funkcija ne vrati 0 ili

–1, čime se garantira da su sva spremna djeca pokupljena. U ovom uvodnom primjeru nismo se bavili tom mogućnošću jer nam vremenski razmak između djece to ne nalaže.

Glavna pouka. Roditelj nije ni jednom eksplicitno pozvao `wait()` u svojoj glavnoj petlji — sva djeca su uredno pokupljena. Petlja roditelja bavi se isključivo svojim “korisnim” poslom (brojanjem sekundi), a sustav za pokupljanje radi neovisno, kroz signale. Time se izbjegava i potreba za stalnim provjerama “je li završilo neko dijete”, i opasnost da pokupljanje propustimo (što bi dovelo do gomilanja zombija).

Prevođenje

Direktorij dolazi s priloženim **Makefile**-om koji prati iste konvencije kao i Makefile datoteke u prethodnim poglavljima (varijable `CC`, `CFLAGS`, `LDFLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`). Detaljan opis strukture i korake gradnje Makefilea vidjeti u [../P02-Osnove_programiranja/README.md](#).

Tipična uporaba:

```
make          # gradi zadani cilj (potvrdi)
make all      # gradi sve primjere
make stoperica # gradi samo zadani primjer
make clean    # briše izvršne i objektne datoteke
```